

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống quản lý và đặt vé sự kiện

PHẠM HỒNG DƯƠNG

duong.ph225824@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: ThS. Lê Bá Vui

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống quản lý và đặt vé sự kiện

PHẠM HỒNG DƯƠNG

duong.ph225824@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: ThS. Lê Bá Vui _____

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Lời đầu tiên em muốn gửi đến ba mẹ của em, những người luôn ở bên trong suốt chặng đường đến lúc em chuẩn bị bước sang một trang mới trong cuộc đời hết mực yêu thương và giúp đỡ em. Bố mẹ lúc nào cũng là chỗ dựa vững chắc nhất để em có thể yên tâm học tập và làm việc tại đại học, em biết rằng dù có đi đến đâu, đạt được những đỉnh cao nào thì em luôn nhớ đến cha mẹ đầu tiên. Em cũng muốn gửi lời cảm ơn này đến anh chị bởi chặng đường suốt 4 năm đại học khi xa mái nhà, xa gia đình thì vẫn có anh chị ở bên chăm sóc, gửi những lời động viên mỗi khi vấp ngã. Từ năm ba cho đến thời điểm ra trường, thầy Vui luôn tận tình hướng dẫn chỉ bảo, đưa ra những hạn chế để em khắc phục, đồng hành cùng thầy từ Nghiên cứu đề án 1 cho đến khi Đề án tốt nghiệp, em cảm thấy rất biết ơn khi được thầy trau dồi không chỉ bằng khía cạnh chuyên môn cũng như mang dấu ấn tinh thần của một người cha. Lời cuối cùng em cũng muốn gửi đến các bạn của em, những người suốt ngày nhắc nhở deadline hay phần đầu nỗ lực cùng em trong suốt 4 năm đại học.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh nhu cầu tham gia các sự kiện giải trí, học thuật và cộng đồng ngày càng tăng, việc tìm kiếm thông tin, đặt vé và quản lý quá trình tham dự trên thiết bị di động vẫn còn nhiều bất cập. Nhiều hệ thống hiện nay chỉ giải quyết từng phần của bài toán, chẳng hạn tập trung vào hiển thị sự kiện hoặc thanh toán, nhưng chưa liên kết chặt chẽ giữa khám phá sự kiện, quản lý vé, giao tiếp với ban tổ chức và nhắc lịch. Từ thực tế đó, đồ án lựa chọn hướng tiếp cận phát triển một hệ thống đặt vé sự kiện theo mô hình mobile-first, kết hợp ứng dụng di động và backend tập trung. Hướng tiếp cận này phù hợp với thói quen sử dụng điện thoại thông minh và cho phép tích hợp đồng bộ các chức năng nghiệp vụ quan trọng.

Giải pháp được xây dựng gồm ứng dụng frontend phát triển bằng React Native và Expo, cùng backend sử dụng Node.js, Express, TypeScript và MongoDB. Hệ thống hỗ trợ đăng ký, đăng nhập, tìm kiếm và lọc sự kiện, xem chi tiết, lưu sự kiện yêu thích, đặt vé theo từng hạng vé, xác nhận thanh toán, sinh mã QR cho vé, check-in tại sự kiện, trao đổi qua chat, nhận thông báo và nhắc lịch tự động trước thời gian diễn ra sự kiện. Hệ thống cũng hỗ trợ phân quyền người dùng, người tổ chức và quản trị viên, đồng thời tích hợp chức năng gợi ý nội dung sự kiện bằng AI. Đóng góp chính của đồ án là xây dựng được một hệ thống đặt vé sự kiện tương đối hoàn chỉnh theo hướng ứng dụng thực tế, kết nối nhiều chức năng trên cùng một nền tảng. Kết quả đạt được là một ứng dụng có thể đáp ứng các luồng nghiệp vụ cốt lõi của bài toán đặt vé và quản lý sự kiện trên thiết bị di động.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Khảo sát hiện trạng	5
2.1.1 Đặc thù của bài toán đặt vé và quản lý sự kiện	5
2.1.2 Khảo sát một số ứng dụng tương tự.....	6
2.1.3 Nhận xét và các chức năng cần ưu tiên phát triển	7
2.2 Tổng quan chức năng	7
2.2.1 Biểu đồ use case tổng quát	8
2.2.2 Biểu đồ use case phân rã theo nhóm các chức năng	9
2.2.3 Quy trình nghiệp vụ	11
2.3 Đặc tả chức năng	12
2.3.1 Đặc tả use case Tạo sự kiện.....	12
2.3.2 Đặc tả use case Đặt vé và xác nhận thanh toán	12
2.3.3 Đặc tả use case Gửi lời mời và thông báo	13
2.3.4 Đặc tả use case Check-in vé bằng mã QR	13
2.4 Yêu cầu phi chức năng	14
2.4.1 Yêu cầu về hiệu năng.....	14
2.4.2 Yêu cầu về độ tin cậy và tính nhất quán dữ liệu	14
2.4.3 Yêu cầu về bảo mật	15
2.4.4 Yêu cầu về khả năng sử dụng.....	15

2.4.5 Yêu cầu về khả năng bảo trì và mở rộng	15
2.4.6 Yêu cầu kỹ thuật triển khai.....	15
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	16
3.1 Kiến trúc ứng dụng di động kết hợp dịch vụ backend	16
3.2 React Native và Expo cho lớp ứng dụng khách	16
3.3 Node.js và Express cho lớp dịch vụ nghiệp vụ	17
3.4 MongoDB và Mongoose cho lưu trữ dữ liệu.....	17
3.5 JWT cho xác thực và phân quyền	18
3.6 Socket.IO cho giao tiếp thời gian thực	18
3.7 Thông báo đẩy và nhắc lịch sự kiện	18
3.8 Mã QR trong quản lý vé điện tử	19
3.9 Hỗ trợ sinh nội dung sự kiện bằng AI	19
3.10 Kết chương.....	20
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	21
4.1 Thiết kế kiến trúc.....	21
4.1.1 Lựa chọn kiến trúc phần mềm	21
4.1.2 Thiết kế tổng quan.....	23
4.1.3 Thiết kế chi tiết gói	25
4.2 Thiết kế chi tiết.....	27
4.2.1 Thiết kế giao diện	27
4.2.2 Thiết kế lớp	31
4.2.3 Thiết kế cơ sở dữ liệu	34
4.3 Xây dựng ứng dụng.....	37
4.3.1 Thư viện và công cụ sử dụng.....	37
4.3.2 Kết quả đạt được	39

4.3.3 Minh họa các chức năng chính	43
4.4 Kiểm thử.....	45
4.4.1 Mục tiêu và phạm vi kiểm thử	46
4.4.2 Kỹ thuật kiểm thử và môi trường kiểm thử.....	46
4.4.3 Thiết kế các trường hợp kiểm thử chính.....	47
4.4.4 Tổng kết kết quả kiểm thử	50
4.5 Triển khai	51
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	55
5.1 Giải pháp thích ứng đa nền tảng cho các chức năng phụ thuộc thiết bị	55
5.1.1 Bài toán và bối cảnh	55
5.1.2 Giải pháp	56
5.1.3 Kết quả đạt được	57
5.2 Giải pháp bảo toàn tính nhất quán cho luồng đặt vé nhiều hạng kết hợp thanh toán và check-in	57
5.2.1 Bài toán và bối cảnh	57
5.2.2 Giải pháp	58
5.2.3 Kết quả đạt được	59
5.3 Giải pháp xây dựng cơ chế nhắc lịch và thông báo hợp nhất, hạn chế gửi trùng	59
5.3.1 Bài toán và bối cảnh	59
5.3.2 Giải pháp	60
5.3.3 Kết quả đạt được	60
5.4 Giải pháp tích hợp AI hỗ trợ tạo nội dung sự kiện với cơ chế dự phòng nhiều tầng	61
5.4.1 Bài toán và bối cảnh	61
5.4.2 Giải pháp	61
5.4.3 Kết quả đạt được	62

5.5 Nhận xét chung về các đóng góp nổi bật	62
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	64
6.1 Kết luận	64
6.2 Hướng phát triển.....	65
TÀI LIỆU THAM KHẢO.....	67

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống event-booking-app . . .	8
Hình 2.2	Biểu đồ phân rã theo nhóm các use case của hệ thống	10
Hình 2.3	Biểu đồ hoạt động của quy trình nghiệp vụ tham gia sự kiện . .	11
Hình 4.1	Biểu đồ gói tổng quan của hệ thống event-booking-app	24
Hình 4.2	Biểu đồ thiết kế gói cho nghiệp vụ đặt vé, thanh toán và tham dự sự kiện	26
Hình 4.3	Wireframe minh họa giao diện chức năng tạo sự kiện	30
Hình 4.4	Wireframe minh họa giao diện chức năng đặt vé	30
Hình 4.5	Biểu đồ trình tự cho use case Tạo sự kiện	32
Hình 4.6	Biểu đồ trình tự cho use case Đặt vé và xác nhận thanh toán . .	33
Hình 4.7	Biểu đồ trình tự cho use case Check-in vé bằng mã QR	33
Hình 4.8	Biểu đồ thực thể liên kết của hệ thống event-booking-app . . .	37
Hình 4.9	Giao diện chức năng tạo sự kiện	43
Hình 4.10	Giao diện chức năng đặt vé	44
Hình 4.11	Giao diện chức năng check-in vé	45
Hình 4.12	Mô hình triển khai thử nghiệm của hệ thống event-booking-app	52

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh một số ứng dụng tương tự với định hướng của giải pháp đề xuất	6
Bảng 4.1	Danh sách công cụ và thư viện sử dụng ở phía frontend của hệ thống	38
Bảng 4.2	Danh sách công cụ, thư viện và môi trường phát triển ở phía backend của hệ thống	39
Bảng 4.3	Các sản phẩm đầu ra chính đạt được trong quá trình thực hiện đồ án	41
Bảng 4.4	Một số thông tin thống kê về quy mô và kết quả hiện thực của hệ thống	42
Bảng 4.5	Các trường hợp kiểm thử chính cho chức năng xác thực người dùng	48
Bảng 4.6	Các trường hợp kiểm thử chính cho chức năng tạo sự kiện . . .	49
Bảng 4.7	Các trường hợp kiểm thử chính cho chức năng đặt vé và xác nhận thanh toán	50

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AI	Trí tuệ nhân tạo (Artificial Intelligence)
API	Giao diện lập trình ứng dụng (Application Programming Interface)
CDN	Mạng phân phối nội dung (Content Delivery Network)
CORS	Cơ chế chia sẻ tài nguyên khác nguồn (Cross-Origin Resource Sharing)
ERD	Biểu đồ thực thể liên kết (Entity Relationship Diagram)
HTTP	Giao thức truyền tải siêu văn bản (HyperText Transfer Protocol)
HTTPS	Giao thức truyền tải siêu văn bản bảo mật (HyperText Transfer Protocol Secure)
JSON	Định dạng trao đổi dữ liệu JavaScript Object Notation
JWT	Mã thông báo web dạng JSON (JSON Web Token)
MVC	Mô hình Model – View – Controller
NoSQL	Nhóm cơ sở dữ liệu phi quan hệ (Not Only SQL)
PaaS	Nền tảng như một dịch vụ (Platform as a Service)
QR	Mã phản hồi nhanh (Quick Response)
REST	Phong cách kiến trúc chuyển giao trạng thái đại diện (Representational State Transfer)
SQL	Ngôn ngữ truy vấn có cấu trúc (Structured Query Language)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Sự phát triển mạnh của Internet di động và các nền tảng mạng xã hội đã làm thay đổi rõ rệt cách người dùng tiếp cận thông tin, tương tác cộng đồng và tham gia các hoạt động trực tuyến lẫn ngoại tuyến. Theo báo cáo Digital 2026 của DataReportal, số lượng danh tính người dùng mạng xã hội trên toàn cầu đã vượt mốc 5 tỷ, trong khi Việt Nam tiếp tục duy trì tỷ lệ người dùng Internet và mạng xã hội ở mức cao so với quy mô dân số [1], [2]. Trong bối cảnh đó, nhu cầu tìm kiếm, theo dõi và tham gia các sự kiện như hội thảo, workshop, chương trình giải trí, hoạt động cộng đồng hay các buổi gặp gỡ chuyên môn ngày càng gia tăng. Sự kiện không còn được quảng bá qua một kênh đơn lẻ mà thường xuất hiện đồng thời trên mạng xã hội, các hội nhóm trực tuyến, ứng dụng di động và các trang chuyên biệt về tổ chức sự kiện.

Tuy nhiên, chính sự phân tán thông tin này lại làm phát sinh một bài toán thực tiễn đáng chú ý. Người dùng thường gặp khó khăn khi phải theo dõi nhiều nguồn khác nhau để tìm sự kiện phù hợp với sở thích, vị trí, thời gian và ngân sách của mình. Sau khi tìm được sự kiện, quá trình đăng ký tham gia, thanh toán, lưu vé, nhận thông báo nhắc lịch và trao đổi với ban tổ chức nhiều khi vẫn diễn ra rời rạc trên các công cụ khác nhau. Đối với người tổ chức, việc quản lý thông tin sự kiện, thiết lập nhiều hạng vé, theo dõi số lượng người tham gia, gửi thông báo và kiểm soát check-in cũng đòi hỏi nhiều thao tác thủ công nếu chưa có một nền tảng tích hợp thống nhất. Thực tế này cho thấy bài toán đặt vé và quản lý sự kiện trên thiết bị di động không chỉ là bài toán giao diện hiển thị thông tin, mà còn là bài toán kết nối các quy trình nghiệp vụ liên quan đến khám phá sự kiện, giao tiếp, giao dịch và kiểm soát tham dự.

Tầm quan trọng của bài toán này còn được thể hiện qua sự xuất hiện của nhiều nền tảng và nghiên cứu liên quan tới gợi ý, tổ chức và quản lý sự kiện xã hội. Meetup là một ví dụ tiêu biểu cho xu hướng kết nối cộng đồng thông qua các sự kiện theo nhóm, cho thấy nhu cầu tham gia các hoạt động dựa trên sở thích chung là rất lớn [3]. Trong nghiên cứu học thuật, nhiều công trình đã tiếp cận bài toán từ góc độ gợi ý sự kiện, tối ưu lựa chọn sự kiện hoặc lựa chọn địa điểm phù hợp cho nhóm người dùng [4]–[6]. Dù vậy, các hướng tiếp cận này chủ yếu tập trung vào một số khía cạnh chuyên biệt như đề xuất sự kiện hoặc địa điểm, trong khi nhu cầu thực tế của người dùng cuối lại đòi hỏi một hệ thống có thể hỗ trợ trọn vẹn hơn từ khâu khám phá đến khâu tham gia sự kiện. Nếu giải quyết tốt bài toán này, hệ

thông không chỉ mang lại lợi ích cho người tham dự trong việc tiết kiệm thời gian, tăng trải nghiệm và giảm rủi ro bỏ lỡ thông tin, mà còn hỗ trợ người tổ chức nâng cao hiệu quả vận hành và khả năng tiếp cận cộng đồng mục tiêu.

1.2 Mục tiêu và phạm vi đề tài

Hiện nay, bài toán sự kiện số đang được giải quyết theo hai hướng chính. Hướng thứ nhất là các nền tảng ứng dụng thực tế, tiêu biểu như Meetup hoặc các hệ thống quản lý sự kiện trực tuyến, tập trung vào khả năng công bố sự kiện, kết nối cộng đồng và hỗ trợ người dùng đăng ký tham gia [3]. Hướng thứ hai là các nghiên cứu học thuật về gợi ý sự kiện, phân tích hành vi người dùng và tối ưu quyết định tham gia sự kiện hoặc lựa chọn địa điểm tổ chức phù hợp [4]–[6]. Các công trình này cho thấy bài toán sự kiện có thể được tiếp cận dưới nhiều góc độ như khai thác dữ liệu xã hội, cá nhân hóa nội dung, hỗ trợ nhóm người dùng hoặc nâng cao hiệu quả ra quyết định.

Mặc dù vậy, các nền tảng phổ biến trên thị trường thường chỉ mạnh ở một số chức năng riêng lẻ như quảng bá sự kiện, bán vé hoặc kết nối cộng đồng, trong khi nhiều nghiên cứu học thuật lại tập trung vào mô hình đề xuất mà chưa đi tới một sản phẩm tích hợp đầy đủ các luồng nghiệp vụ của bài toán. Từ góc nhìn người dùng cuối, những hạn chế thường gặp là thông tin sự kiện phân tán, việc quản lý vé và nhắc lịch chưa thật sự đồng bộ, tương tác giữa người tham gia và ban tổ chức còn rời rạc, và quy trình xác nhận tham dự tại sự kiện chưa được số hóa trọn vẹn. Từ góc nhìn người tổ chức, việc quản lý các hạng vé, số lượng người tham gia, kiểm soát check-in và gửi thông báo vẫn có thể gây tốn thời gian nếu thiếu một hệ thống tập trung.

Trên cơ sở đó, mục tiêu của đề tài là xây dựng một ứng dụng đặt vé và quản lý sự kiện trên thiết bị di động, hướng tới việc hỗ trợ đầy đủ các nhu cầu cốt lõi của người tham dự và người tổ chức trong cùng một nền tảng. Cụ thể, hệ thống được định hướng phát triển các chức năng chính gồm đăng ký và xác thực người dùng, tìm kiếm và lọc sự kiện, xem chi tiết sự kiện, tạo và quản lý sự kiện, quản lý nhiều hạng vé, đặt vé và xác nhận thanh toán, sinh mã QR cho vé điện tử, check-in tại sự kiện, gửi thông báo, nhắc lịch, lưu sự kiện yêu thích và hỗ trợ trao đổi qua chat. Phạm vi của đề tài tập trung vào xây dựng phiên bản ứng dụng thử nghiệm có khả năng vận hành các luồng nghiệp vụ chính của bài toán, chưa đặt trọng tâm vào triển khai thương mại quy mô lớn hoặc tối ưu hạ tầng phục vụ lượng truy cập cực lớn.

1.3 Định hướng giải pháp

Từ các mục tiêu nêu ở phần 1.2, đề tài lựa chọn định hướng phát triển hệ thống theo mô hình ứng dụng di động kết hợp dịch vụ backend tập trung. Theo định

hướng này, phần mềm được xây dựng như một nền tảng mobile-first, trong đó ứng dụng di động đóng vai trò giao tiếp trực tiếp với người dùng, còn backend đảm nhiệm các nghiệp vụ xử lý dữ liệu, xác thực, quản lý sự kiện, quản lý vé, thông báo và trao đổi thời gian thực. Định hướng này được lựa chọn vì phù hợp với đặc trưng sử dụng thường xuyên trên điện thoại thông minh, đồng thời thuận lợi cho việc mở rộng tính năng, đồng bộ dữ liệu và tích hợp các dịch vụ hỗ trợ trong tương lai.

Theo hướng tiếp cận đã chọn, giải pháp của đề tài là xây dựng một hệ thống gồm ứng dụng khách trên nền tảng React Native và một backend sử dụng Node.js, Express, TypeScript và MongoDB để quản lý dữ liệu tập trung. Hệ thống cho phép người dùng khám phá sự kiện theo danh mục và từ khóa, lưu sự kiện quan tâm, đặt vé theo hạng vé cụ thể, nhận vé điện tử có mã QR, nhận thông báo và nhắc lịch trước khi sự kiện diễn ra. Ở chiều ngược lại, người tổ chức có thể tạo sự kiện, cấu hình nhiều mức vé, theo dõi tình trạng tham gia và thực hiện kiểm soát check-in. Bên cạnh đó, hệ thống còn hỗ trợ giao tiếp qua chat và tích hợp chức năng gợi ý nội dung sự kiện bằng AI nhằm hỗ trợ người tổ chức khi tạo mới sự kiện.

Đóng góp chính của đồ án là đề xuất và hiện thực hóa được một mô hình ứng dụng đặt vé sự kiện theo hướng tích hợp nhiều chức năng nghiệp vụ trên cùng một nền tảng di động, thay vì tách rời thành nhiều công cụ độc lập. Kết quả mong muốn của đề tài là xây dựng được một sản phẩm phần mềm thử nghiệm có khả năng đáp ứng các luồng nghiệp vụ cốt lõi của bài toán đặt vé và quản lý sự kiện, làm cơ sở cho việc đánh giá, cải tiến và mở rộng trong các giai đoạn phát triển tiếp theo.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức thành năm chương. Chương 2 trình bày nội dung khảo sát và phân tích yêu cầu của bài toán đặt vé, quản lý và tham dự sự kiện trên thiết bị di động. Trên cơ sở khảo sát một số ứng dụng và nền tảng tương tự, chương này làm rõ hiện trạng của bài toán, xây dựng biểu đồ use case tổng quát, use case phân rã, quy trình nghiệp vụ, đặc tả một số use case quan trọng và xác định các yêu cầu phi chức năng mà hệ thống cần đáp ứng.

Chương 3 giới thiệu những nền tảng lý thuyết và công nghệ được sử dụng để phát triển hệ thống. Nội dung chương tập trung làm rõ cơ sở kỹ thuật của mô hình client-server theo hướng mobile-first, các công nghệ được lựa chọn cho frontend, backend, cơ sở dữ liệu, xác thực, giao tiếp thời gian thực, thông báo đẩy, mã mã phản hồi nhanh (Quick Response - QR) và hỗ trợ AI, từ đó tạo nền tảng cho các quyết định thiết kế và hiện thực ở các chương tiếp theo.

Chương 4 trình bày quá trình phân tích thiết kế, triển khai và đánh giá hệ thống event-booking-app. Trong chương này, đồ án mô tả kiến trúc phần mềm đã chọn,

biểu đồ gói tổng quan và chi tiết, thiết kế giao diện, thiết kế lớp, thiết kế cơ sở dữ liệu, các công cụ và thư viện sử dụng, kết quả hiện thực, một số màn hình chức năng tiêu biểu, hoạt động kiểm thử và mô hình triển khai thử nghiệm của hệ thống trong môi trường thực tế.

Chương 5 tập trung vào các giải pháp và đóng góp nổi bật nhất của đề án. Thay vì lặp lại phần mô tả thiết kế tổng quát, chương này đi sâu phân tích những vấn đề kỹ thuật quan trọng mà hệ thống đã giải quyết, bao gồm giải pháp thích ứng đa nền tảng giữa mobile và web, giải pháp bảo đảm tính nhất quán cho luồng đặt vé kết hợp thanh toán và check-in, cơ chế nhắc lịch và thông báo hợp nhất hạn chế gửi trùng, cùng với mô-đun AI hỗ trợ tạo nội dung sự kiện có cơ chế dự phòng nhiều tầng.

Cuối cùng, Chương 6 đưa ra kết luận chung của đề án và định hướng phát triển trong tương lai. Trên cơ sở những kết quả đã đạt được, chương này tổng hợp lại mức độ hoàn thành mục tiêu của đề tài, nêu các hạn chế còn tồn tại và đề xuất những hướng mở rộng tiếp theo nhằm tiếp tục hoàn thiện hệ thống cả về chức năng, trải nghiệm người dùng lẫn khả năng triển khai thực tế.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này tập trung khảo sát hiện trạng của bài toán đặt vé và quản lý sự kiện trên thiết bị di động, từ đó làm rõ các yêu cầu cần thiết cho hệ thống được phát triển trong đồ án. Trên cơ sở phân tích các nền tảng tương tự và đối chiếu với đặc điểm nghiệp vụ của bài toán, chương này xác định những chức năng cốt lõi và các ràng buộc phi chức năng mà hệ thống cần đáp ứng.

2.1 Khảo sát hiện trạng

Trong phạm vi đồ án này, khảo sát hiện trạng được thực hiện chủ yếu dựa trên hai nguồn chính là phân tích các ứng dụng tương tự đang tồn tại và đối chiếu với đặc điểm nghiệp vụ của hệ thống *event-booking-app* đang được phát triển. Cách tiếp cận này phù hợp với bối cảnh của một đồ án ứng dụng, khi mục tiêu không chỉ là nhận diện nhu cầu của người dùng cuối mà còn là xác định rõ các khoảng trống chức năng mà hệ thống đề xuất cần giải quyết. Qua việc xem xét luồng nghiệp vụ hiện tại của hệ thống, có thể thấy sản phẩm hướng tới ba nhóm người dùng chính là người tham dự sự kiện, người tổ chức và quản trị viên. Do đó, hiện trạng cần được khảo sát không chỉ ở khía cạnh mua vé, mà còn ở khía cạnh khám phá sự kiện, quản lý thông tin, tương tác giữa các bên và kiểm soát việc tham dự.

2.1.1 Đặc thù của bài toán đặt vé và quản lý sự kiện

Đối với người tham dự, nhu cầu cơ bản không dừng lại ở việc tìm thấy một sự kiện phù hợp, mà còn bao gồm khả năng theo dõi sự kiện yêu thích, nhận thông báo đúng thời điểm, đặt vé nhanh chóng, lưu trữ vé điện tử thuận tiện và được hỗ trợ trong quá trình tham gia sự kiện. Đối với người tổ chức, bài toán lại mở rộng thêm các nhu cầu về tạo sự kiện, cấu hình nhiều hạng vé, quản lý số lượng người tham gia, gửi thông báo và xác nhận check-in tại chỗ. Nếu các khâu này bị tách rời trên nhiều nền tảng khác nhau, trải nghiệm tổng thể sẽ trở nên rời rạc, khó theo dõi và làm tăng chi phí thao tác cho cả người dùng lẫn người tổ chức.

Từ góc nhìn đó, một hệ thống đặt vé sự kiện hiện đại cần thỏa mãn đồng thời ba nhóm chức năng. Nhóm thứ nhất là chức năng khám phá và theo dõi sự kiện, bao gồm tìm kiếm, lọc, xem chi tiết, lưu sự kiện và nhận nhắc lịch. Nhóm thứ hai là chức năng giao dịch và kiểm soát tham dự, bao gồm đặt vé, xác nhận thanh toán, tạo vé điện tử và check-in bằng mã số hoặc mã QR. Nhóm thứ ba là chức năng tương tác và hỗ trợ cộng đồng, bao gồm thông báo, lời mời, nhắn tin và các cơ chế kết nối giữa người tham dự với người tổ chức. Đây cũng chính là cơ sở để đánh giá mức độ phù hợp của các ứng dụng tương tự trong phần tiếp theo.

2.1.2 Khảo sát một số ứng dụng tương tự

Trong số các nền tảng hiện có, Meetup và Ticketmaster là hai trường hợp tiêu biểu nhưng đại diện cho hai định hướng khác nhau của bài toán sự kiện. Meetup nhấn mạnh vào việc hình thành cộng đồng, tìm kiếm nhóm theo sở thích và tham gia các hoạt động địa phương hoặc trực tuyến [3]. Ngược lại, Ticketmaster tập trung mạnh vào khâu khám phá sự kiện quy mô lớn, mua vé điện tử, quản lý vé trên di động, chuyển vé và bán lại vé trong hệ sinh thái của mình. Việc khảo sát hai nền tảng này giúp nhận diện rõ sự khác biệt giữa hướng tiếp cận thiên về cộng đồng và hướng tiếp cận thiên về thương mại hóa vé điện tử.

Nền tảng	Điểm mạnh nổi bật	Hạn chế so với bài toán của đề án	Giá trị tham khảo cho hệ thống
Meetup [3]	Mạnh về khám phá cộng đồng, nhóm sở thích, sự kiện địa phương và kết nối giữa những người có cùng mối quan tâm.	Chức năng đặt vé, quản lý nhiều hạng vé, xác nhận thanh toán và kiểm soát check-in không phải trọng tâm chính; chưa nhấn mạnh mạnh vào quy trình vé điện tử khép kín.	Tham khảo cách tổ chức sự kiện theo cộng đồng, gọi mở nhu cầu kết hợp giữa sự kiện và tương tác xã hội trong ứng dụng.
Ticketmaster	Mạnh về tìm kiếm sự kiện, vé điện tử, quản lý vé trên điện thoại, chuyển vé, bán vé và hỗ trợ người dùng trong ngày diễn ra sự kiện.	Thiên về các sự kiện thương mại quy mô lớn; ít nhấn mạnh chiều tương tác cộng đồng, trò chuyện trực tiếp hoặc nhu cầu tổ chức sự kiện linh hoạt cho nhóm nhỏ.	Tham khảo luồng mua vé, quản lý vé điện tử, truy cập vé trên di động và cơ chế kiểm soát vé thuận tiện.
Giải pháp đề xuất	Hướng tới tích hợp khám phá sự kiện, tạo và quản lý sự kiện, nhiều hạng vé, vé điện tử, thông báo, lời mời, chat và check-in trong cùng một nền tảng di động.	Quy mô triển khai hiện tại còn ở mức thử nghiệm, chưa tối ưu cho lượng người dùng cực lớn hoặc hệ sinh thái phân phối vé quy mô thương mại.	Kết hợp ưu điểm của hướng cộng đồng và hướng quản lý vé điện tử, phù hợp với phạm vi ứng dụng thực tế cỡ vừa.

Bảng 2.1: So sánh một số ứng dụng tương tự với định hướng của giải pháp đề xuất

Từ bảng so sánh trên có thể thấy, Meetup có ưu thế rõ rệt ở khía cạnh xây dựng

cộng đồng và tạo động lực tham gia sự kiện thông qua sở thích chung. Đây là đặc điểm quan trọng vì bài toán sự kiện trong thực tế không chỉ liên quan đến giao dịch mua vé mà còn liên quan tới trải nghiệm kết nối, tương tác và duy trì sự quan tâm của người dùng đối với các hoạt động trong tương lai. Tuy nhiên, nếu chỉ dựa vào hướng tiếp cận như Meetup, hệ thống sẽ thiếu đi những chức năng thiết yếu cho bài toán đặt vé như quản lý nhiều loại vé, xử lý trạng thái thanh toán hay hỗ trợ kiểm soát tham dự bằng vé điện tử.

Ở chiều ngược lại, Ticketmaster thể hiện rõ ưu thế trong quy trình vé điện tử và quản lý việc tham dự trên di động. Việc cho phép người dùng lưu, truy cập, chuyển hoặc sử dụng vé điện tử trực tiếp trên điện thoại là một điểm tham khảo quan trọng đối với Giải pháp đề xuất. Tuy nhiên, mô hình này thiên nhiều hơn về phân phối vé cho các sự kiện lớn, trong khi bài toán của đề tài cần linh hoạt hơn để phục vụ cả các hoạt động cộng đồng, workshop, sự kiện học thuật và các chương trình có quy mô vừa và nhỏ. Điều đó dẫn tới yêu cầu hệ thống không chỉ phải hỗ trợ đặt vé, mà còn phải duy trì khả năng kết nối giữa người dùng và ban tổ chức qua thông báo, lời mời và trao đổi trực tiếp.

2.1.3 Nhận xét và các chức năng cần ưu tiên phát triển

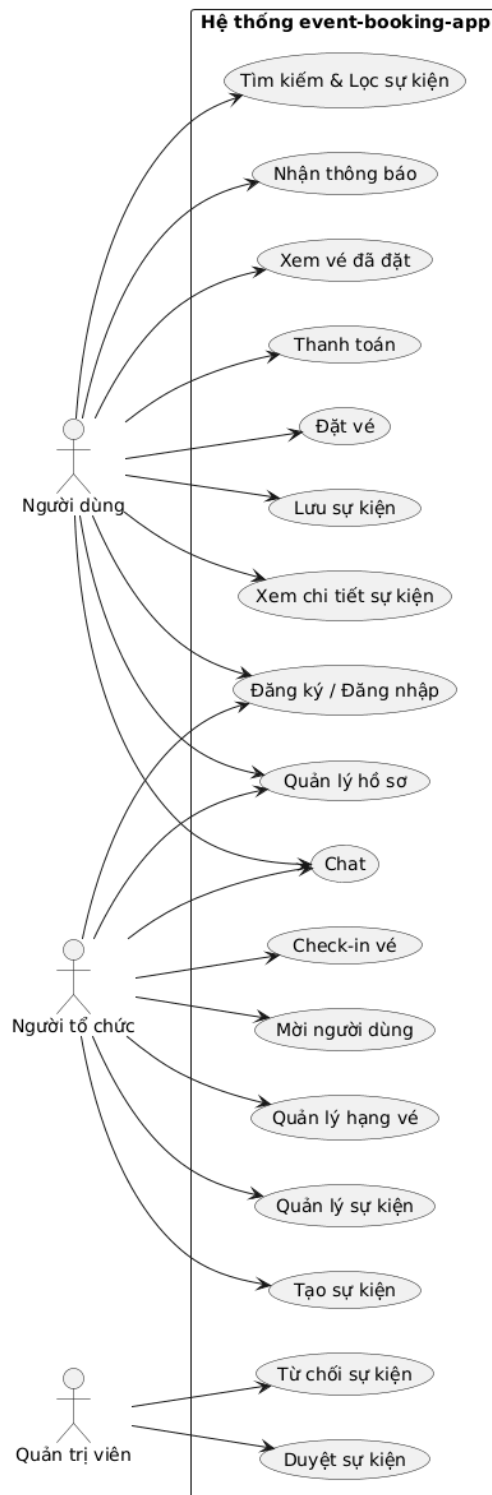
Từ khảo sát hiện trạng, có thể rút ra rằng không có một ứng dụng đơn lẻ nào trong số các nền tảng tham khảo đồng thời bao phủ cân bằng cả ba chiều: khám phá sự kiện, quản lý vé điện tử và tương tác cộng đồng theo phạm vi mà đề tài đặt ra. Vì vậy, Giải pháp đề xuất cần được định hướng như một nền tảng tích hợp, vừa kế thừa khả năng tổ chức và kết nối cộng đồng, vừa hỗ trợ quy trình mua vé và tham dự sự kiện trên thiết bị di động. Trên cơ sở đó, các chức năng cần ưu tiên phát triển gồm quản lý tài khoản người dùng, tìm kiếm và lọc sự kiện, xem chi tiết sự kiện, lưu sự kiện yêu thích, tạo và quản lý sự kiện, quản lý nhiều hạng vé, đặt vé, xác nhận thanh toán, sinh vé điện tử, check-in, gửi thông báo, lời mời và hỗ trợ nhắn tin giữa các bên liên quan. Các chức năng này tạo thành nền tảng cho những phần tổng quan chức năng và đặc tả chức năng sẽ được trình bày ở giai đoạn tiếp theo của báo cáo.

2.2 Tổng quan chức năng

Từ kết quả khảo sát ở phần 2.1, có thể thấy hệ thống cần hỗ trợ đồng thời nhiều nhóm chức năng khác nhau cho ba tác nhân chính là người dùng, người tổ chức và quản trị viên. Ở mức tổng quan, các chức năng của hệ thống được chia thành bốn nhóm lớn. Nhóm thứ nhất là quản lý tài khoản và hồ sơ người dùng, bao gồm đăng ký, đăng nhập, khôi phục mật khẩu, cập nhật hồ sơ và thiết lập nhận thông báo. Nhóm thứ hai là khám phá và theo dõi sự kiện, bao gồm tìm kiếm, lọc, xem chi

tiết, lưu sự kiện yêu thích và xem các sự kiện đã lưu hoặc đã tham gia. Nhóm thứ ba là xử lý giao dịch và tham dự sự kiện, bao gồm đặt vé, xác nhận thanh toán, lưu vé điện tử, hiển thị mã QR và kiểm tra tham dự tại sự kiện. Nhóm cuối cùng là hỗ trợ tương tác và vận hành hệ thống, bao gồm tạo sự kiện, quản lý hạng vé, nhắn tin, gửi lời mời, gửi thông báo và duyệt sự kiện ở phía quản trị viên.

2.2.1 Biểu đồ use case tổng quát



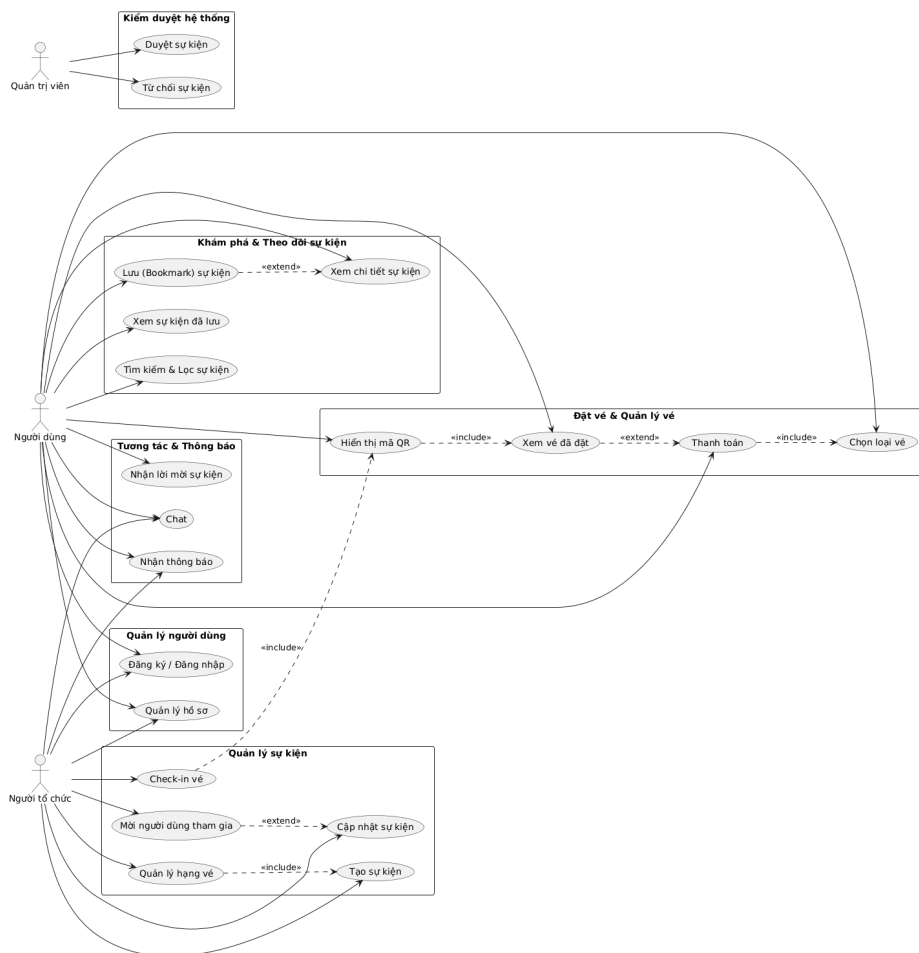
Hình 2.1: Biểu đồ use case tổng quát của hệ thống event-booking-app

Trong hệ thống có ba tác nhân chính. Tác nhân thứ nhất là người dùng, đại diện cho người tham dự sự kiện. Người dùng thực hiện các chức năng như đăng ký và đăng nhập, tìm kiếm và lọc sự kiện, xem chi tiết sự kiện, lưu sự kiện quan tâm, đặt vé, xác nhận thanh toán, xem vé điện tử, trò chuyện với người tổ chức và nhận thông báo từ hệ thống. Tác nhân thứ hai là người tổ chức, là người có quyền tạo và quản lý sự kiện của mình. Ngoài các chức năng cơ bản như người dùng thông thường, người tổ chức còn có thể tạo sự kiện, cập nhật thông tin sự kiện, quản lý hạng vé, mời người dùng tham gia và quét mã QR để check-in cho người tham dự. Tác nhân thứ ba là quản trị viên, chịu trách nhiệm duyệt hoặc từ chối các sự kiện do người tổ chức gửi lên trước khi chúng được hiển thị công khai trong hệ thống.

Người dùng tập trung vào khám phá và tham gia sự kiện, người tổ chức tập trung vào vận hành sự kiện, còn quản trị viên tập trung vào kiểm soát nội dung và trạng thái duyệt. Cách tổ chức này giúp bảo đảm luồng nghiệp vụ rõ ràng, đồng thời hỗ trợ triển khai cơ chế phân quyền tương ứng ở phía backend.

2.2.2 Biểu đồ use case phân rã theo nhóm các chức năng

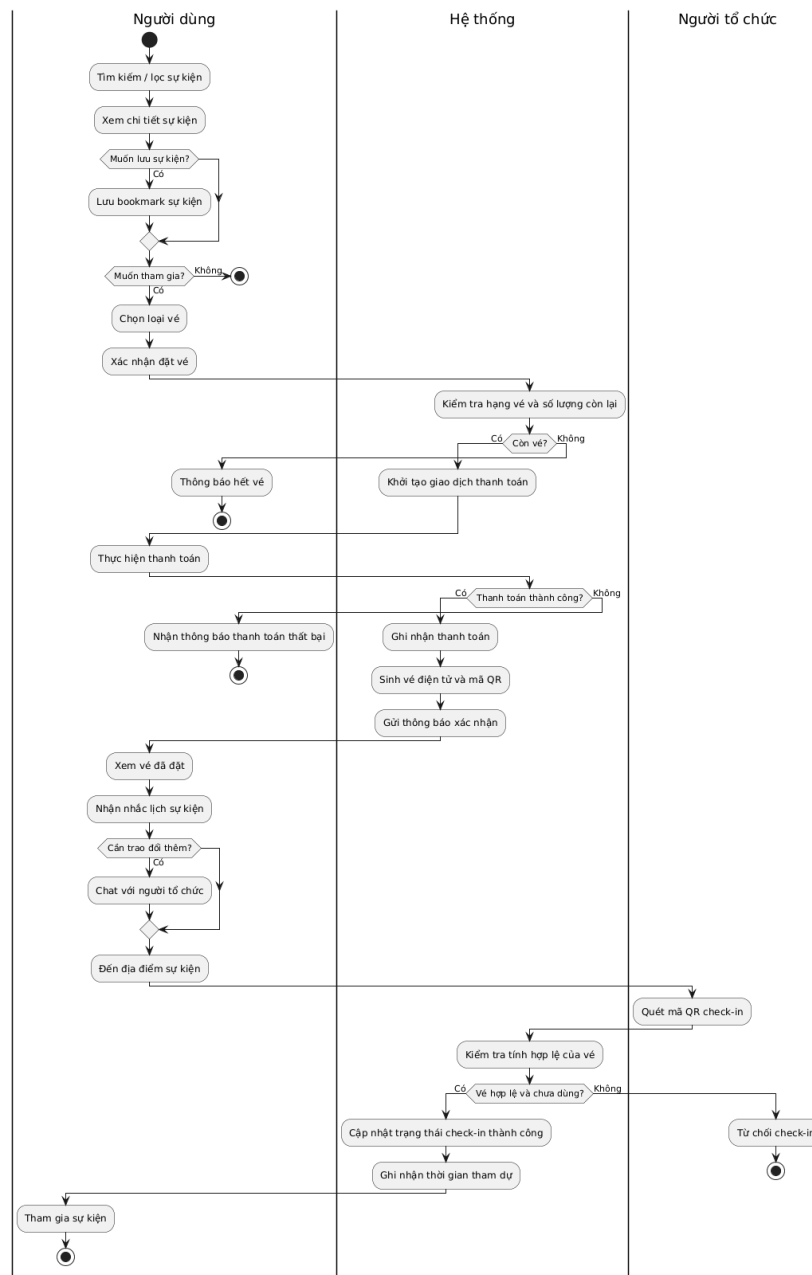
Trong số các use case mức cao, use case “Đặt vé và tham dự sự kiện” là use case quan trọng nhất vì nó kết nối trực tiếp các chức năng khám phá sự kiện, lựa chọn hạng vé, xác nhận thanh toán, phát hành vé điện tử và kiểm soát check-in. Hình 2.2 trình bày biểu đồ phân rã của use case này.



Hình 2.2: Biểu đồ phân rã theo nhóm các use case của hệ thống

Theo Hình 2.2, các use case của hệ thống được nhóm thành những cụm chức năng chính thay vì tách rời từng thao tác đơn lẻ. Nhóm “Quản lý người dùng” bao gồm các chức năng đăng ký, đăng nhập và cập nhật hồ sơ. Nhóm “Khám phá và theo dõi sự kiện” bao gồm tìm kiếm, lọc, xem chi tiết và lưu sự kiện. Nhóm “Đặt vé và quản lý vé” tập trung vào lựa chọn loại vé, thanh toán và xem vé đã đặt. Nhóm “Tương tác và thông báo” phản ánh các chức năng chat và nhận thông báo. Với phía người tổ chức, nhóm “Quản lý sự kiện” bao gồm tạo sự kiện, quản lý sự kiện, quản lý hạng vé, mời người dùng và check-in vé. Cuối cùng, phía quản trị viên đảm nhiệm nhóm chức năng kiểm duyệt hệ thống thông qua việc duyệt hoặc từ chối sự kiện. Cách phân nhóm này giúp làm rõ cấu trúc nghiệp vụ tổng thể của hệ thống và là cơ sở thuận lợi để lựa chọn các use case quan trọng cần đặc tả ở phần sau.

2.2.3 Quy trình nghiệp vụ



Hình 2.3: Biểu đồ hoạt động của quy trình nghiệp vụ tham gia sự kiện

Quy trình nghiệp vụ tiêu biểu của hệ thống là quy trình tham gia sự kiện, được mô tả trong Hình 2.3. Đây là một quy trình kết hợp nhiều use case khác nhau, bắt đầu từ lúc người dùng tìm kiếm sự kiện, xem chi tiết, có thể lưu lại để theo dõi, sau đó lựa chọn loại vé và thực hiện thanh toán. Sau khi thanh toán thành công, hệ thống phát hành vé điện tử và hỗ trợ gửi thông báo xác nhận. Trước thời điểm diễn ra chương trình, người dùng có thể nhận thông báo nhắc lịch và nếu cần có thể trao đổi thêm với người tổ chức qua chức năng chat. Khi đến địa điểm tổ chức, người dùng xuất trình vé điện tử có mã QR; người tổ chức quét mã để hệ thống xác thực

và cập nhật trạng thái check-in. Như vậy, quy trình này không chỉ giới hạn ở một thao tác mua vé, mà phản ánh đầy đủ một chuỗi hoạt động liên kết từ khám phá, giao dịch, theo dõi cho đến tham dự thực tế.

2.3 Đặc tả chức năng

Trên cơ sở biểu đồ tổng quan và quy trình nghiệp vụ ở phần 2.2, đồ án lựa chọn bốn use case quan trọng để đặc tả chi tiết. Đây là các use case phản ánh rõ nhất những nghiệp vụ cốt lõi của hệ thống, bao gồm phía người tổ chức, người dùng và quá trình tương tác giữa các tác nhân trong suốt vòng đời của một sự kiện. Bốn use case được lựa chọn gồm: tạo sự kiện, đặt vé và xác nhận thanh toán, gửi lời mời và thông báo, cùng với check-in vé bằng mã QR.

2.3.1 Đặc tả use case Tạo sự kiện

Tên use case: Tạo sự kiện.

Tiền điều kiện: Người tổ chức đã đăng nhập thành công và có quyền sử dụng chức năng tạo sự kiện.

Hậu điều kiện: Một sự kiện mới được lưu trong hệ thống ở trạng thái chờ duyệt, đồng thời các hạng vé ban đầu của sự kiện cũng được khởi tạo.

Luồng sự kiện chính: Người tổ chức truy cập chức năng tạo sự kiện và nhập các thông tin cần thiết như tiêu đề, mô tả, danh mục, thời gian, địa điểm, hình ảnh và danh sách hạng vé. Hệ thống kiểm tra tính hợp lệ của dữ liệu đầu vào, bao gồm các trường bắt buộc cũng như thông tin của từng hạng vé như tên vé, giá vé và số lượng vé. Khi toàn bộ thông tin hợp lệ, hệ thống tạo bản ghi sự kiện mới, gán người tạo là người tổ chức hiện tại, thiết lập trạng thái duyệt ban đầu là chờ duyệt và lưu các hạng vé tương ứng.

Luồng phát sinh: Nếu người tổ chức bỏ trống các trường bắt buộc như tiêu đề, danh mục, ngày hoặc địa điểm, hệ thống từ chối lưu và yêu cầu bổ sung thông tin. Nếu danh sách hạng vé không hợp lệ, chẳng hạn không có hạng vé nào, giá vé âm hoặc quota không hợp lệ, hệ thống thông báo lỗi và không tạo sự kiện. Nếu dữ liệu gửi lên sai định dạng, hệ thống trả về thông báo lỗi tương ứng.

2.3.2 Đặc tả use case Đặt vé và xác nhận thanh toán

Tên use case: Đặt vé và xác nhận thanh toán.

Tiền điều kiện: Người dùng đã đăng nhập; sự kiện tồn tại trong hệ thống; hạng vé được chọn hợp lệ và còn đủ số lượng theo quota.

Hậu điều kiện: Giao dịch thanh toán được ghi nhận; vé được phát hành ở dạng điện tử; mã QR của vé được sinh ra để phục vụ việc tham dự sự kiện.

Luồng sự kiện chính: Người dùng chọn một sự kiện mong muốn, sau đó lựa chọn hạng vé và số lượng vé cần mua. Hệ thống kiểm tra tính hợp lệ của lựa chọn, tính tổng giá trị giao dịch và khởi tạo các bản ghi vé ở trạng thái chờ thanh toán cùng với bản ghi thanh toán tương ứng. Người dùng tiếp tục xác nhận thanh toán. Khi thanh toán thành công, hệ thống cập nhật trạng thái giao dịch, đánh dấu vé là đã thanh toán, tăng số lượng vé đã bán của hạng vé tương ứng, đồng thời sinh dữ liệu QR cho từng vé điện tử. Sau đó người dùng có thể xem lại vé đã đặt trong ứng dụng.

Luồng phát sinh: Nếu hạng vé không tồn tại hoặc không thuộc sự kiện đang chọn, hệ thống dừng thao tác và thông báo lỗi. Nếu số lượng vé yêu cầu vượt quá quota còn lại, hệ thống từ chối đặt vé. Nếu tổng giá tiền gửi lên không khớp với cấu hình giá vé của hệ thống, hệ thống trả về lỗi xác thực giao dịch. Nếu thanh toán không thành công, các vé tương ứng được giữ ở trạng thái thất bại hoặc không được xác nhận hoàn tất.

2.3.3 Đặc tả use case Gửi lời mời và thông báo

Tên use case: Gửi lời mời và thông báo.

Tiền điều kiện: Tác nhân thực hiện đã đăng nhập; sự kiện cần gửi lời mời tồn tại; danh sách người nhận hợp lệ.

Hậu điều kiện: Các thông báo hoặc lời mời được tạo trong hệ thống; nếu phù hợp, thông báo đẩy được gửi tới thiết bị của người nhận.

Luồng sự kiện chính: Người dùng hoặc người tổ chức chọn một sự kiện và xác định danh sách người nhận muốn mời tham gia. Hệ thống kiểm tra sự tồn tại của sự kiện, sau đó tạo các bản ghi thông báo cho từng người nhận với nội dung lời mời tương ứng. Nếu người nhận đã bật thông báo và đã đăng ký thiết bị, hệ thống tiếp tục gửi thông báo đẩy tới thiết bị di động của họ. Ngoài lời mời sự kiện, cùng cơ chế này còn được dùng để ghi nhận và gửi các thông báo hệ thống như xác nhận thanh toán, tin nhắn mới hoặc nhắc lịch.

Luồng phát sinh: Nếu sự kiện không tồn tại, hệ thống hủy thao tác và thông báo lỗi. Nếu danh sách người nhận rỗng hoặc không hợp lệ, hệ thống không tạo lời mời. Nếu người nhận đã tắt thông báo hoặc chưa đăng ký thiết bị nhận thông báo, hệ thống vẫn có thể lưu bản ghi thông báo nội bộ nhưng không gửi thông báo đẩy ra bên ngoài.

2.3.4 Đặc tả use case Check-in vé bằng mã QR

Tên use case: Check-in vé bằng mã QR.

Tiền điều kiện: Người tổ chức đã đăng nhập; là chủ sở hữu của sự kiện tương

ứng; người tham dự đã có vé hợp lệ ở trạng thái thanh toán thành công.

Hậu điều kiện: Vé được cập nhật trạng thái đã check-in nếu hợp lệ; ngược lại hệ thống từ chối check-in và giữ nguyên trạng thái vé.

Luồng sự kiện chính: Người tổ chức mở chức năng check-in và quét mã QR từ vé điện tử của người tham dự. Dữ liệu quét được gửi tới hệ thống để phân tích. Hệ thống kiểm tra mã vé có đúng định dạng hay không, vé có thuộc sự kiện hiện tại hay không, trạng thái thanh toán của vé đã hợp lệ hay chưa và vé đã từng được sử dụng trước đó chưa. Nếu tất cả điều kiện đều thỏa mãn, hệ thống đánh dấu vé là đã check-in, ghi nhận thời điểm check-in và người thực hiện xác nhận. Sau đó hệ thống trả về kết quả check-in thành công cho người tổ chức.

Luồng phát sinh: Nếu mã QR sai định dạng, hệ thống từ chối xử lý và thông báo mã không hợp lệ. Nếu vé không thuộc sự kiện hiện tại, hệ thống thông báo sai sự kiện. Nếu vé chưa được thanh toán, hệ thống từ chối check-in. Nếu vé đã được sử dụng trước đó, hệ thống trả về trạng thái đã check-in và không cập nhật lại dữ liệu.

2.4 Yêu cầu phi chức năng

Ngoài các chức năng nghiệp vụ, hệ thống còn phải đáp ứng một số yêu cầu phi chức năng để bảo đảm khả năng vận hành ổn định, an toàn và thuận tiện cho người sử dụng. Các yêu cầu này được xác định dựa trên đặc điểm của ứng dụng di động có tích hợp backend, xử lý dữ liệu sự kiện, vé điện tử và tương tác gần thời gian thực.

2.4.1 Yêu cầu về hiệu năng

Hệ thống cần bảo đảm thời gian phản hồi hợp lý đối với các chức năng thường xuyên được sử dụng như đăng nhập, tải danh sách sự kiện, xem chi tiết sự kiện, gửi tin nhắn và truy xuất vé điện tử. Đối với người dùng di động, trải nghiệm bị ảnh hưởng mạnh nếu thời gian tải dữ liệu quá lâu hoặc thao tác xác nhận thanh toán, mở vé và check-in diễn ra chậm. Vì vậy, ứng dụng cần tổ chức API gọn nhẹ, hạn chế truy vấn dư thừa và tối ưu số lượng dữ liệu trả về ở các màn hình chính.

2.4.2 Yêu cầu về độ tin cậy và tính nhất quán dữ liệu

Do hệ thống xử lý nhiều thực thể liên quan như người dùng, sự kiện, vé, thanh toán và thông báo, dữ liệu cần được cập nhật nhất quán giữa ứng dụng khách và máy chủ. Trạng thái vé, số lượng vé đã bán, thông tin thanh toán và trạng thái check-in phải phản ánh đúng thực tế tại thời điểm sử dụng. Hệ thống cũng cần giảm thiểu lỗi phát sinh khi nhiều người dùng đồng thời thao tác trên cùng một sự kiện, đặc biệt trong các trường hợp đặt vé hoặc xác nhận tham dự.

2.4.3 Yêu cầu về bảo mật

Hệ thống cần bảo vệ thông tin tài khoản và dữ liệu nghiệp vụ thông qua cơ chế xác thực, phân quyền và kiểm soát truy cập phù hợp. Các API liên quan đến tài khoản, vé, thanh toán, tin nhắn và quản trị cần chỉ cho phép truy cập khi người dùng đã được xác thực hợp lệ. Ngoài ra, các dữ liệu nhạy cảm như mật khẩu, token và thông tin thao tác của người dùng phải được quản lý an toàn, tránh lộ lọt hoặc bị sử dụng trái phép.

2.4.4 Yêu cầu về khả năng sử dụng

Ứng dụng hướng tới người dùng cuối thao tác chủ yếu trên điện thoại, vì vậy giao diện cần rõ ràng, dễ hiểu và hạn chế các bước xử lý phức tạp. Các chức năng quan trọng như tìm sự kiện, đặt vé, xem vé đã mua, nhận thông báo và gửi tin nhắn cần được bố trí trực quan. Hệ thống cũng cần hỗ trợ trải nghiệm liên tục giữa các màn hình, giúp người dùng không bị đứt quãng khi chuyển từ khám phá sự kiện sang đặt vé hoặc từ thông báo sang xem chi tiết nội dung liên quan.

2.4.5 Yêu cầu về khả năng bảo trì và mở rộng

Do đây là hệ thống có nhiều nhóm chức năng, mã nguồn cần được tổ chức theo cấu trúc rõ ràng để thuận lợi cho việc bảo trì và mở rộng sau này. Các thành phần như xác thực, sự kiện, vé, thông báo, nhắn tin và nhắc lịch nên được tách thành các mô-đun tương đối độc lập, giúp dễ sửa lỗi, bổ sung tính năng và kiểm thử. Yêu cầu này đặc biệt quan trọng vì hệ thống có thể được mở rộng thêm các cổng thanh toán thực tế, công cụ quản trị nâng cao hoặc cơ chế gợi ý cá nhân hóa trong các phiên bản tiếp theo.

2.4.6 Yêu cầu kỹ thuật triển khai

Hệ thống cần có khả năng hoạt động trên môi trường di động phổ biến và kết nối được với backend thông qua mạng Internet thông thường. Về phía lưu trữ, cơ sở dữ liệu cần đủ linh hoạt để quản lý các thực thể thay đổi theo nghiệp vụ như sự kiện, hạng vé, thông báo và phòng chat. Về phía tích hợp, hệ thống cũng cần hỗ trợ các dịch vụ bổ sung như thông báo đẩy, sinh mã QR và xử lý nội dung hỗ trợ bằng giao diện lập trình ứng dụng (Application Programming Interface - API), từ đó tạo nền tảng kỹ thuật phù hợp cho việc hiện thực hóa sản phẩm.

Từ kết quả khảo sát trong chương này có thể thấy bài toán đặt vé và quản lý sự kiện trên thiết bị di động đòi hỏi một hệ thống tích hợp nhiều chức năng hơn so với các nền tảng chỉ thiên về cộng đồng hoặc bán vé. Việc phân tích các ứng dụng tương tự đã giúp xác định rõ những chức năng cốt lõi cần ưu tiên, làm rõ các yêu cầu phi chức năng mà hệ thống phải đáp ứng để vận hành ổn định trong thực tế.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này trình bày các nền tảng kỹ thuật và công nghệ chính được sử dụng để xây dựng hệ thống đặt vé và quản lý sự kiện trên thiết bị di động. Do đặc thù của bài toán yêu cầu đồng thời hỗ trợ nhiều nghiệp vụ như quản lý tài khoản, tổ chức sự kiện, đặt vé, thông báo, nhắc lịch, trò chuyện và kiểm soát check-in, giải pháp kỹ thuật cần bảo đảm tính linh hoạt khi phát triển, khả năng mở rộng hợp lý và thuận tiện cho việc tích hợp nhiều chức năng trên cùng một hệ thống. Vì vậy, đồ án lựa chọn kiến trúc client-server theo hướng mobile-first, trong đó ứng dụng di động đảm nhiệm tương tác với người dùng, còn máy chủ backend chịu trách nhiệm xử lý nghiệp vụ và lưu trữ dữ liệu tập trung.

3.1 Kiến trúc ứng dụng di động kết hợp dịch vụ backend

Kiến trúc tổng thể của hệ thống được xây dựng theo mô hình ứng dụng khách và máy chủ. Mô hình này được lựa chọn để giải quyết yêu cầu đồng bộ dữ liệu giữa nhiều nhóm người dùng như người tham dự, người tổ chức và quản trị viên, đồng thời hỗ trợ quản lý nghiệp vụ tập trung như xác thực, quản lý sự kiện, quản lý vé, xử lý thông báo và kiểm soát quyền truy cập. Với bài toán của đề tài, một số hướng tiếp cận thay thế có thể kể đến như ứng dụng web thuần, ứng dụng desktop hoặc mô hình backendless sử dụng hoàn toàn dịch vụ bên thứ ba. Tuy nhiên, ứng dụng web thuần thường kém phù hợp hơn trong trải nghiệm di động chuyên biệt, ứng dụng desktop không phù hợp với bối cảnh sử dụng thường xuyên khi tham gia sự kiện, còn mô hình backendless tuy rút ngắn thời gian phát triển nhưng làm giảm khả năng kiểm soát nghiệp vụ tùy biến. Vì vậy, mô hình ứng dụng di động kết hợp backend riêng được xem là phù hợp hơn cho yêu cầu của hệ thống.

3.2 React Native và Expo cho lớp ứng dụng khách

Phía ứng dụng khách của hệ thống được phát triển bằng React Native kết hợp với Expo. React Native cho phép xây dựng ứng dụng di động đa nền tảng bằng JavaScript/TypeScript, đồng thời cung cấp khả năng tái sử dụng lớn về cấu trúc giao diện và logic hiển thị. Expo hỗ trợ đáng kể trong quá trình phát triển thông qua hệ sinh thái thư viện sẵn có cho thông báo, định vị, camera, quét mã và nhiều tính năng thường gặp trong ứng dụng di động hiện đại. Hai công nghệ này được sử dụng để giải quyết yêu cầu xây dựng nhanh một ứng dụng mobile-first có nhiều màn hình nghiệp vụ như đăng nhập, khám phá sự kiện, đặt vé, quản lý hồ sơ, thông báo, nhắn tin và check-in.

Các lựa chọn thay thế cho lớp ứng dụng khách có thể là native Android/iOS, Flutter hoặc Progressive Web App. Phát triển native mang lại khả năng tối ưu cao

nhưng làm tăng chi phí phát triển song song cho hai nền tảng. Flutter có lợi thế về hiệu năng giao diện, tuy nhiên trong phạm vi đề tài, React Native kết hợp Expo phù hợp hơn nhờ hệ sinh thái gần gũi với JavaScript, dễ tích hợp với các thư viện hiện có và rút ngắn thời gian hiện thực hóa sản phẩm thử nghiệm.

3.3 Node.js và Express cho lớp dịch vụ nghiệp vụ

Phía máy chủ của hệ thống sử dụng Node.js và Express. Node.js phù hợp với các ứng dụng cần xử lý nhiều kết nối đồng thời và có số lượng lớn yêu cầu I/O như truy vấn dữ liệu, xác thực người dùng, gửi thông báo và trao đổi thời gian thực. Express được lựa chọn để xây dựng các API Representational State Transfer (REST) cho những nghiệp vụ chính như quản lý tài khoản, sự kiện, vé, thanh toán, nhắc lịch, bookmark, đánh giá và trò chuyện, với dữ liệu trao đổi chủ yếu ở định dạng JavaScript Object Notation (JSON). Cặp công nghệ này giải quyết yêu cầu xây dựng một backend thống nhất, dễ tổ chức theo mô hình route-controller-service và thuận lợi trong việc mở rộng chức năng sau này.

Một số phương án thay thế có thể dùng cho backend là Spring Boot, Django hoặc NestJS. Spring Boot mạnh về cấu trúc enterprise nhưng tương đối nặng cho phạm vi đồ án ứng dụng cỡ vừa. Django cung cấp nhiều thành phần tích hợp sẵn nhưng không đồng bộ về ngôn ngữ với phía ứng dụng khách. NestJS là lựa chọn hiện đại hơn trong hệ sinh thái Node.js, tuy nhiên Express được ưu tiên trong đề tài vì đơn giản, phổ biến và phù hợp với mức độ kiểm soát trực tiếp mà nhóm phát triển mong muốn.

3.4 MongoDB và Mongoose cho lưu trữ dữ liệu

Hệ thống sử dụng MongoDB làm hệ quản trị cơ sở dữ liệu và Mongoose làm thư viện ánh xạ dữ liệu cho Node.js. Việc lựa chọn này nhằm giải quyết yêu cầu lưu trữ dữ liệu có cấu trúc linh hoạt như người dùng, sự kiện, hạng vé, vé điện tử, thanh toán, phòng chat, tin nhắn và thông báo. Trong bài toán sự kiện, nhiều thực thể có thể thay đổi về thuộc tính theo từng phiên bản phát triển, chẳng hạn sự kiện có thể mở rộng thêm hạng vé, trạng thái duyệt hay thông tin tọa độ. Mô hình tài liệu của MongoDB, vốn thuộc nhóm cơ sở dữ liệu phi quan hệ (Not Only SQL - NoSQL), giúp thích nghi tốt với sự thay đổi đó, trong khi Mongoose hỗ trợ định nghĩa schema, kiểm tra dữ liệu, quan hệ tham chiếu và thao tác với cơ sở dữ liệu thuận tiện hơn.

Các lựa chọn thay thế có thể kể đến như MySQL, PostgreSQL hoặc Firebase Firestore. Hệ quản trị quan hệ phù hợp với dữ liệu chuẩn hóa chặt chẽ nhưng có thể làm tăng độ phức tạp khi mô hình nghiệp vụ cần mở rộng linh hoạt trong giai đoạn thử nghiệm. Firestore hỗ trợ thời gian thực tốt nhưng việc tự kiểm soát toàn

bộ nghiệp vụ phía máy chủ sẽ hạn chế hơn. Vì vậy, MongoDB kết hợp Mongoose là lựa chọn phù hợp giữa tính linh hoạt phát triển và khả năng kiểm soát dữ liệu.

3.5 JWT cho xác thực và phân quyền

Để giải quyết yêu cầu xác thực người dùng và bảo vệ các tài nguyên nghiệp vụ của hệ thống, đề tài sử dụng cơ chế xác thực dựa trên JSON Web Token (JWT) [7]. Sau khi đăng nhập thành công, máy chủ cấp token cho người dùng, và token này được gửi kèm trong các yêu cầu tới những API cần xác thực. Cách tiếp cận này phù hợp với ứng dụng di động vì cho phép duy trì trạng thái đăng nhập tương đối gọn nhẹ, không phụ thuộc phiên làm việc lưu trên máy chủ theo kiểu session truyền thống. Ngoài xác thực, hệ thống còn áp dụng phân quyền theo vai trò người dùng và quản trị viên để kiểm soát các chức năng quản trị hoặc nghiệp vụ chuyên biệt.

Các phương án thay thế cho JWT là session-based authentication hoặc OAuth hoàn chỉnh từ bên thứ ba. Session phù hợp với web truyền thống nhưng kém linh hoạt hơn trong môi trường ứng dụng di động phân tán. OAuth rất mạnh với kịch bản đăng nhập liên thông, tuy nhiên vượt quá phạm vi cốt lõi của đề tài. Do đó, JWT được lựa chọn vì đáp ứng tốt yêu cầu xác thực API, đơn giản trong triển khai và phù hợp với kiến trúc hệ thống hiện tại.

3.6 Socket.IO cho giao tiếp thời gian thực

Đối với yêu cầu trao đổi tin nhắn và cập nhật trạng thái phòng chat gần thời gian thực, hệ thống sử dụng Socket.IO. Công nghệ này cho phép thiết lập kết nối hai chiều giữa máy khách và máy chủ, phù hợp với bài toán gửi tin nhắn, tham gia phòng trò chuyện và phát sự kiện cập nhật ngay khi có nội dung mới. Trong bối cảnh ứng dụng sự kiện, tính năng giao tiếp trực tiếp giữa người dùng với người tổ chức hoặc giữa các thành viên liên quan làm tăng khả năng tương tác và hỗ trợ xử lý nhanh các tình huống phát sinh.

Các lựa chọn thay thế có thể là polling định kỳ, Server-Sent Events hoặc dịch vụ chat của bên thứ ba. Polling dễ triển khai nhưng gây lãng phí tài nguyên và độ trễ cao hơn. Server-Sent Events chủ yếu phù hợp với luồng dữ liệu một chiều từ máy chủ xuống máy khách. Dịch vụ bên thứ ba rút ngắn thời gian triển khai nhưng làm giảm khả năng tùy biến nghiệp vụ. Vì vậy, Socket.IO là lựa chọn phù hợp hơn cho chức năng chat của hệ thống.

3.7 Thông báo đẩy và nhắc lịch sự kiện

Hệ thống tích hợp Expo Notifications để hỗ trợ thông báo đẩy trên thiết bị di động. Công nghệ này được sử dụng để giải quyết yêu cầu gửi thông báo khi có tin nhắn mới, xác nhận thanh toán, thay đổi liên quan tới sự kiện và nhắc lịch trước

thời điểm diễn ra sự kiện. Bên cạnh đó, backend triển khai bộ lập lịch kiểm tra các sự kiện sắp diễn ra và gửi thông báo trước các mốc thời gian nhất định. Đây là thành phần quan trọng đối với bài toán sự kiện, bởi người dùng không chỉ cần mua vé mà còn cần được nhắc đúng lúc để tránh bỏ lỡ chương trình đã đăng ký.

Một số lựa chọn thay thế có thể là Firebase Cloud Messaging kết hợp giải pháp native riêng hoặc các nền tảng thông báo thương mại. Tuy nhiên, Expo Notifications thuận lợi hơn trong phạm vi đề tài vì tích hợp tốt với hệ sinh thái Expo, giảm đáng kể khối lượng cấu hình và phù hợp với quy mô hệ thống thử nghiệm.

3.8 Mã QR trong quản lý vé điện tử

Để giải quyết yêu cầu số hóa vé và kiểm soát check-in tại sự kiện, đề tài sử dụng cơ chế sinh và đọc mã QR cho từng vé điện tử thông qua thư viện tạo mã phù hợp trên nền Node.js. Mỗi vé sau khi thanh toán thành công được gắn một chuỗi dữ liệu định danh, từ đó hỗ trợ kiểm tra tính hợp lệ khi người tổ chức thực hiện check-in. Hướng tiếp cận này giúp giảm phụ thuộc vào vé giấy, rút ngắn thời gian xác nhận tham dự và tạo trải nghiệm thuận tiện hơn cho cả người dùng lẫn người tổ chức.

Những phương án thay thế cho kiểm soát check-in có thể là mã vạch một chiều, danh sách thủ công hoặc xác nhận bằng thông tin định danh cá nhân. So với các phương án này, mã QR có ưu điểm là dễ sinh tự động, dễ quét trên thiết bị di động và phù hợp với ngữ cảnh triển khai nhanh trong môi trường ứng dụng sự kiện.

3.9 Hỗ trợ sinh nội dung sự kiện bằng AI

Ngoài các nghiệp vụ cốt lõi, hệ thống còn tích hợp khả năng gợi ý nội dung sự kiện bằng mô hình ngôn ngữ lớn thông qua dịch vụ API. Thành phần này được sử dụng để hỗ trợ người tổ chức khi cần đề xuất tiêu đề, mô tả ngắn và các hạng vé ban đầu cho sự kiện. Trong bài toán thực tế, nhiều người tổ chức gặp khó khăn khi phải đồng thời chuẩn bị nội dung quảng bá và cấu hình chương trình. Việc bổ sung chức năng hỗ trợ bằng trí tuệ nhân tạo (Artificial Intelligence - AI) không thay thế vai trò của người dùng, nhưng giúp rút ngắn thời gian chuẩn bị và tăng tính thuận tiện cho thao tác tạo sự kiện.

Các hướng thay thế cho bài toán này có thể là nhập tay hoàn toàn, sử dụng mẫu nội dung cố định hoặc tích hợp một công cụ gợi ý rule-based. Tuy nhiên, nhập tay hoàn toàn làm tăng thời gian thao tác, trong khi mẫu cố định thường thiếu linh hoạt. Vì vậy, hướng tích hợp AI được lựa chọn như một thành phần hỗ trợ mở rộng, phù hợp với xu thế cá nhân hóa nội dung trong các ứng dụng hiện đại.

3.10 Kết chương

Chương này đã trình bày các nền tảng kỹ thuật và công nghệ chính được sử dụng trong hệ thống đặt vé và quản lý sự kiện của đồ án. Mỗi lựa chọn công nghệ đều được xem xét dựa trên yêu cầu nghiệp vụ tương ứng, đồng thời đối chiếu với một số phương án thay thế để làm rõ lý do lựa chọn. Từ việc sử dụng kiến trúc client-server theo hướng mobile-first, React Native và Expo cho ứng dụng khách, Node.js và Express cho dịch vụ backend, MongoDB cho lưu trữ dữ liệu, JWT cho xác thực, Socket.IO cho thời gian thực, thông báo đẩy cho nhắc lịch, mã QR cho vé điện tử và AI cho hỗ trợ tạo nội dung, hệ thống đã hình thành được nền tảng kỹ thuật tương đối đầy đủ để triển khai các chức năng cốt lõi của bài toán. Trên cơ sở này, chương tiếp theo có thể tập trung trình bày chi tiết hơn về thiết kế, hiện thực và đánh giá hệ thống.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày quá trình phân tích thiết kế, hiện thực và đánh giá hệ thống đặt vé và quản lý sự kiện được phát triển trong đồ án. Trước hết, chương làm rõ kiến trúc phần mềm được lựa chọn và cách áp dụng kiến trúc đó vào hệ thống thực tế. Tiếp theo, chương mô tả thiết kế tổng quan, thiết kế chi tiết, quá trình xây dựng ứng dụng, kết quả đạt được, kiểm thử và định hướng triển khai. Các nội dung trong chương này đóng vai trò kết nối giữa phần khảo sát yêu cầu ở các chương trước với phần đóng góp và tổng kết ở các chương sau, qua đó cho thấy hệ thống đã được tổ chức và hiện thực hóa như thế nào trên cả phía ứng dụng di động lẫn phía máy chủ.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Đối với bài toán đặt vé và quản lý sự kiện trên thiết bị di động, hệ thống cần đồng thời hỗ trợ nhiều chức năng khác nhau như quản lý tài khoản, tìm kiếm sự kiện, quản lý sự kiện, đặt vé, thanh toán, nhắn tin, thông báo và check-in bằng mã QR. Các chức năng này có liên hệ chặt chẽ với nhau, dùng chung dữ liệu và cùng phục vụ một luồng nghiệp vụ xuyên suốt từ lúc người dùng khám phá sự kiện cho tới lúc tham dự thực tế. Vì vậy, kiến trúc phù hợp nhất cho hệ thống trong phạm vi đồ án là kiến trúc client-server theo hướng mobile-first, kết hợp với tổ chức phần mềm theo mô hình ba lớp ở phía máy chủ. Theo cách hiểu này, ứng dụng di động đóng vai trò lớp giao diện và tương tác với người dùng, máy chủ backend đóng vai trò lớp xử lý nghiệp vụ, còn cơ sở dữ liệu là lớp lưu trữ dữ liệu tập trung.

Kiến trúc ba lớp được lựa chọn thay cho các hướng như microservice hay backendless vì nó phù hợp hơn với quy mô hiện tại của hệ thống. Nếu sử dụng microservice, mỗi nhóm chức năng như tài khoản, sự kiện, vé, thông báo hay chat có thể được tách thành các dịch vụ độc lập. Tuy nhiên, với phạm vi của đồ án, cách tiếp cận đó làm tăng độ phức tạp triển khai, quản lý giao tiếp giữa các dịch vụ và đồng bộ dữ liệu, trong khi lượng người dùng và số lượng giao dịch chưa ở mức cần phân tán mạnh. Ngược lại, kiến trúc đơn khối có tổ chức theo lớp giúp hệ thống dễ xây dựng, dễ bảo trì hơn trong giai đoạn phát triển đầu tiên, đồng thời vẫn đủ rõ ràng để mở rộng khi cần. So với mô hình backendless, việc xây dựng backend riêng còn cho phép kiểm soát chặt chẽ hơn các nghiệp vụ đặc thù như quản lý nhiều hạng vé, kiểm soát trạng thái thanh toán, gửi lời mời, kiểm tra vé đã dùng hay chưa và xác nhận check-in theo sự kiện cụ thể.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Xét trên hệ thống cụ thể của đề tài, lớp giao diện và tương tác được hiện thực bằng ứng dụng di động React Native kết hợp Expo. Thành phần này bao gồm các màn hình chức năng trong thư mục `frontend/pages`, các thành phần giao diện dùng lại trong `frontend/components`, hệ điều hướng trong `frontend/-navigators`, cùng các ngữ cảnh quản lý trạng thái như xác thực và bản địa hóa trong `frontend/context`. Nhiệm vụ của lớp này là tiếp nhận thao tác của người dùng, tổ chức luồng di chuyển giữa các màn hình, hiển thị dữ liệu và gửi yêu cầu đến backend thông qua các service như `auth.service`, `event.service`, `ticket.service`, `notification.service` hay `chat.service`. Nói cách khác, đây là phần gần nhất với người dùng cuối và chịu trách nhiệm thể hiện toàn bộ chức năng của hệ thống trên thiết bị di động.

Lớp xử lý nghiệp vụ được tổ chức ở phía backend theo một dạng mô hình Model – View – Controller (MVC) điều chỉnh cho ứng dụng API và dịch vụ thời gian thực. Trong hệ thống này, vai trò tương đương với phần Model được thể hiện bởi các schema và model Mongoose trong thư mục `backend/src/models`, nơi quản lý cấu trúc dữ liệu của người dùng, sự kiện, vé, thanh toán, thông báo, bookmark, chatroom và message. Vai trò tương đương với phần Controller là các controller trong `backend/src/controllers`, phụ trách xử lý từng luồng nghiệp vụ như đăng nhập, tạo sự kiện, đặt vé, xác nhận thanh toán, gửi lời mời, nhắn tin hay check-in. Bổ sung cho đó là các middleware và service trong `backend/src/middleware`, `backend/src/services` và `backend/src/utills`, dùng để giải quyết những nghiệp vụ dùng chung như xác thực JWT, gửi email, gửi thông báo đẩy, sinh mã QR, gợi ý nội dung sự kiện bằng AI hoặc lập lịch nhắc sự kiện. Phần View trong mô hình MVC truyền thống không được đặt ở phía backend như các ứng dụng web server-rendered, mà được chuyển sang ứng dụng di động ở frontend. Đây là điểm điều chỉnh quan trọng để phù hợp với kiến trúc ứng dụng di động hiện đại. Chi tiết hơn về ba hướng xử lý đáng chú ý là luồng đặt vé – thanh toán – check-in, cơ chế nhắc lịch thông báo hợp nhất và mô-đun AI có fallback sẽ được phân tích lần lượt tại các Mục 5.2, 5.3 và 5.4 của Chương 5.

Ở mức tổ chức truy cập, các route trong `backend/src/routes` đóng vai trò cổng vào của lớp dịch vụ. Mỗi nhóm route như `/api/auth`, `/api/users`, `/api/events`, `/api/tickets`, `/api/notifications` hay `/api/chat` tiếp nhận yêu cầu từ ứng dụng khách, sau đó chuyển tiếp cho controller tương ứng. Điều này tạo nên một chuỗi xử lý tương đối rõ ràng: người dùng thao tác trên giao diện di động, service ở frontend gửi yêu cầu giao thức truyền tải siêu văn bản (HyperText Transfer Protocol - HTTP) hoặc sự kiện socket tới backend, route tiếp nhận yêu cầu, controller xử lý nghiệp vụ, model truy cập dữ liệu, sau đó kết quả

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

được trả ngược về giao diện. Với các chức năng gần thời gian thực như chat, hệ thống mở rộng thêm một kênh giao tiếp bằng Socket.IO ở tệp `backend/src/socket.ts`. Đây có thể xem là một thành phần bổ sung cho lớp xử lý nghiệp vụ, giúp hệ thống vượt ra ngoài mô hình request-response thuần túy mà vẫn không phá vỡ cấu trúc tổng thể.

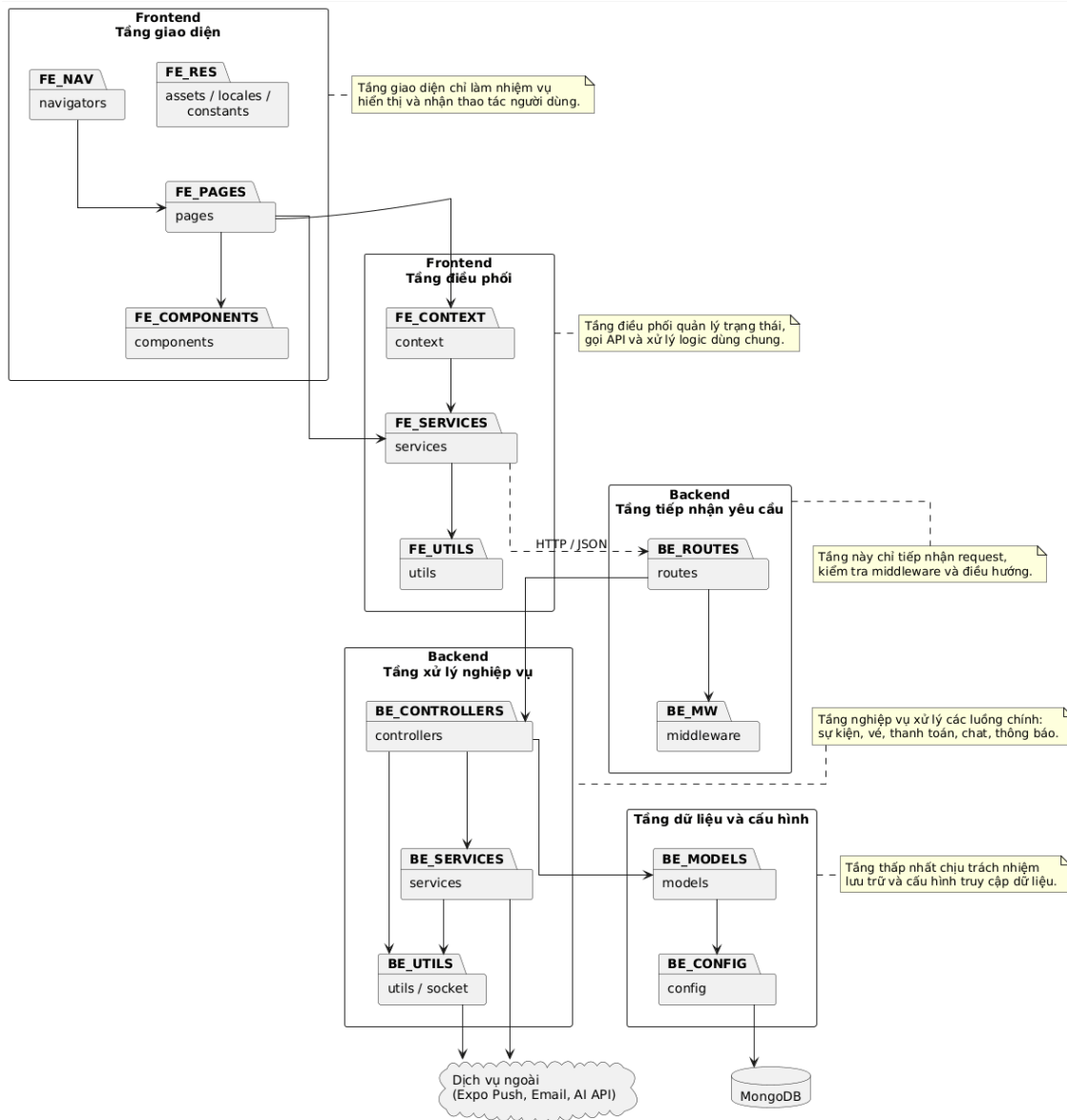
Lớp dữ liệu của hệ thống được xây dựng trên MongoDB và được truy cập thông qua Mongoose. Việc dùng cơ sở dữ liệu tài liệu giúp hệ thống linh hoạt trong việc biểu diễn các thực thể có cấu trúc thay đổi theo nghiệp vụ, chẳng hạn sự kiện có thể có nhiều hạng vé, trạng thái duyệt, thông tin tọa độ hoặc dữ liệu liên quan đến người tổ chức. Với bài toán của đề tài, dữ liệu không chỉ cần được lưu trữ mà còn phải phản ánh quan hệ giữa nhiều thực thể như người dùng – sự kiện – vé – thanh toán – thông báo. Bởi vậy, lớp dữ liệu không chỉ đóng vai trò lưu trữ mà còn là nền tảng để bảo đảm tính nhất quán cho các thao tác nghiệp vụ quan trọng như tăng số lượng vé đã bán, đánh dấu vé đã check-in hoặc ghi nhận lời mời đã gửi.

Từ góc nhìn tổng thể, kiến trúc được áp dụng cho hệ thống có thể mô tả là kiến trúc client-server ba lớp, trong đó backend được tổ chức theo tinh thần MVC điều chỉnh cho ứng dụng API. Đây là lựa chọn phù hợp với sản phẩm của đồ án vì vừa bảo đảm tính mạch lạc trong thiết kế, vừa đủ linh hoạt để tích hợp thêm các thành phần mở rộng như thông báo đẩy, chat thời gian thực, AI hỗ trợ nội dung và kiểm soát tham dự bằng mã QR. Quan trọng hơn, kiến trúc này cho phép nhóm chức năng của hệ thống được phân chia tương đối rõ ràng, giúp cho việc phát triển, bảo trì và mở rộng về sau thuận lợi hơn so với một cấu trúc mã nguồn rời rạc hoặc quá phức tạp so với quy mô hiện tại của đề tài.

4.1.2 Thiết kế tổng quan

Biểu đồ gói tổng quan của hệ thống được xây dựng theo hướng phân tầng rõ ràng giữa phía ứng dụng di động và phía máy chủ. Mục tiêu của cách tổ chức này là giúp các thành phần đảm nhiệm đúng trách nhiệm của mình, hạn chế sự phụ thuộc chồng chéo và thuận lợi cho việc bảo trì sau này. Ở mức khái quát, hệ thống được chia thành các nhóm gói chính gồm: nhóm giao diện người dùng ở frontend, nhóm điều phối và truy cập dịch vụ ở frontend, nhóm tiếp nhận yêu cầu ở backend, nhóm xử lý nghiệp vụ và tích hợp ở backend, cùng với nhóm dữ liệu và cấu hình ở tầng thấp nhất.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.1: Biểu đồ gói tổng quan của hệ thống event-booking-app

Ở phía frontend, các package có thể được chia thành hai tầng chính. Tầng giao diện bao gồm pages, components, navigators, assets, locales và constants. Trong đó, pages chứa các màn hình nghiệp vụ cụ thể như đăng ký, đặt vé, quản lý sự kiện, chat và thông báo; components chứa các thành phần giao diện dùng lại; navigators chịu trách nhiệm tổ chức luồng điều hướng; còn constants, assets và locales cung cấp các tài nguyên dùng chung. Tầng điều phối ở frontend bao gồm context, services và utils. Package context lưu trạng thái dùng chung như thông tin xác thực và ngôn ngữ, package services thực hiện giao tiếp với backend thông qua API, còn utils xử lý các hàm phụ trợ. Cấu trúc này đảm bảo giao diện không truy cập trực tiếp tới backend hay dữ liệu lưu trữ, mà luôn đi qua tầng service và context tương ứng.

Ở phía backend, package routes là lớp tiếp nhận yêu cầu từ bên ngoài và phân

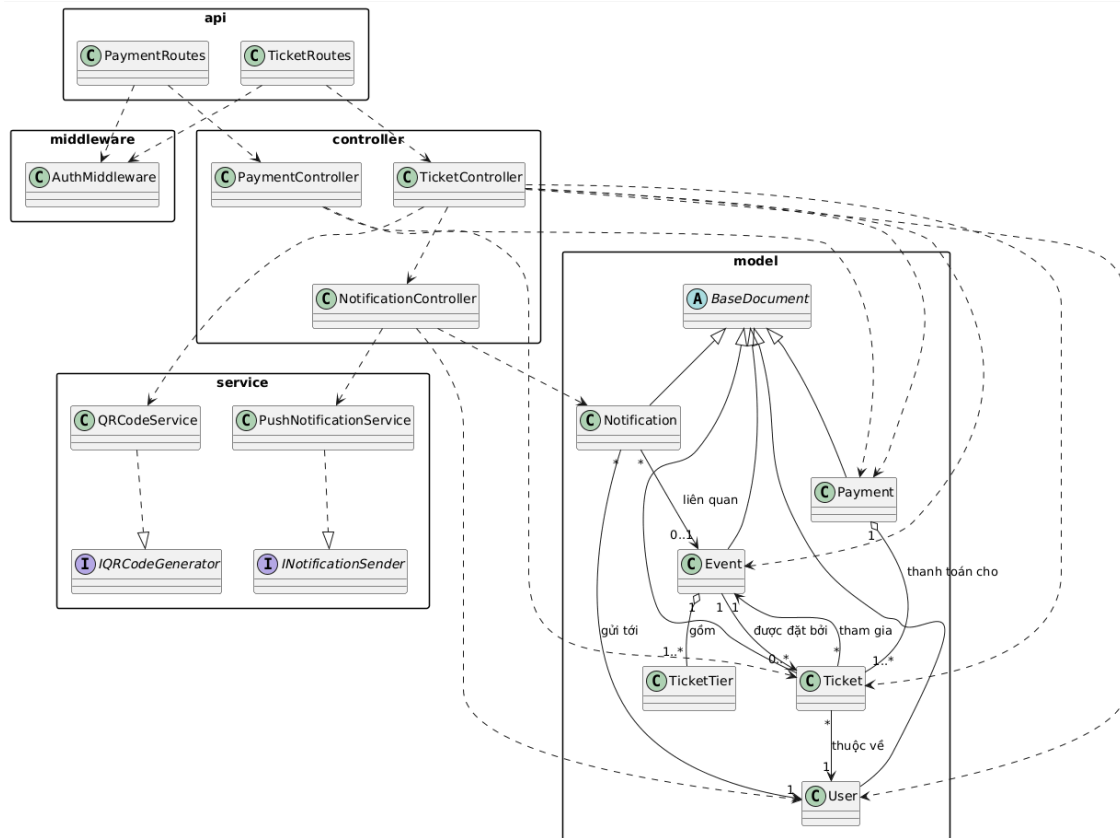
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

phối yêu cầu đến các controller tương ứng. Package `middleware` cung cấp các cơ chế kiểm tra xác thực, phân quyền và xử lý lỗi dùng chung cho toàn hệ thống. Package `controllers` là trung tâm điều phối nghiệp vụ, nơi hiện thực các luồng như đăng nhập, tạo sự kiện, đặt vé, gửi lời mời, check-in và xử lý thông báo. Package `services` và `utils` bổ sung các chức năng hỗ trợ hoặc tích hợp như sinh gợi ý nội dung sự kiện bằng AI, gửi email, gửi thông báo đẩy, lập lịch nhắc sự kiện, sinh mã QR và xử lý socket thời gian thực. Package `models` đại diện cho tầng dữ liệu nghiệp vụ, mô tả các thực thể như người dùng, sự kiện, vé, thanh toán, thông báo, bookmark, chatroom và tin nhắn. Cuối cùng, package `config` chịu trách nhiệm kết nối cơ sở dữ liệu và nạp biến môi trường, tạo nền tảng kỹ thuật cho toàn bộ các package phía trên.

4.1.3 Thiết kế chi tiết gói

Để làm rõ cách tổ chức các lớp trong hệ thống, đề tài lựa chọn nhóm package giải quyết nghiệp vụ đặt vé, thanh toán và tham dự sự kiện để mô tả chi tiết. Đây là nhóm package quan trọng vì nó kết nối trực tiếp nhiều thực thể nghiệp vụ nhất của hệ thống, đồng thời bao trùm các thao tác cốt lõi như chọn vé, xử lý thanh toán, sinh vé điện tử, gửi thông báo và check-in bằng mã QR. Ở mức thiết kế gói, biểu đồ tập trung vào năm nhóm thành phần chính là `api`, `middleware`, `controller`, `service` và `model`. Mỗi nhóm được phân công một vai trò rõ ràng nhằm tránh chồng chéo trách nhiệm và giúp luồng xử lý nghiệp vụ có thể theo dõi được từ đầu tới cuối.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.2: Biểu đồ thiết kế gói cho nghiệp vụ đặt vé, thanh toán và tham dự sự kiện

Trong thiết kế này, package `api` gồm các lớp như `TicketRoutes` và `PaymentRoutes`, chịu trách nhiệm tiếp nhận yêu cầu từ phía ứng dụng khách và chuyển yêu cầu sang tầng điều phối. Package `middleware` chứa lớp `AuthMiddleware`, được dùng để kiểm tra token xác thực và bảo vệ các chức năng chỉ dành cho người dùng đã đăng nhập. Package `controller` bao gồm các lớp như `TicketController`, `PaymentController` và `NotificationController`. Đây là nơi tổ chức luồng nghiệp vụ chính, chẳng hạn kiểm tra loại vé, khởi tạo giao dịch, xác nhận thanh toán, gọi dịch vụ sinh mã QR, cập nhật trạng thái vé và gửi thông báo sau giao dịch. Package `service` bao gồm các dịch vụ hỗ trợ như `QRCodeService` và `PushNotificationService`, cùng với các interface tương ứng như `IQRCodeGenerator` và `INotificationSender`. Việc đưa interface vào thiết kế giúp tách biệt giữa phần hợp đồng chức năng và phần hiện thực cụ thể, thuận lợi cho việc thay thế hoặc mở rộng dịch vụ về sau. Package `model` bao gồm các lớp dữ liệu chính như `Event`, `Ticket`, `Payment`, `Notification`, `User` và `TicketTier`, cùng lớp cơ sở trừu tượng `BaseDocument`.

Các quan hệ giữa các lớp trong biểu đồ gói thể hiện rõ cách vận hành của hệ thống. Quan hệ phụ thuộc (dependency) xuất hiện khi các lớp route sử dụng mid-

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

dleware và controller, hoặc khi controller cần gọi service và truy cập model để xử lý nghiệp vụ. Quan hệ thực thi (implementation) xuất hiện ở chỗ `QRCodeService` hiện thực `IQRCodeGenerator` và `PushNotificationService` hiện thực `INotificationSender`. Quan hệ kế thừa (inheritance) được dùng để mô tả việc các model nghiệp vụ kế thừa từ lớp cơ sở `BaseDocument`. Quan hệ kết tập (aggregation) có thể thấy trong trường hợp `Payment` tập hợp nhiều đối tượng `Ticket` thuộc cùng một giao dịch, trong khi quan hệ hợp thành (composition) thể hiện ở việc `Event` bao gồm nhiều `TicketTier` như một phần cấu thành của sự kiện. Ngoài ra, quan hệ kết hợp (association) được dùng để phản ánh các mối liên hệ nghiệp vụ như mỗi `Ticket` gắn với một `User` và một `Event`, hoặc một `Notification` được gửi tới một `User` và có thể liên quan tới một `Event` cụ thể.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Hệ thống được định hướng là một ứng dụng mobile-first, do đó thiết kế giao diện tập trung chủ yếu vào môi trường điện thoại thông minh và các thiết bị di động cầm tay. Về mặt mục tiêu hiển thị, ứng dụng hướng tới các màn hình có độ phân giải phổ biến trên smartphone hiện nay, hỗ trợ tốt ở cả các thiết bị cỡ nhỏ, thiết bị kích thước trung bình và máy tính bảng. Trong mã nguồn hiện tại, nhiều màn hình như trang chủ, trang chi tiết sự kiện, màn hình chọn vé hay màn hình chat đều sử dụng cơ chế `useWindowDimensions` để điều chỉnh giao diện theo chiều rộng thực tế của thiết bị. Điều này cho thấy thiết kế giao diện của hệ thống không cố định ở một kích thước duy nhất, mà hướng tới khả năng thích nghi với nhiều khung hiển thị. Trên thiết bị nhỏ, khoảng cách lề, chiều cao ô nhập liệu, kích thước biểu tượng và cỡ chữ được thu gọn để bảo đảm không tràn bố cục; trong khi với máy tính bảng, các thành phần được mở rộng hợp lý để tận dụng không gian hiển thị.

Về số lượng màu sắc và định hướng hiển thị, hệ thống sử dụng môi trường giao diện đồ họa tiêu chuẩn của thiết bị di động với dải màu đầy đủ, hỗ trợ hiển thị văn bản, ảnh, icon, nền khối, gradient và phản hồi trạng thái bằng màu. Trong mã nguồn, bảng màu của ứng dụng được chuẩn hóa tập trung trong các tệp `frontend/constants/colors.ts` và `frontend/constants/theme.ts`. Màu chủ đạo của hệ thống là cam `#F76B10`, được dùng để nhấn mạnh các thao tác chính như đặt vé, xác nhận hoặc điều hướng tới các chức năng nổi bật. Bên cạnh đó, hệ thống sử dụng thêm các màu phụ như xanh dương, vàng, xanh lá, trắng, xám và các màu trạng thái để biểu diễn thông tin khác nhau. Cách tổ chức này giúp giao diện có tính thống nhất cao và hỗ trợ việc thay đổi chủ đề hoặc mở rộng sang dark mode về sau.

Nguyên tắc tổ chức giao diện

Thiết kế giao diện của hệ thống được xây dựng theo hướng ưu tiên khả năng thao tác nhanh, dễ hiểu và phù hợp với hành vi sử dụng trên điện thoại. Các màn hình nghiệp vụ chính đều tuân theo cấu trúc ba phần: vùng điều hướng phía trên, vùng nội dung trung tâm và vùng thao tác chính ở cuối màn hình hoặc tại những vị trí dễ chạm. Ví dụ, trang chủ gồm phần tiêu đề tài khoản và vị trí người dùng, thanh tìm kiếm, các nhóm danh mục và danh sách sự kiện; màn hình chi tiết sự kiện tổ chức nội dung theo trình tự banner – thông tin sự kiện – thao tác chính; còn màn hình mua vé và thanh toán được thiết kế theo luồng nhập liệu từng bước. Việc phân chia như vậy giúp người dùng nhanh chóng nhận biết phần nào dùng để xem thông tin, phần nào dùng để thao tác và phần nào là phản hồi của hệ thống.

Các thành phần giao diện cũng được chuẩn hóa theo một số quy tắc nhất quán. Nút thao tác chính thường dùng nền đậm hoặc màu nhấn, bo góc lớn và chữ đậm để tạo sự nổi bật. Các ô nhập liệu sử dụng nền sáng, viền rõ ràng, khoảng đệm tương đối lớn để dễ nhập trên màn hình cảm ứng. Các danh sách, thẻ sự kiện và vùng thông tin dùng bo góc mềm để giữ tính nhất quán với tổng thể thiết kế hiện đại của ứng dụng. Với các chức năng có tính hệ quả trực tiếp như thanh toán, xóa hoặc từ chối sự kiện, giao diện luôn cố gắng cung cấp dấu hiệu nhận biết rõ ràng thông qua màu sắc, vị trí nút và thông báo phản hồi. Hệ thống cũng sử dụng icon phổ biến như menu, tìm kiếm, vị trí, tin nhắn, vé và các trạng thái tăng giảm số lượng để giảm khối lượng văn bản phải hiển thị trên màn hình.

Chuẩn hóa vị trí thông điệp và phản hồi hệ thống

Một yêu cầu quan trọng của ứng dụng di động là thông điệp phản hồi phải xuất hiện đúng lúc và đủ rõ để người dùng không bị mất phương hướng khi thao tác. Trong thiết kế hiện tại, thông điệp phản hồi được tổ chức theo ba mức. Mức thứ nhất là phản hồi cục bộ ngay trong thành phần giao diện, ví dụ trạng thái nút bị mờ khi không thể bấm, số lượng vé không thể tăng vượt quota hoặc biểu tượng thay đổi theo trạng thái đã lưu. Mức thứ hai là phản hồi qua các hộp thoại cảnh báo hoặc thông báo ngăn khi thao tác thành công hay thất bại, chẳng hạn khi thêm thẻ, duyệt sự kiện hoặc gặp lỗi kết nối. Mức thứ ba là phản hồi ở mức hệ thống thông qua notification và push notification, áp dụng cho các tình huống như lời mời sự kiện, nhắc lịch, thanh toán thành công hoặc có tin nhắn mới. Việc phân tách phản hồi như vậy giúp cho người dùng luôn nhận được thông tin phù hợp với mức độ quan trọng của thao tác.

Định hướng thiết kế cho các màn hình quan trọng

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Đối với màn hình trang chủ, định hướng thiết kế là ưu tiên khám phá nhanh. Người dùng cần nhìn thấy ngay thông tin cá nhân tối thiểu, thanh tìm kiếm, danh mục sự kiện và các sự kiện nổi bật. Vì vậy, bố cục được tổ chức theo chiều dọc, dùng thẻ sự kiện và các thanh danh mục nằm ngang để giảm thao tác cuộn không cần thiết. Đối với màn hình chi tiết sự kiện, thiết kế hướng tới việc cung cấp thông tin đầy đủ nhưng không gây quá tải, nên banner hình ảnh được đặt ở đầu trang, sau đó là khối thông tin tóm tắt, mô tả, thông tin người tổ chức và các hành động như đặt vé, nhắn tin hoặc lưu sự kiện. Đối với màn hình mua vé, thiết kế cần làm nổi bật việc lựa chọn hạng vé, số lượng và tổng tiền để người dùng dễ ra quyết định. Còn đối với màn hình chat, thiết kế được tổ chức theo mẫu hội thoại quen thuộc với khung tin nhắn, thời gian gửi, ô nhập liệu và phản hồi đọc tin nhằm giảm thời gian làm quen của người sử dụng.

Nhận xét chung về thiết kế giao diện

Thiết kế giao diện của hệ thống được xây dựng theo hướng thực dụng, ưu tiên tính dễ dùng và sự nhất quán hơn là những hiệu ứng trình diễn phức tạp. Điểm mạnh của thiết kế nằm ở việc chuẩn hóa tương đối rõ ràng màu, kiểu chữ, vùng nhập liệu, biểu tượng và phản hồi hệ thống, đồng thời có xem xét tới khả năng thích nghi với nhiều kích thước màn hình khác nhau. Dù đây mới là một sản phẩm ở mức thử nghiệm, định hướng giao diện hiện tại đã đủ để làm nền tảng cho các luồng nghiệp vụ chính như khám phá sự kiện, đặt vé, thanh toán, quản lý vé và tương tác giữa các bên. Trong các bước tiếp theo, phần minh họa bằng ảnh giao diện thực tế có thể tiếp tục được bổ sung để làm rõ hơn cách những nguyên tắc thiết kế này được hiện thực hóa trên từng màn hình cụ thể.

Hình ảnh Wireframe 1 số giao diện:

9:41

← Tạo sự kiện

Ảnh đại diện

Chọn ảnh

Tiêu đề sự kiện

Workshop Mobile Event App

Danh mục

Music

Ngày tổ chức

2026-07-15

Thời gian

19:30

Địa điểm

Hà Nội

Mô tả

Mô tả ngắn về sự kiện...

Hạng vé

Hạng vé	Giá vé	Quota	
Standard	100000	100	🗑️
VIP	250000	30	🗑️

+ Thêm hạng vé

Gợi ý nội dung bằng AI

Lưu nháp

Gửi duyệt

Hình 4.3: Wireframe minh họa giao diện chức năng tạo sự kiện

9:41

← Đặt vé

Sự kiện Mobile Event Booking Workshop

Địa điểm Hà Nội

Thời gian 19:30 - 15/07/2026

Loại vé	Giá	Còn lại	Chọn
Standard	100000	100	⊗
VIP	250000	30	○

Số lượng - 2 +

Thành tiền 200000 VNĐ

Phương thức thanh toán Wallet

Mã giảm giá

✓ Xác nhận thanh toán

Hình 4.4: Wireframe minh họa giao diện chức năng đặt vé

4.2.2 Thiết kế lớp

Trong hệ thống đặt vé và quản lý sự kiện của đồ án, có nhiều lớp dữ liệu và lớp xử lý cùng phối hợp để thực hiện các nghiệp vụ. Tuy nhiên, nếu xét theo mức độ trung tâm đối với luồng nghiệp vụ chính, bốn lớp quan trọng nhất là `User`, `Event`, `Ticket` và `Payment`. Đây là các lớp phản ánh trực tiếp ba thực thể cốt lõi của hệ thống là người dùng, sự kiện và giao dịch đặt vé. Những lớp khác như `Notification`, `Bookmark`, `ChatRoom`, `Message` hay các lớp tiện ích hỗ trợ sẽ được xem là các lớp bổ trợ.

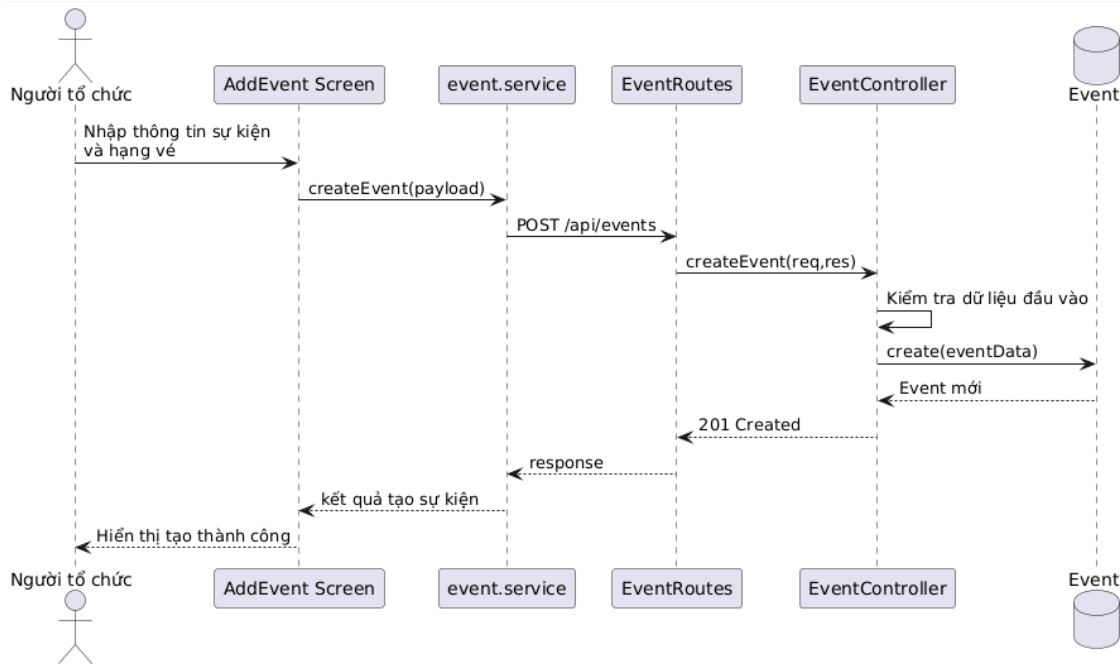
Lớp User

Lớp `User` đại diện cho người sử dụng hệ thống, bao gồm cả người dùng thông thường lẫn quản trị viên. Về mặt thuộc tính, lớp này lưu trữ các thông tin định danh và hồ sơ như `name`, `email`, `password`, `avatar`, `country`, `interests`, `location`, `description`, cùng với các thông tin điều khiển nghiệp vụ như `role`, `verified`, `notificationsEnabled`, `expoPushToken`, `passwordResetCode` và `passwordResetExpires`. Các thuộc tính này cho phép hệ thống không chỉ xác thực người dùng mà còn cá nhân hóa hồ sơ, quản lý quyền truy cập và hỗ trợ các chức năng thông báo đẩy hoặc khôi phục mật khẩu.

Về phương thức, lớp `User` có phương thức quan trọng `matchPassword()` để đối chiếu mật khẩu người dùng nhập vào với mật khẩu đã mã hóa trong cơ sở dữ liệu. Ngoài ra, lớp còn tham gia vào cơ chế tiền xử lý trước khi lưu thông qua thao tác mã hóa mật khẩu bằng `pre-save hook`. Về ý nghĩa thiết kế, `User` là lớp đầu mối cho hầu hết các use case của hệ thống vì mọi nghiệp vụ như đặt vé, tạo sự kiện, nhận thông báo hay nhắn tin đều gắn với một người dùng cụ thể.

Lớp Event

Lớp `Event` đại diện cho một sự kiện được công bố trên hệ thống. Các thuộc tính chính của lớp bao gồm `title`, `description`, `category`, `price`, `date`, `time`, `location`, `coordinates`, `images`, `member`, `attendees`, `rating`, `status`, `approvalStatus`, `organizer` và `ticketTiers`. Trong đó, `organizer` liên kết sự kiện với người tạo sự kiện, còn `ticketTiers` cho phép một sự kiện có nhiều hạng vé khác nhau, mỗi hạng có tên, giá, quota và số lượng đã bán.



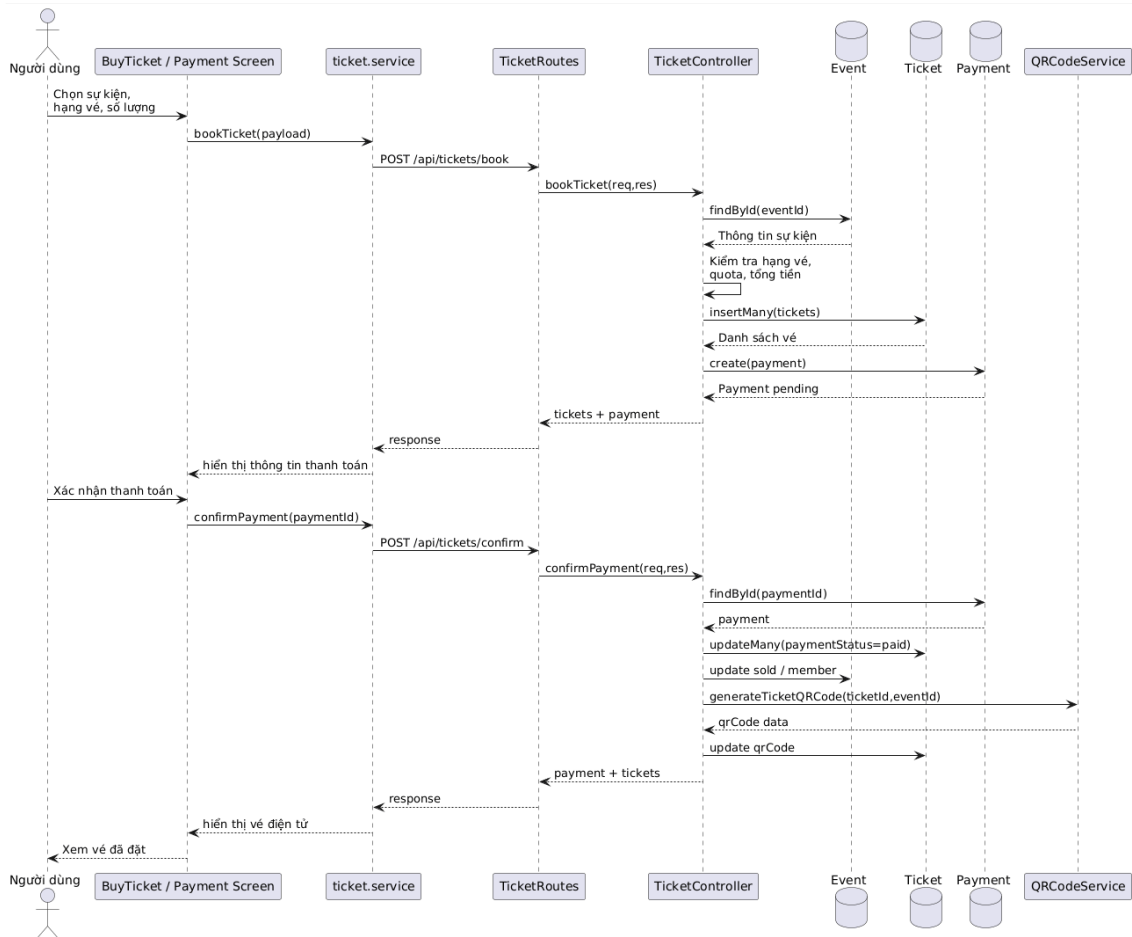
Hình 4.5: Biểu đồ trình tự cho use case Tạo sự kiện

Về mặt phương thức, lớp `Event` không biểu diễn các phương thức nghiệp vụ riêng biệt theo kiểu lớp hướng đối tượng thuần túy, nhưng nó là đối tượng được thao tác xuyên suốt trong các controller như tạo sự kiện, cập nhật sự kiện, duyệt sự kiện, thống kê người tham gia và hiển thị chi tiết sự kiện. Ý nghĩa thiết kế của lớp này nằm ở chỗ nó là trung tâm của phần nghiệp vụ sự kiện: mọi thao tác từ tìm kiếm, đặt vé, thông báo cho tới kiểm soát check-in đều quy chiếu về một đối tượng sự kiện cụ thể.

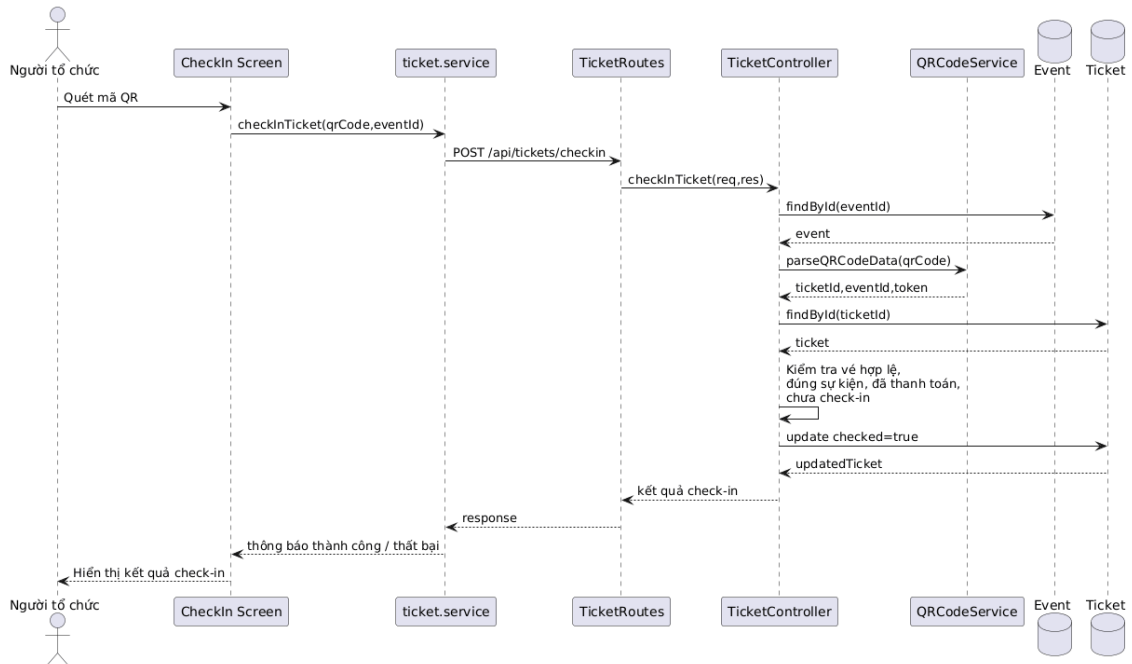
Lớp Ticket

Lớp `Ticket` đại diện cho vé điện tử của người dùng đối với một sự kiện cụ thể. Các thuộc tính của lớp gồm `user`, `event`, `price`, `ticketType`, `tierName`, `seatInfo`, `qrCode`, `paymentStatus`, `checked`, `checkedAt`, `checkedBy` và `bookedAt`. Nhờ những thuộc tính này, một vé không chỉ là minh chứng cho việc mua thành công, mà còn là thực thể phản ánh trạng thái thanh toán, dữ liệu QR và trạng thái tham dự tại sự kiện.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.6: Biểu đồ trình tự cho use case Đặt vé và xác nhận thanh toán



Hình 4.7: Biểu đồ trình tự cho use case Check-in vé bằng mã QR

Ở góc độ nghiệp vụ, lớp `Ticket` tham gia vào các thao tác như khởi tạo vé khi

người dùng đặt chỗ, cập nhật trạng thái thanh toán khi giao dịch được xác nhận, lưu dữ liệu QR để phục vụ việc check-in và cập nhật trạng thái đã sử dụng sau khi quét vé thành công. Đây là lớp có vai trò cầu nối giữa người dùng và sự kiện, bởi nó là thực thể cụ thể hóa mối quan hệ “người dùng tham dự một sự kiện nào đó thông qua một hạng vé xác định”.

Lớp Payment

Lớp Payment biểu diễn giao dịch thanh toán gắn với một hoặc nhiều vé trong hệ thống. Các thuộc tính chính gồm `user`, `event`, `tickets`, `amount`, `method`, `status` và `transactionId`. Cấu trúc này cho phép hệ thống quản lý thanh toán như một thực thể tách biệt thay vì chỉ lưu trực tiếp trạng thái tiền trên từng vé. Điều đó giúp việc truy vết giao dịch, cập nhật trạng thái thanh toán, thống kê doanh thu hoặc tích hợp cổng thanh toán thực tế trong tương lai thuận lợi hơn.

Về phương thức nghiệp vụ, lớp Payment được thao tác chủ yếu trong luồng đặt vé và xác nhận thanh toán. Khi người dùng chọn hạng vé và số lượng vé, hệ thống tạo một giao dịch ở trạng thái chờ xử lý. Sau khi xác nhận thành công, Payment được cập nhật sang trạng thái thành công, đồng thời kéo theo việc cập nhật trạng thái các vé liên quan và số lượng vé đã bán ở sự kiện tương ứng. Vì vậy, lớp này đóng vai trò như một lớp điều phối tài chính, bảo đảm sự nhất quán giữa vé, số lượng vé đã bán và trạng thái giao dịch.

Nhận xét chung về thiết kế lớp

Bốn lớp trên phản ánh lõi nghiệp vụ của hệ thống theo một cấu trúc khá mạch lạc. `User` biểu diễn tác nhân sử dụng hệ thống, `Event` biểu diễn đối tượng trung tâm mà người dùng quan tâm, `Ticket` biểu diễn quyền tham dự của người dùng đối với sự kiện, còn `Payment` biểu diễn giao dịch tài chính đi kèm với việc phát hành vé. Mối liên hệ giữa các lớp này tạo nên phần lớn luồng nghiệp vụ chính trong hệ thống. Việc thiết kế lớp theo hướng tách biệt vai trò như vậy giúp mã nguồn dễ mở rộng hơn, chẳng hạn khi cần bổ sung thêm các chức năng như hủy vé, hoàn tiền, thống kê doanh thu hay xếp chỗ theo khu vực.

4.2.3 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng MongoDB làm hệ quản trị cơ sở dữ liệu, vì vậy mô hình dữ liệu của đề tài được thiết kế theo hướng tài liệu (document-oriented) thay vì quan hệ thuần túy như trong các hệ quản trị Structured Query Language (SQL). Tuy nhiên, về bản chất nghiệp vụ, các collection trong MongoDB vẫn biểu diễn những thực thể và quan hệ tương tự một biểu đồ thực thể liên kết (Entity Rela-

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

tionship Diagram - ERD). Do đó, việc thiết kế cơ sở dữ liệu vẫn cần bắt đầu từ việc xác định các thực thể chính, các thuộc tính quan trọng và mối liên hệ giữa chúng. Trên cơ sở đó, hệ thống được tổ chức thành các collection trung tâm như User, Event, Ticket, Payment, Notification, Bookmark, Reminder, Review, ChatRoom, Message và Card.

Thực thể User

Thực thể User lưu thông tin người dùng của hệ thống, bao gồm cả người dùng thông thường và quản trị viên. Các thuộc tính quan trọng của thực thể này gồm họ tên, email, mật khẩu đã mã hóa, avatar, quốc gia, sở thích, vị trí, mô tả hồ sơ, vai trò, trạng thái xác thực, tùy chọn nhận thông báo và token thiết bị phục vụ thông báo đẩy. Đây là thực thể trung tâm của hệ thống vì hầu hết các nghiệp vụ đều quy chiếu về người dùng, từ tạo sự kiện, đặt vé, gửi tin nhắn, nhận thông báo cho đến quản lý bookmark và lời nhắc.

Thực thể Event

Thực thể Event mô tả một sự kiện được tạo trong hệ thống. Các thuộc tính chính gồm tiêu đề, mô tả, danh mục, giá mặc định, ngày, thời gian, địa điểm, tọa độ, hình ảnh, số lượng người tham gia, điểm đánh giá, trạng thái sự kiện và trạng thái duyệt. Mỗi sự kiện liên kết với một người tổ chức thông qua thuộc tính organizer. Ngoài ra, sự kiện còn chứa một cấu trúc lồng ticketTiers để mô tả các hạng vé, trong đó mỗi hạng vé có tên, giá, quota và số lượng vé đã bán. Việc đặt ticketTiers bên trong tài liệu sự kiện là một lựa chọn phù hợp với MongoDB vì các hạng vé luôn gắn chặt với một sự kiện cụ thể và thường được truy xuất cùng nhau.

Thực thể Ticket và Payment

Thực thể Ticket biểu diễn vé điện tử mà người dùng nhận được khi đặt tham gia một sự kiện. Mỗi vé liên kết với một người dùng và một sự kiện, đồng thời lưu giá vé, loại vé, tên hạng vé, thông tin vị trí ghế nếu có, dữ liệu mã QR, trạng thái thanh toán, trạng thái check-in và thời điểm đặt vé. Đây là thực thể cầu nối giữa người dùng với sự kiện, phản ánh trực tiếp việc một người dùng tham gia một sự kiện nào đó qua một hạng vé xác định.

Thực thể Payment dùng để quản lý giao dịch thanh toán. Mỗi thanh toán liên kết với một người dùng, một sự kiện và một tập vé tương ứng. Các thuộc tính chính của nó gồm tổng số tiền, phương thức thanh toán, trạng thái giao dịch và mã giao dịch. Việc tách Payment thành một thực thể riêng giúp hệ thống dễ mở rộng sang

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

các kịch bản thực tế hơn như hoàn tiền, theo dõi giao dịch, thống kê doanh thu hoặc tích hợp cổng thanh toán thật trong tương lai.

Các thực thể hỗ trợ tương tác và theo dõi

Bên cạnh các thực thể cốt lõi, hệ thống còn có nhiều thực thể hỗ trợ cho trải nghiệm người dùng. Thực thể `Bookmark` lưu mối liên hệ giữa người dùng và sự kiện được lưu quan tâm, qua đó hỗ trợ tính năng theo dõi sự kiện. Thực thể `Reminder` lưu các lời nhắc gắn giữa người dùng và sự kiện, giúp hệ thống xác định thời điểm cần gửi nhắc lịch. Thực thể `Review` lưu đánh giá và bình luận của người dùng cho sự kiện, phục vụ tính năng phản hồi và cập nhật điểm đánh giá trung bình. Thực thể `Notification` lưu các thông báo nội bộ mà người dùng nhận được, bao gồm thông báo hệ thống, lời mời, nhắc lịch, thông báo thanh toán hoặc thông báo chat.

Ngoài ra, nhóm chức năng giao tiếp thời gian thực được tổ chức thông qua hai thực thể `ChatRoom` và `Message`. `ChatRoom` lưu thông tin một phòng chat, có thể gắn với một sự kiện và một danh sách thành viên. `Message` lưu các tin nhắn phát sinh trong từng phòng, bao gồm người gửi, nội dung, loại tin nhắn, tệp đính kèm và danh sách người đã đọc. Cuối cùng, thực thể `Card` dùng để lưu thông tin thẻ thanh toán đã lưu của người dùng ở mức tối giản, bao gồm thương hiệu thẻ, bốn số cuối, tháng hết hạn, năm hết hạn và trạng thái thẻ mặc định.

Mối quan hệ giữa các thực thể

Từ góc nhìn E-R, mối quan hệ giữa các thực thể trong hệ thống có thể mô tả như sau. Một `User` có thể tạo nhiều `Event`, trong khi mỗi `Event` chỉ có một người tổ chức chính. Một `User` có thể sở hữu nhiều `Ticket`, còn mỗi `Ticket` chỉ thuộc về một người dùng và một sự kiện. Một `Payment` có thể liên kết với nhiều `Ticket`, nhưng mỗi vé trong ngữ cảnh hệ thống hiện tại chỉ gắn với một giao dịch thanh toán tại một thời điểm. Một `User` có thể tạo nhiều `Bookmark`, `Reminder`, `Review`, `Notification`, `Card` và `Message`. Một `Event` có thể có nhiều `Ticket`, `Review`, `Notification`, `Reminder` và `Bookmark`. Đối với nhóm chat, một `ChatRoom` có thể chứa nhiều `Message` và nhiều thành viên là các `User`, còn mỗi `Message` thuộc về đúng một phòng chat và một người gửi.

Đặc điểm thiết kế dữ liệu trong MongoDB

Do sử dụng MongoDB, các quan hệ trên không được hiện thực bằng khóa ngoại như trong hệ quản trị quan hệ, mà chủ yếu thông qua các trường tham chiếu `ObjectId` và các cấu trúc lồng. Cách làm này giúp hệ thống vừa giữ được tính linh

hoạt khi mô hình dữ liệu thay đổi, vừa đủ rõ ràng để tổ chức các nghiệp vụ phức tạp. Những quan hệ cần truy xuất thường xuyên cùng nhau, chẳng hạn các hạng vé của một sự kiện, được nhúng trực tiếp trong tài liệu Event. Ngược lại, các thực thể có vòng đời riêng như Ticket, Payment, Notification hay Message được tách thành collection riêng để hỗ trợ cập nhật độc lập và mở rộng tốt hơn.

Thiết kế cơ sở dữ liệu của hệ thống thể hiện rõ đặc điểm của một ứng dụng sự kiện có nhiều nghiệp vụ đan xen. Cấu trúc dữ liệu hiện tại đáp ứng được các chức năng cốt lõi như quản lý người dùng, quản lý sự kiện, đặt vé, thanh toán, thông báo, nhắc lịch, nhắc tin và đánh giá. Đồng thời, việc kết hợp giữa tài liệu lỏng và tham chiếu đối tượng giúp mô hình dữ liệu vừa giữ được sự linh hoạt của NoSQL, vừa bảo đảm khả năng mở rộng khi hệ thống cần bổ sung thêm các chức năng như hoàn tiền, báo cáo thống kê, đề xuất sự kiện cá nhân hóa hoặc quản lý sơ đồ chỗ ngồi phức tạp hơn.



Trong quá trình phát triển hệ thống, đề tài sử dụng kết hợp nhiều công cụ, ngôn ngữ lập trình, thư viện và framework ở cả phía frontend lẫn backend. Việc lựa chọn

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

các thành phần này dựa trên yêu cầu xây dựng ứng dụng di động đa nền tảng, backend xử lý nghiệp vụ tập trung, cơ sở dữ liệu linh hoạt và các chức năng hỗ trợ như bản đồ, thông báo đẩy, chat thời gian thực, mã QR và kiểm thử tự động. Do số lượng công cụ sử dụng tương đối nhiều, nội dung được chia thành hai bảng để thuận tiện theo dõi trong bố cục trang dọc.

Nhóm/Mục đích	Công cụ/Thư viện	Phiên bản	Vai trò sử dụng
Ngôn ngữ lập trình frontend	TypeScript	5.9.2	Phát triển ứng dụng khách với kiểu dữ liệu tĩnh, thuận tiện bảo trì và tổ chức service.
Framework mobile	Expo	54.0.33	Cung cấp nền tảng phát triển mobile-first, hỗ trợ chạy thử nhanh trên Android, iOS và web.
Framework giao diện mobile	React Native	0.81.5	Xây dựng giao diện đa nền tảng cho điện thoại và tablet.
Thư viện giao diện lõi	React	19.1.0	Quản lý component, trạng thái và vòng đời hiển thị ở frontend.
Điều hướng ứng dụng	React Navigation	7.1.18 – 7.8.5	Tổ chức điều hướng giữa các màn hình đăng nhập, sự kiện, thanh toán, chat và quản trị.
Giao diện và theme	NativeWind + Tailwind CSS + React Native Paper	4.2.1 / 3.4.18 / 5.14.5	Chuẩn hóa màu sắc, khoảng cách, kiểu component và theme cho toàn ứng dụng.
Giao tiếp API	Axios	1.13.2	Gửi yêu cầu HTTP từ frontend tới backend.
Thông báo đẩy	Expo Notifications + Expo Device	0.32.16 / 8.0.10	Gửi và nhận push notification cho thanh toán, lời mời, nhắc lịch và tin nhắn mới.
Bản đồ và vị trí	react-native-maps + expo-location + Leaflet + react-native-web	1.20.1 / 19.0.8 / 1.9.4 / 0.21.0	Hỗ trợ chọn vị trí và triển khai bản đồ tương thích cho mobile và web.
Camera và mã QR	expo-camera + react-native-qrcode-svg	17.0.10 / 6.3.21	Quét mã QR và hiển thị vé điện tử ở phía người dùng.
Hình ảnh và media	expo-image-picker	17.0.10	Chọn ảnh cho sự kiện hoặc hồ sơ người dùng.
Lưu trữ cục bộ	@react-native-async-storage/async-storage	2.2.0	Lưu token và thông tin đăng nhập ở thiết bị người dùng.
Thư viện tiện ích	dayjs + uuid	1.11.19 / 11.1.0	Định dạng thời gian và sinh định danh hỗ trợ cho các thao tác phụ trợ.

Bảng 4.1: Danh sách công cụ và thư viện sử dụng ở phía frontend của hệ thống

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Nhóm/Mục đích	Công cụ/Thư viện	Phiên bản	Vai trò sử dụng
Ngôn ngữ lập trình backend	TypeScript	5.9.3	Xây dựng mã nguồn backend có kiểu dữ liệu tĩnh, dễ bảo trì và mở rộng.
Môi trường chạy backend	Node.js LTS	v18+	Chạy server Express, scheduler và Socket.IO cho hệ thống.
Framework web backend	Express	5.1.0	Xây dựng các route và controller cho các API nghiệp vụ.
Kết nối dữ liệu	Mongoose	8.19.1	Kết nối MongoDB, định nghĩa schema và thao tác với dữ liệu nghiệp vụ.
Hệ quản trị cơ sở dữ liệu	MongoDB	8.x ecosystem	Lưu trữ người dùng, sự kiện, vé, thanh toán, thông báo, chatroom và message theo mô hình NoSQL.
Xác thực và bảo mật	JSON Web Token + bcryptjs + helmet + cors + express-validator	9.0.2 / 3.0.2 / 7.0.0 / 2.8.5 / 7.2.1	Xác thực người dùng, mã hóa mật khẩu, bảo vệ header HTTP và kiểm tra dữ liệu đầu vào.
Thời gian thực	Socket.IO	4.8.3	Hỗ trợ chat thời gian thực và cập nhật phòng trò chuyện giữa các bên tham gia.
Email hệ thống	nodemailer	7.0.9	Gửi email khôi phục mật khẩu và các thông báo liên quan nếu cấu hình đầy đủ.
Sinh mã QR	qrcode	1.5.4	Sinh dữ liệu và ảnh mã QR cho vé điện tử ở phía backend.
Gọi dịch vụ ngoài	Axios	1.13.2	Kết nối tới các dịch vụ ngoài như AI API hoặc các endpoint tích hợp.
Công cụ phát triển backend	ts-node-dev	2.0.0	Chạy server TypeScript ở chế độ phát triển với khả năng reload khi sửa mã nguồn.
Công cụ build frontend	babel-preset-expo	54.0.10	Hỗ trợ quá trình biên dịch và đóng gói ứng dụng Expo.
Công cụ kiểm thử	Jest + ts-jest	30.2.0 / 29.4.5	Viết và chạy kiểm thử đơn vị cho controller, helper và nghiệp vụ backend.
Công cụ sinh dữ liệu mẫu	seed script	Tự xây dựng	Tạo dữ liệu thử nghiệm cho MongoDB để phục vụ kiểm thử thủ công và trình diễn.

Bảng 4.2: Danh sách công cụ, thư viện và môi trường phát triển ở phía backend của hệ thống

4.3.2 Kết quả đạt được

Kết quả đạt được của đề án không chỉ dừng ở mức hoàn thiện mã nguồn, mà đã hình thành một hệ thống phần mềm tương đối hoàn chỉnh cho bài toán đặt vé và quản lý sự kiện trên thiết bị di động. Sản phẩm chính của đề án gồm ba thành phần cốt lõi. Thành phần thứ nhất là ứng dụng khách được phát triển bằng React Native và Expo, chịu trách nhiệm cung cấp giao diện tương tác cho người dùng trên điện thoại và trên môi trường web phục vụ thử nghiệm. Thành phần thứ hai là máy chủ backend được xây dựng bằng Node.js, Express và TypeScript, đảm nhiệm xử lý các nghiệp vụ như xác thực người dùng, quản lý sự kiện, đặt vé, thanh toán, gửi thông báo, nhắc lịch và chat thời gian thực. Thành phần thứ ba là mô hình dữ liệu MongoDB cùng hệ thống schema Mongoose, tạo nền tảng lưu trữ và quản lý các thực thể như người dùng, sự kiện, vé, thanh toán, thông báo, bookmark, reminder, review và chat.

Ngoài ba thành phần cốt lõi trên, đề án còn tạo ra một số sản phẩm hỗ trợ quan trọng. Một là bộ dữ liệu mẫu phục vụ thử nghiệm thủ công và trình diễn hệ thống thông qua script seed cho backend. Hai là bộ kiểm thử tự động ở mức controller

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

cho các nhóm chức năng quan trọng như xác thực người dùng, tạo sự kiện và đặt vé. Ba là tập các tài liệu thiết kế và minh họa như biểu đồ use case, biểu đồ gói, biểu đồ trình tự, wireframe giao diện và biểu đồ E-R, giúp mô tả toàn bộ hệ thống từ góc nhìn phân tích, thiết kế đến triển khai. Nhờ đó, kết quả đạt được của đồ án không chỉ là một sản phẩm chạy được, mà còn là một bộ artefact kỹ thuật khá đầy đủ phục vụ cho việc bảo trì, mở rộng và đánh giá hệ thống.

Về mặt ý nghĩa, các sản phẩm trên cho thấy hệ thống đã đáp ứng được các luồng nghiệp vụ quan trọng nhất của bài toán. Người dùng có thể đăng nhập, tìm kiếm sự kiện, xem chi tiết, lưu sự kiện, đặt vé, nhận vé điện tử và nhận thông báo. Người tổ chức có thể tạo sự kiện, quản lý hạng vé, mời người tham gia và thực hiện check-in bằng mã QR. Quản trị viên có thể kiểm soát trạng thái duyệt sự kiện. Đồng thời, việc hệ thống đã có bản build cho backend, bản export web cho frontend và dữ liệu seed cho thấy sản phẩm không chỉ tồn tại ở mức mã nguồn mà đã có thể phục vụ trình diễn và kiểm thử trong môi trường gần với thực tế hơn.

Do dự án được tổ chức chủ yếu theo module và hàm nghiệp vụ, thay vì hoàn toàn theo cấu trúc lớp thuần túy, phần thống kê dưới đây sử dụng số lượng mô-đun, thực thể dữ liệu và thành phần chính của hệ thống để phản ánh quy mô sản phẩm một cách sát thực hơn. Bảng 4.3 tóm tắt các sản phẩm đầu ra chính của đồ án, còn Bảng 4.4 trình bày các số liệu thống kê nổi bật của ứng dụng.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Sản phẩm	Thành phần	Ý nghĩa, vai trò
Ứng dụng khách	Mã nguồn frontend React Native/Expo, các màn hình, component, service và context	Cung cấp giao diện cho người dùng cuối, hiện thực các luồng khám phá sự kiện, đặt vé, quản lý vé, chat và thông báo.
Máy chủ backend	Mã nguồn Express/TypeScript, route, controller, middleware, service, utility	Xử lý nghiệp vụ trung tâm của hệ thống, bảo vệ API, quản lý dữ liệu và điều phối các chức năng thời gian thực.
Cơ sở dữ liệu	Các schema/model MongoDB-Mongoose cho User, Event, Ticket, Payment, Notification, Bookmark, Reminder, Review, ChatRoom, Message, Card	Lưu trữ dữ liệu nghiệp vụ và bảo đảm tính nhất quán cho các thao tác cốt lõi của hệ thống.
Bản build triển khai	backend/dist và frontend/dist	Cho phép đóng gói và triển khai sản phẩm trên môi trường chạy thử và môi trường web.
Dữ liệu thử nghiệm	Script sinh dữ liệu mẫu backend/script-s/seed.js	Hỗ trợ kiểm thử thủ công, trình diễn sản phẩm và đối chiếu dữ liệu khi phát triển.
Bộ kiểm thử	Các ca kiểm thử trong backend/tests	Hỗ trợ kiểm tra tính đúng đắn của một số controller quan trọng và giảm rủi ro hồi quy khi sửa mã nguồn.
Tài liệu thiết kế	Use case, package diagram, sequence diagram, wireframe, ERD	Hỗ trợ mô tả, đánh giá và mở rộng hệ thống ở các giai đoạn tiếp theo.

Bảng 4.3: Các sản phẩm đầu ra chính đạt được trong quá trình thực hiện đồ án

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chỉ số thống kê	Giá trị	Ghi chú
Tổng số dòng mã nguồn chính (không tính <code>node_modules</code> , <code>dist</code> , <code>.expo</code>)	20 372 dòng	Tính trên các tệp <code>.ts</code> , <code>.tsx</code> , <code>.js</code> của backend và frontend.
Số tệp mã nguồn phía backend (<code>backend/src</code>)	46 tệp	Bao gồm cấu hình, controller, middleware, model, route, service và utility.
Số tệp mã nguồn phía frontend	87 tệp	Tính trên các tệp TypeScript/TSX của frontend, không tính thư viện cài đặt.
Số màn hình nghiệp vụ frontend	35 màn hình	Tính theo số tệp trong thư mục <code>frontend/pages</code> .
Số component giao diện dùng lại	23 component	Tính theo số tệp trong <code>frontend/components</code> .
Số service phía frontend	15 service	Phục vụ gọi API, chat, thông báo, thanh toán, upload, v.v.
Số model/thực thể dữ liệu backend	11 model	Gồm User, Event, Ticket, Payment, Notification, Bookmark, Reminder, Review, ChatRoom, Message, Card.
Số controller backend	11 controller	Mỗi controller xử lý một nhóm nghiệp vụ chính của hệ thống.
Số route backend	11 route module	Tổ chức endpoint theo từng nhóm chức năng.
Số utility backend	6 utility module	Bao gồm sinh token, QR, push notification, email, scheduler, v.v.
Số tệp kiểm thử backend	4 tệp	Gồm các ca kiểm thử và helper phục vụ kiểm thử.
Dung lượng mã nguồn backend chính	256 KB	Thư mục <code>backend/src</code> .
Dung lượng kiểm thử và script seed	52 KB	Gồm <code>backend/tests</code> và <code>backend/scripts</code> .
Dung lượng phần giao diện frontend chính	472 KB	Thư mục <code>frontend/pages</code> .
Dung lượng component frontend	140 KB	Thư mục <code>frontend/components</code> .
Dung lượng service frontend	72 KB	Thư mục <code>frontend/services</code> .
Dung lượng tài nguyên frontend	272 KB	Thư mục <code>frontend/assets</code> , chưa tính phụ thuộc ngoài.
Dung lượng bản build backend	260 KB	Thư mục <code>backend/dist</code> .
Dung lượng bản export web frontend	8.2 MB	Thư mục <code>frontend/dist</code> , phục vụ triển khai thử nghiệm trên web.

Bảng 4.4: Một số thông tin thống kê về quy mô và kết quả hiện thực của hệ thống

4.3.3 Minh họa các chức năng chính

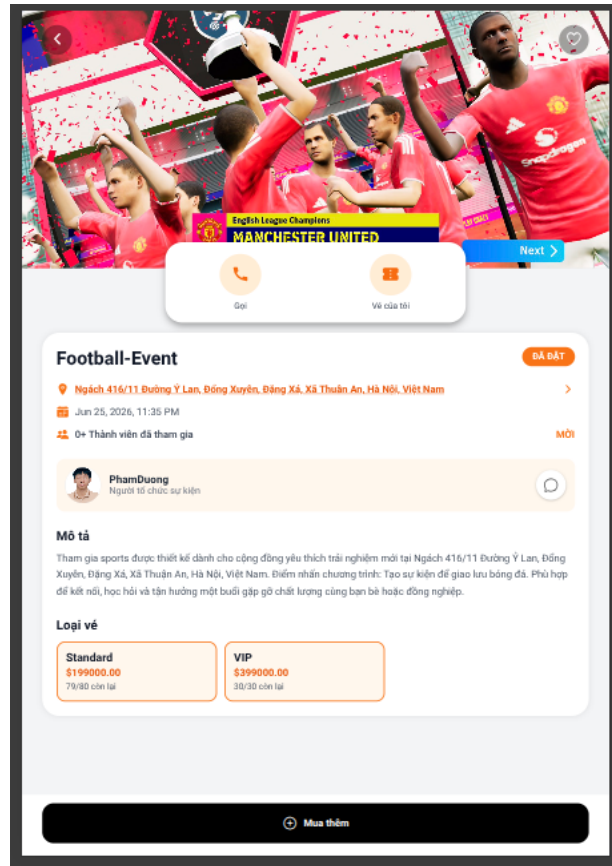
Để minh họa rõ hơn kết quả hiện thực của hệ thống, đề tài lựa chọn ba chức năng tiêu biểu nhất để đưa ra hình ảnh giao diện sản phẩm, bao gồm tạo sự kiện, đặt vé và check-in vé. Đây là ba chức năng đại diện cho ba góc nhìn quan trọng của hệ thống: phía người tổ chức, phía người tham dự và phía kiểm soát việc tham gia sự kiện. Các hình minh họa dưới đây là giao diện thực tế của sản phẩm sau khi hiện thực, không phải wireframe hay giao diện khái niệm. Vì vậy, chúng phản ánh trực tiếp cách các yêu cầu nghiệp vụ đã được chuyển hóa thành các màn hình tương tác trong hệ thống.

The screenshot displays the 'Tạo sự kiện mới' (Create New Event) screen. At the top, there's a large dashed box with a '+' icon and the text 'Thêm ảnh bìa' (Add cover image). Below this, the form includes several sections: 'Tên sự kiện *' (Event Name) with a text input field; 'Mô tả nhanh cho AI' (AI Quick Description) with a text input field and a placeholder example: 'Ví dụ: workshop cho người mới bắt đầu, phong cách trẻ trung, khoảng 100 khách, muốn có vé VIP và vé thường'; a button 'Gợi ý nội dung bằng AI' (Suggest content with AI); 'Loại sự kiện *' (Event Type) with a dropdown menu; 'Chọn ngày & giờ *' (Select date & time) with a date/time picker showing '25 Jun 2026 • 09:56'; 'Địa điểm *' (Location) with a text input field and a location pin icon; 'Loại vé *' (Ticket Type) with a dropdown menu and a '+ Thêm loại vé' (Add ticket type) button; and 'Mô tả sự kiện *' (Event Description) with a text input field. The bottom navigation bar contains four icons: a home icon, a calendar icon, a central '+' button, and a user profile icon.

Hình 4.9: Giao diện chức năng tạo sự kiện

Hình 4.9 minh họa giao diện tạo sự kiện dành cho người tổ chức. Màn hình này cho phép nhập các thông tin chính của sự kiện như tiêu đề, mô tả, danh mục, thời gian, địa điểm và các hạng vé. Điểm đáng chú ý là giao diện được thiết kế theo luồng nhập liệu rõ ràng, giúp người tổ chức có thể tạo sự kiện từng bước mà không bị rối bởi quá nhiều thông tin trên cùng một màn hình. Ngoài ra, việc bố trí khu vực quản lý hạng vé ngay trong giao diện tạo sự kiện cho thấy hệ thống đã hỗ trợ khá tốt đặc thù nghiệp vụ của bài toán, thay vì chỉ dừng ở mức tạo một bản tin sự

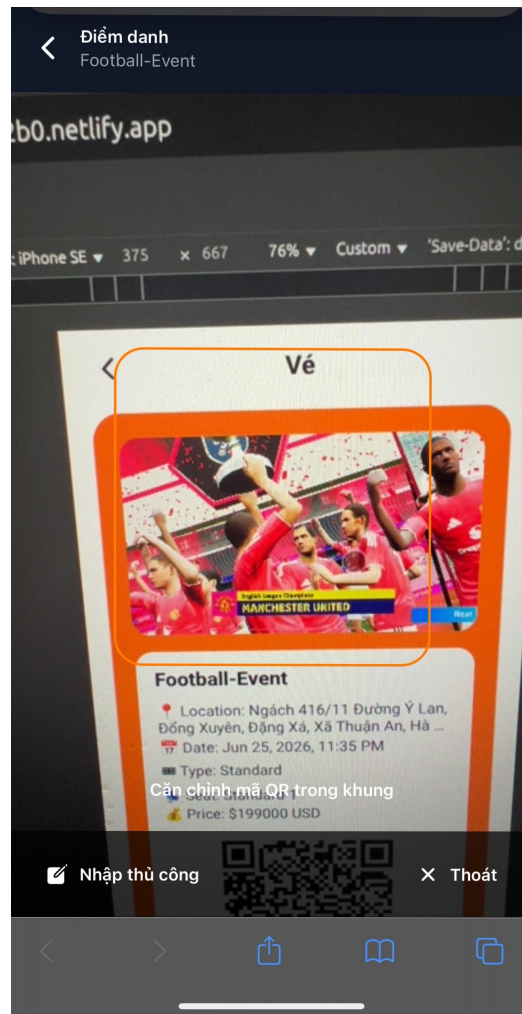
kiện đơn giản.



Hình 4.10: Giao diện chức năng đặt vé

Hình 4.10 minh họa giao diện đặt vé dành cho người dùng. Tại màn hình này, người dùng có thể lựa chọn hạng vé, điều chỉnh số lượng vé và xem tổng tiền trước khi xác nhận thanh toán. Giao diện tập trung làm nổi bật các yếu tố quan trọng nhất đối với quyết định mua vé, bao gồm loại vé, đơn giá, số lượng còn lại và tổng giá trị giao dịch. Cách bố trí này giúp người dùng thao tác nhanh, đồng thời hạn chế

sai sót khi lựa chọn vé cho sự kiện.



Hình 4.11: Giao diện chức năng check-in vé

Hình 4.11 minh họa giao diện check-in vé bằng mã QR. Đây là một trong những chức năng thú vị nhất của hệ thống vì nó kết nối trực tiếp giữa dữ liệu vé điện tử và thao tác tham dự ngoài thực tế. Màn hình được thiết kế để người tổ chức có thể quét mã, nhận phản hồi ngay về tính hợp lệ của vé và cập nhật trạng thái tham dự của người dùng. Chức năng này góp phần hoàn thiện vòng đời của một giao dịch đặt vé, từ lúc người dùng chọn mua vé cho tới khi được xác nhận tham dự sự kiện tại địa điểm tổ chức.

4.4 Kiểm thử

Kiểm thử là bước quan trọng để đánh giá xem hệ thống có hiện thực đúng các yêu cầu nghiệp vụ cốt lõi hay không, đồng thời giúp phát hiện sớm các lỗi xử lý dữ liệu, lỗi xác thực đầu vào và các sai lệch trong luồng nghiệp vụ giữa các thành phần của backend. Đối với hệ thống đặt vé và quản lý sự kiện trong đề án này, không phải mọi chức năng đều có mức độ quan trọng như nhau. Những chức năng cần được ưu tiên kiểm thử trước là các chức năng trực tiếp ảnh hưởng tới trải nghiệm sử dụng và tính đúng đắn của dữ liệu, bao gồm: xác thực người dùng, tạo sự kiện,

đặt vé và xác nhận thanh toán. Đây là các luồng nền tảng, bởi nếu các luồng này hoạt động sai thì hầu hết các chức năng khác như quản lý sự kiện, thông báo, nhắc lịch hay check-in cũng sẽ bị ảnh hưởng theo.

4.4.1 Mục tiêu và phạm vi kiểm thử

Trong phạm vi chương này, đề tài tập trung trình bày kiểm thử cho ba nhóm chức năng quan trọng nhất của hệ thống:

- Nhóm chức năng xác thực người dùng, bao gồm đăng ký tài khoản, đăng nhập và khởi tạo yêu cầu quên mật khẩu.
- Nhóm chức năng tạo sự kiện, bao gồm kiểm tra dữ liệu đầu vào và khởi tạo các hạng vé của sự kiện.
- Nhóm chức năng đặt vé và xác nhận thanh toán, bao gồm kiểm tra giá vé, tạo vé tạm thời, tạo giao dịch thanh toán, cập nhật trạng thái thanh toán và cập nhật số lượng vé đã bán.

Việc lựa chọn ba nhóm chức năng trên là hợp lý vì chúng phản ánh trọn vẹn chuỗi nghiệp vụ chính của hệ thống: người dùng đăng nhập vào hệ thống, người tổ chức tạo sự kiện, người tham dự chọn hạng vé và đặt vé, sau đó giao dịch được xác nhận để phát hành vé hợp lệ. Nếu ba nhóm chức năng này hoạt động ổn định thì có thể xem backend đã xử lý đúng phần nghiệp vụ cốt lõi nhất của đề tài.

Kiểm thử trong phần này tập trung vào backend, cụ thể là các controller xử lý nghiệp vụ. Đây là nơi tổng hợp nhiều điều kiện ràng buộc nhất như xác thực dữ liệu, truy cập model, gọi các hàm hỗ trợ và trả kết quả cho frontend. Ngoài ra, để hỗ trợ quá trình thử nghiệm thủ công và đối chiếu dữ liệu thực tế, đề tài còn xây dựng script sinh dữ liệu mẫu cho MongoDB, bao gồm người dùng, sự kiện, vé, thanh toán và các dữ liệu liên quan. Tuy nhiên, các ca kiểm thử tự động được trình bày dưới đây chủ yếu là kiểm thử đơn vị, chạy độc lập và không phụ thuộc trực tiếp vào cơ sở dữ liệu thật.

4.4.2 Kỹ thuật kiểm thử và môi trường kiểm thử

Đề tài sử dụng kết hợp nhiều kỹ thuật kiểm thử nhằm vừa kiểm tra đúng đầu ra của chức năng, vừa cô lập được từng đơn vị xử lý để xác định nguyên nhân khi có lỗi. Trước hết, kỹ thuật chính được áp dụng là **kiểm thử đơn vị** (*unit testing*) ở mức controller. Mỗi ca kiểm thử tập trung vào một hành vi cụ thể của hàm xử lý, chẳng hạn khi thiếu dữ liệu đầu vào thì phải trả lỗi gì, khi dữ liệu hợp lệ thì có tạo bản ghi đúng định dạng hay không, hoặc khi giao dịch thành công thì hệ thống có cập nhật đủ các trạng thái liên quan hay không.

Về góc nhìn thiết kế ca kiểm thử, đề tài sử dụng đồng thời các kỹ thuật sau:

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

- **Phân lớp tương đương** (*equivalence partitioning*): chia dữ liệu đầu vào thành các nhóm hợp lệ và không hợp lệ, ví dụ email tồn tại và email không tồn tại, giá vé đúng và giá vé sai, hoặc có và không có hạng vé.
- **Kiểm thử giá trị biên và kiểm tra điều kiện ràng buộc**: tập trung vào các điều kiện dễ gây lỗi như số lượng vé nhỏ hơn 1, thiếu danh sách `ticket-Tiers`, giá thanh toán không khớp với tổng giá dự kiến hoặc trạng thái giao dịch chuyển từ `pending` sang `success`.
- **Kiểm thử hộp đen ở mức hành vi đầu ra**: xem controller như một đơn vị nhận request và trả response, từ đó kiểm tra mã trạng thái HTTP và dữ liệu JSON trả về.
- **Kiểm thử hộp trắng ở mức luồng xử lý**: dùng mock để quan sát xem controller có gọi đúng model, đúng utility và đúng số lần cần thiết hay không, ví dụ có gọi tạo `Payment`, có cập nhật `Ticket`, có phát sinh mã QR hay không.

Về công cụ, backend sử dụng Jest kết hợp `ts-jest` để chạy kiểm thử đối với mã nguồn TypeScript. Các đối tượng `req` và `res` của Express được mô phỏng bằng helper riêng để tách biệt controller khỏi môi trường chạy thật của server. Các model Mongoose như `User`, `Event`, `Ticket`, `Payment`, `Notification` và các utility như gửi email, tạo token, gửi push notification hay sinh mã QR được mock lại. Cách tiếp cận này giúp kiểm thử tập trung vào logic nghiệp vụ của controller thay vì bị phụ thuộc vào mạng, cơ sở dữ liệu hoặc dịch vụ ngoài.

Song song với kiểm thử đơn vị, đề tài chuẩn bị dữ liệu thử nghiệm bằng script seed MongoDB để hỗ trợ kiểm thử thủ công ở mức hệ thống. Dữ liệu mẫu bao gồm các tài khoản người dùng, quản trị viên, một số sự kiện với nhiều hạng vé khác nhau, vé đã đặt, đánh giá và bookmark. Dữ liệu seed giúp sinh viên có thể đăng nhập vào ứng dụng, thử tạo sự kiện, đặt vé và quan sát phản hồi trên giao diện trong môi trường gần với vận hành thực tế hơn.

4.4.3 Thiết kế các trường hợp kiểm thử chính

Nhóm chức năng xác thực người dùng

Nhóm chức năng này được xem là điểm vào của toàn bộ hệ thống. Nếu đăng ký, đăng nhập hoặc khởi tạo quên mật khẩu xử lý sai thì người dùng sẽ không thể tiếp cận các nghiệp vụ phía sau. Bảng 4.5 trình bày các ca kiểm thử chính đã được thực hiện cho nhóm chức năng này.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Mã ca	Mục tiêu	Dữ liệu/điều kiện đầu vào	Kết quả mong đợi	Kết quả
AUTH-01	Đăng ký tài khoản hợp lệ	Tên, email và mật khẩu hợp lệ; email chưa tồn tại	Tạo người dùng mới, trả về mã trạng thái 201 cùng token xác thực	Đạt
AUTH-02	Từ chối đăng ký trùng email	Email đã tồn tại trong hệ thống	Trả về mã lỗi 400 và thông báo email đã được sử dụng	Đạt
AUTH-03	Đăng nhập hợp lệ	Email tồn tại, mật khẩu đúng	Trả về thông tin người dùng và token đăng nhập	Đạt
AUTH-04	Từ chối đăng nhập sai tài khoản	Email không tồn tại hoặc không tìm thấy người dùng	Trả về lỗi thông tin đăng nhập không hợp lệ	Đạt
AUTH-05	Khởi tạo quên mật khẩu	Email hợp lệ đã tồn tại trong hệ thống	Lưu mã đặt lại mật khẩu, gửi email và trả về thông báo thành công	Đạt

Bảng 4.5: Các trường hợp kiểm thử chính cho chức năng xác thực người dùng

Các ca kiểm thử của nhóm này chủ yếu áp dụng phân lớp tương đương và kiểm thử hộp đen. Với AUTH-01 và AUTH-03, dữ liệu đầu vào nằm trong lớp hợp lệ nên hệ thống phải trả kết quả thành công. Ngược lại, AUTH-02 và AUTH-04 đại diện cho lớp dữ liệu không hợp lệ hoặc không thỏa điều kiện nghiệp vụ, vì vậy hệ thống phải trả lỗi đúng mã trạng thái. Riêng AUTH-05 kiểm tra một luồng có thêm tác động phụ là gửi email, nên trong kiểm thử đơn vị phần gửi email được mock để xác minh controller có gọi đúng dịch vụ mà không phụ thuộc vào máy chủ thư thật.

Nhóm chức năng tạo sự kiện

Tạo sự kiện là chức năng trung tâm của phía người tổ chức. Bên cạnh các thuộc tính cơ bản như tiêu đề, danh mục, thời gian và địa điểm, hệ thống còn yêu cầu phải có ít nhất một hạng vé hợp lệ. Điều này khiến chức năng tạo sự kiện trở thành nơi thích hợp để kiểm tra các ràng buộc đầu vào và cách backend chuẩn hóa dữ liệu trước khi lưu.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Mã ca	Mục tiêu	Dữ liệu/điều kiện đầu vào	Kết quả mong đợi	Kết quả
EVT-01	Từ chối tạo sự kiện thiếu hạng vé	Có tiêu đề, danh mục, ngày và địa điểm nhưng không có <code>ticket-Tiers</code>	Trả về mã lỗi 400 và thông báo cần ít nhất một hạng vé	Đạt
EVT-02	Tạo sự kiện hợp lệ	Đầy đủ thông tin sự kiện và danh sách hạng vé hợp lệ	Tạo sự kiện thành công; mỗi hạng vé được chuẩn hóa thêm trường <code>sold = 0</code>	Đạt

Bảng 4.6: Các trường hợp kiểm thử chính cho chức năng tạo sự kiện

Ở nhóm kiểm thử này, kỹ thuật được sử dụng nổi bật là kiểm tra điều kiện ràng buộc và kiểm thử hộp trắng ở mức luồng xử lý. Cụ thể, ca EVT-01 đại diện cho lớp dữ liệu không hợp lệ, xác minh controller phát hiện đúng trường thông tin bắt buộc còn thiếu. Ca EVT-02 kiểm tra lớp dữ liệu hợp lệ và đồng thời quan sát lời gọi tới model `Event.create` để chắc chắn hệ thống không chỉ lưu dữ liệu đầu vào thô mà còn bổ sung trạng thái mặc định như `status = upcoming`, `approvalStatus = PENDING` và số vé đã bán ban đầu bằng 0 cho từng hạng vé.

Nhóm chức năng đặt vé và xác nhận thanh toán

Đây là nhóm chức năng có nhiều bước xử lý liên tiếp nhất và ảnh hưởng trực tiếp đến tính đúng đắn của dữ liệu vé cũng như giao dịch. Khi người dùng đặt vé, hệ thống phải kiểm tra sự kiện có tồn tại không, hạng vé có hợp lệ không, số tiền có khớp với giá trị dự kiến không, sau đó mới được tạo vé tạm và tạo giao dịch thanh toán. Khi thanh toán thành công, hệ thống tiếp tục cập nhật trạng thái giao dịch, đánh dấu vé là đã thanh toán, tăng số lượng vé đã bán, cập nhật tổng số thành viên tham gia, phát sinh mã QR và tạo thông báo.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Mã ca	Mục tiêu	Dữ liệu/điều kiện đầu vào	Kết quả mong đợi	Kết quả
TKT-01	Từ chối đặt vé với giá sai	Sự kiện và hạng vé tồn tại, nhưng tổng tiền gửi lên không khớp với đơn giá nhân số lượng	Trả về lỗi 400, không tạo vé và không tạo giao dịch	Đạt
TKT-02	Đặt vé hợp lệ	Hạng vé hợp lệ, số lượng hợp lệ, tổng tiền đúng	Tạo danh sách vé trạng thái <code>pending</code> và tạo một bản ghi thanh toán trạng thái <code>pending</code>	Đạt
TKT-03	Xác nhận thanh toán thành công	Giao dịch tồn tại, thuộc đúng người dùng, tham số <code>success = true</code>	Cập nhật trạng thái thanh toán, cập nhật vé, tăng số lượng vé đã bán, tạo mã QR và thông báo	Đạt

Bảng 4.7: Các trường hợp kiểm thử chính cho chức năng đặt vé và xác nhận thanh toán

Nhóm này là nơi thể hiện rõ nhất việc kết hợp giữa kiểm thử hộp đen và hộp trắng. TKT-01 và TKT-02 chủ yếu kiểm tra hành vi đầu ra của controller dựa trên dữ liệu đầu vào thuộc lớp hợp lệ hoặc không hợp lệ. Trong khi đó, TKT-03 không chỉ cần kiểm tra phản hồi cuối cùng mà còn cần quan sát chuỗi thao tác nội bộ: `Payment` được cập nhật sang trạng thái thành công, `Ticket.updateMany` được gọi để đổi trạng thái vé sang `paid`, `Event.findByIdAndUpdate` được gọi để tăng số lượng vé đã bán và tiện ích sinh mã QR được gọi cho từng vé đã thanh toán. Đây là ví dụ điển hình cho kiểm thử đơn vị theo hướng cô lập phụ thuộc nhưng vẫn bám sát luồng nghiệp vụ thực tế.

4.4.4 Tổng kết kết quả kiểm thử

Sau khi xây dựng và chạy bộ kiểm thử tự động bằng `Jest`, hệ thống backend hiện có tổng cộng **3 test suite** với **10 test case** đã được thực thi thành công. Trong đó:

- 5 test case thuộc nhóm xác thực người dùng.
- 2 test case thuộc nhóm tạo sự kiện.
- 3 test case thuộc nhóm đặt vé và xác nhận thanh toán.

Kết quả chạy kiểm thử cho thấy **10/10 test case đều đạt**, tương ứng với **100% số ca kiểm thử đã thiết kế trong phạm vi hiện tại đều pass**. Điều này cho thấy các controller cốt lõi của hệ thống đang xử lý đúng các tình huống nghiệp vụ quan trọng đã được lựa chọn, đặc biệt là các kiểm tra đầu vào, luồng tạo dữ liệu và các bước cập nhật trạng thái sau giao dịch.

Việc toàn bộ test case đều đạt không có nghĩa là hệ thống đã hoàn toàn không còn lỗi. Thứ nhất, bộ kiểm thử hiện tại chủ yếu là kiểm thử đơn vị ở phía backend

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

nên chưa bao quát đầy đủ kiểm thử tích hợp giữa frontend, backend và cơ sở dữ liệu thật. Thứ hai, các luồng như chat thời gian thực, thông báo đẩy thực tế, kiểm thử hiệu năng, kiểm thử bảo mật hay kiểm thử giao diện người dùng chưa được trình bày chi tiết trong phần này. Thứ ba, một số trường hợp biên sâu hơn như tranh chấp đồng thời khi nhiều người đặt cùng một hạng vé, lỗi mạng khi gửi email hoặc lỗi đồng bộ giữa các dịch vụ ngoài cũng cần được bổ sung trong các vòng kiểm thử sau.

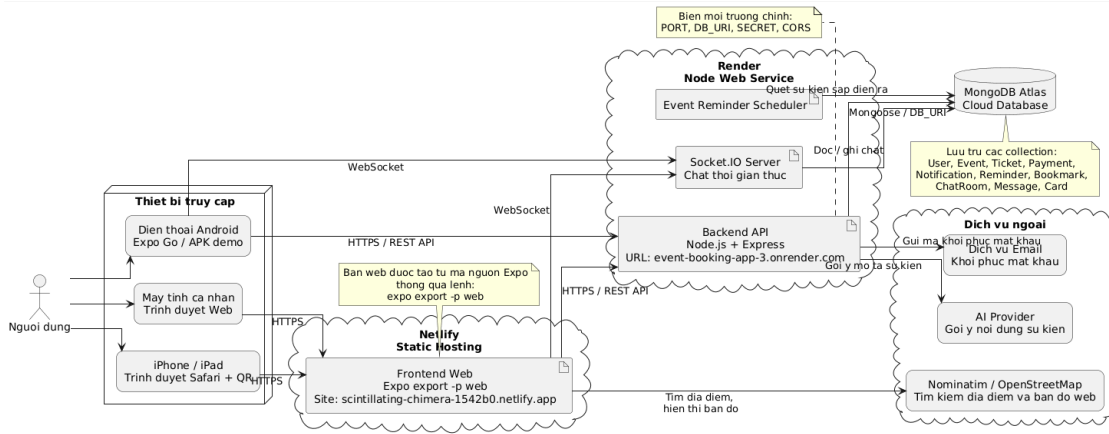
Trong quá trình thực hiện đồ án, nếu có trường hợp kiểm thử không đạt thì nguyên nhân thường sẽ đến từ một trong ba nhóm chính: điều kiện kiểm tra đầu vào chưa đầy đủ, luồng cập nhật dữ liệu liên quan chưa đồng bộ hoặc mock trong kiểm thử chưa phản ánh đúng hành vi thực tế của hệ thống. Tuy nhiên, ở phiên bản được trình bày trong báo cáo này, các ca kiểm thử đã được điều chỉnh lại cho phù hợp với logic hiện thực của mã nguồn và đều cho kết quả đạt yêu cầu. Các trường hợp kiểm thử phụ khác, nếu cần trình bày chi tiết hơn, có thể được đưa vào phần phụ lục để tránh làm phân tán trọng tâm của chương.

4.5 Triển khai

Trong phạm vi đồ án, hệ thống được triển khai theo mô hình client-server trên môi trường Internet, trong đó frontend đóng vai trò client và backend cung cấp các dịch vụ xử lý nghiệp vụ cũng như truy cập dữ liệu. Mô hình này phù hợp với định hướng mobile-first của hệ thống, bởi các thao tác như đăng nhập, tìm kiếm sự kiện, đặt vé, nhận thông báo, chat và check-in đều được thực hiện ở phía người dùng, sau đó gửi yêu cầu tới máy chủ backend để xử lý. Ở giai đoạn triển khai thử nghiệm, hệ thống được tổ chức thành bốn thành phần chính: frontend Expo/React Native, backend Node.js/Express, cơ sở dữ liệu MongoDB và hạ tầng triển khai trực tuyến cho phép truy cập qua Internet.

Mô hình triển khai thử nghiệm

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.12: Mô hình triển khai thử nghiệm của hệ thống event-booking-app

Trong quá trình triển khai thực tế, backend được đưa lên dịch vụ Render dưới dạng một *Node Web Service*. Mã nguồn backend được đặt trong thư mục `backend`, được biên dịch từ TypeScript sang JavaScript thông qua lệnh `npm run build`, sau đó khởi động bằng lệnh `npm start`. Tập cấu hình `backend/render.yaml` được sử dụng để mô tả cấu hình triển khai, bao gồm thư mục gốc, lệnh `build`, lệnh `start` và đường dẫn health check là `/api/health`. Việc bổ sung endpoint health check cho phép kiểm tra nhanh trạng thái hoạt động của dịch vụ sau mỗi lần triển khai và hỗ trợ nền tảng Render giám sát tình trạng khởi động của server.

Backend hiện được truy cập qua địa chỉ `https://event-booking-app-3.onrender.com`. Tại phía máy chủ này, ứng dụng Node.js/Express khởi tạo từ tệp `backend/src/server.ts`, thực hiện các bước kết nối cơ sở dữ liệu, khởi động bộ lập lịch nhắc sự kiện và khởi tạo Socket.IO cho chức năng chat thời gian thực trước khi mở cổng phục vụ yêu cầu. Cổng mặc định của backend là 5000, tuy nhiên trong môi trường triển khai giá trị cổng được cấp động thông qua biến môi trường `PORT`. Các biến môi trường khác như `DB_URI`, `SECRET`, `JWT_EXPIRES_IN`, `CORS`, `EMAIL_USER`, `EMAIL_PASSWORD` hay các biến liên quan đến AI được cấu hình trong môi trường của Render thay vì ghi cứng trong mã nguồn. Cách làm này giúp hệ thống an toàn hơn và thuận tiện khi thay đổi môi trường chạy.

Cơ sở dữ liệu và kết nối dữ liệu

Ở phía dữ liệu, hệ thống sử dụng MongoDB Atlas và kết nối tới backend thông qua biến môi trường `DB_URI`. Việc đặt cơ sở dữ liệu trên một dịch vụ cloud tách biệt với backend giúp tách riêng lớp lưu trữ ra khỏi lớp xử lý nghiệp vụ, nhờ đó việc thay đổi máy chủ ứng dụng không làm ảnh hưởng trực tiếp đến dữ liệu. Với

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

mô hình hiện tại, dữ liệu người dùng, sự kiện, vé, thanh toán, thông báo, lời nhắc, bookmark, chatroom và message được lưu dưới dạng các collection riêng biệt. Đây là một lựa chọn phù hợp với MongoDB vì hệ thống có nhiều thực thể nghiệp vụ liên kết với nhau nhưng vẫn đòi hỏi sự linh hoạt khi mở rộng cấu trúc dữ liệu trong các giai đoạn phát triển tiếp theo.

Triển khai frontend phục vụ thử nghiệm

Ở phía client, hệ thống ban đầu được xây dựng như một ứng dụng di động bằng React Native và Expo. Tuy nhiên, trong quá trình thử nghiệm thực tế, việc phân phối ứng dụng lên iOS gặp ràng buộc về tài khoản Apple Developer. Vì vậy, để thuận tiện cho việc trình diễn và kiểm thử trên nhiều thiết bị, đồ án sử dụng thêm một phương án triển khai web từ chính mã nguồn Expo. Cụ thể, frontend được export ra dạng website tĩnh bằng lệnh `expo export -p web`, sinh ra thư mục `frontend/dist`. Thư mục này sau đó được triển khai lên Netlify theo hình thức static hosting và truy cập qua tên miền thử nghiệm `https://scintillating-chimera-1542b0.netlify.app`. Nhờ đó, người dùng có thể mở hệ thống trên iPhone hoặc máy tính bằng trình duyệt thông qua đường link hoặc mã QR mà không cần cài trực tiếp ứng dụng native.

Trong quá trình chuyển frontend sang môi trường web, một số màn hình phụ thuộc thành phần native đã được điều chỉnh để tương thích. Điển hình là màn hình chọn vị trí `SelectLocation`: trên mobile, màn hình này dùng `react-native-maps`, còn trên web hệ thống sử dụng tệp riêng `SelectLocation.web.tsx` với thư viện `Leaflet` và bản đồ `OpenStreetMap` để vẫn bảo đảm người dùng có thể tìm kiếm địa chỉ, xem bản đồ và chọn vị trí tương tự như trên điện thoại. Điều này cho thấy hệ thống không chỉ được triển khai về mặt hạ tầng mà còn được thích nghi ở mức giao diện để phù hợp với môi trường vận hành thực tế. Phân tích chi tiết về cách tách lớp phụ thuộc nền tảng và giữ lõi nghiệp vụ dùng chung sẽ được trình bày ở Mục 5.1 của Chương 5.

Liên kết giữa frontend và backend

Sau khi backend đã được triển khai trên Render, frontend được cấu hình sử dụng địa chỉ API công khai `https://event-booking-app-3.onrender.com` trong lớp service giao tiếp. Ở phía backend, biến môi trường cơ chế chia sẻ tài nguyên khác nguồn (Cross-Origin Resource Sharing - CORS) được đặt theo tên miền frontend trên Netlify để chỉ cho phép client hợp lệ gửi yêu cầu đến server. Nhờ đó, quá trình đăng nhập, lấy danh sách sự kiện, đọc dữ liệu người dùng và các luồng nghiệp vụ chính có thể được kiểm thử trong môi trường thực tế qua Internet,

thay vì chỉ chạy trong mạng nội bộ. Việc kiểm tra backend được thực hiện thông qua endpoint `/api/health`, còn frontend được kiểm tra bằng cách export lại website sau mỗi lần thay đổi cấu hình và triển khai lại thư mục `dist` lên Netlify.

Thiết bị và cấu hình thử nghiệm

Về phía thiết bị người dùng, hệ thống được thử nghiệm trên điện thoại thông minh Android, điện thoại iPhone và trình duyệt web trên máy tính cá nhân có kết nối Internet ổn định. Với Android, ứng dụng có thể chạy qua Expo Go hoặc tiếp tục đóng gói thành bản APK để cài đặt thủ công. Với iOS, phương án trình diễn chính trong đồ án là truy cập bản web triển khai trên Netlify bằng Safari thông qua đường link hoặc mã QR. Về mặt máy chủ, hệ thống không sử dụng một máy vật lý cài đặt thủ công mà tận dụng mô hình nền tảng như một dịch vụ (Platform as a Service - PaaS): backend chạy trên Render, frontend tính chạy trên Netlify và dữ liệu được lưu trên MongoDB Atlas. Đây là cách triển khai phù hợp với quy mô đồ án vì đơn giản, ít tốn công quản trị hạ tầng nhưng vẫn đủ để tái hiện quá trình vận hành của một hệ thống thực tế có nhiều thành phần phân tán.

Đánh giá mô hình triển khai

Ở mức thử nghiệm, mô hình triển khai hiện tại đã đủ để kiểm tra các luồng nghiệp vụ chính của hệ thống trong môi trường thực tế có kết nối mạng, bao gồm đăng nhập, đọc dữ liệu từ server, tương tác với bản đồ web, quản lý sự kiện và trao đổi dữ liệu qua API. Mô hình này cũng cho phép nhóm phát triển dễ dàng cập nhật backend và frontend một cách độc lập. Tuy nhiên, nếu triển khai ở quy mô lớn hơn trong tương lai, hệ thống vẫn cần được bổ sung thêm các cơ chế như giám sát tài nguyên, lưu log tập trung, tối ưu hiệu năng truy vấn, cấu hình database theo chế độ tin cậy cao, triển khai mạng phân phối nội dung (Content Delivery Network - CDN) cho tài nguyên tĩnh và tăng cường bảo mật đối với khóa bí mật, email, token và dữ liệu người dùng.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này tập trung vào những giải pháp mà tác giả đánh giá là có giá trị nhất trong suốt quá trình thực hiện đề án. Khác với Chương 4 vốn đặt trọng tâm vào mô tả thiết kế, hiện thực và kết quả của toàn hệ thống, chương này đi sâu vào các bài toán khó, các quyết định kỹ thuật quan trọng và các cách tiếp cận đã được lựa chọn để giải quyết các bài toán đó. Mỗi mục dưới đây không nhằm liệt kê lại chức năng đã có, mà nhằm làm rõ vì sao bài toán đó khó, giải pháp nào đã được áp dụng, và giải pháp đó đã mang lại giá trị gì cho sản phẩm.

Trong phạm vi đề án này, bốn đóng góp nổi bật nhất gồm: giải pháp thích ứng đa nền tảng cho các chức năng phụ thuộc thiết bị; giải pháp bảo toàn tính nhất quán cho luồng đặt vé nhiều hạng kết hợp thanh toán và check-in bằng mã QR; giải pháp xây dựng cơ chế nhắc lịch và thông báo hợp nhất, hạn chế gửi trùng; và giải pháp tích hợp mô-đun AI hỗ trợ tạo nội dung sự kiện với cơ chế dự phòng nhiều tầng. Đây đều là những vấn đề có liên quan trực tiếp tới khả năng sử dụng thật của hệ thống, đồng thời thể hiện rõ nhất tư duy phân tích và tổng quát hóa trong quá trình làm đề án.

5.1 Giải pháp thích ứng đa nền tảng cho các chức năng phụ thuộc thiết bị

5.1.1 Bài toán và bối cảnh

Một trong những mục tiêu thực tế của dự án là không chỉ dừng ở ứng dụng mobile chạy trên Android/iOS, mà còn có thể khai thác lại mã nguồn để phục vụ bản chạy trên web. Tuy nhiên, khi chuyển một ứng dụng React Native sang môi trường web, những chức năng phụ thuộc chặt vào thiết bị thường là nơi phát sinh nhiều vấn đề nhất. Trong hệ thống event-booking-app, hai nhóm chức năng nhạy cảm nhất là chọn vị trí trên bản đồ và quét vé bằng camera. Trên thiết bị di động, ứng dụng có thể dùng trực tiếp `react-native-maps`, `expo-location` hoặc camera của Expo. Trên web, các thư viện này không còn giữ nguyên hành vi như trên mobile, hoặc chịu thêm ràng buộc của trình duyệt như giao thức truyền tải siêu văn bản bảo mật (HyperText Transfer Protocol Secure - HTTPS), quyền truy cập camera và cơ chế render khác biệt.

Nếu giải quyết theo cách đơn giản nhất, người phát triển có thể bỏ hẳn các chức năng này khỏi web, hoặc chỉ hỗ trợ ở mức rất hạn chế. Tuy nhiên, cách làm đó khiến giá trị đa nền tảng của hệ thống bị giảm mạnh, đặc biệt trong bối cảnh web lại là môi trường thuận tiện để trình diễn sản phẩm hoặc triển khai thử nghiệm khi chưa đóng gói ứng dụng di động hoàn chỉnh. Ngược lại, nếu viết lại toàn bộ một phiên bản web tách biệt với mobile thì chi phí bảo trì cao, luồng nghiệp vụ dễ sai

khác, và bản thân đồ án sẽ mất đi lợi thế “một lõi nghiệp vụ, nhiều điểm truy cập”. Do đó, bài toán cốt lõi không chỉ là làm cho web chạy được, mà là làm sao thích ứng với khác biệt nền tảng mà vẫn giữ được sự thống nhất ở luồng sử dụng và dữ liệu trao đổi với backend.

5.1.2 Giải pháp

Giải pháp được lựa chọn là tách phần phụ thuộc nền tảng ra khỏi phần nghiệp vụ dùng chung, tức là giữ nguyên các service, route điều hướng, dữ liệu đầu vào/đầu ra và quy tắc nghiệp vụ, nhưng thay thế lớp tương tác thiết bị bằng các adapter riêng cho từng môi trường. Cách tiếp cận này được thể hiện khá rõ qua hai cặp màn hình `SelectLocation.tsx` và `SelectLocation.web.tsx`, cũng như `CheckIn.tsx` và `CheckIn.web.tsx`. Ở đây, tác giả không tách hai ứng dụng độc lập, mà chỉ tách lớp hiện thực giao diện và truy cập thiết bị, còn các tham số điều hướng, cách gọi service, cấu trúc dữ liệu và trạng thái luồng nghiệp vụ vẫn được giữ tương đồng.

Đối với bài toán chọn vị trí, bản mobile sử dụng `MapView` kết hợp `expo-location` để lấy tọa độ hiện tại, di chuyển camera và xác nhận vị trí trên bản đồ. Trong khi đó, bản web chuyển sang dùng `Leaflet` và cơ chế `navigator.geolocation`, đồng thời nạp động stylesheet và icon tương ứng cho môi trường trình duyệt. Điểm quan trọng không nằm ở việc thay thư viện bản đồ, mà ở chỗ toàn bộ quy trình nghiệp vụ vẫn được giữ nguyên: người dùng có thể tìm kiếm địa chỉ qua `Nominatim`, chọn kết quả gợi ý, tinh chỉnh vị trí trên bản đồ, reverse geocode để lấy tên địa điểm, rồi trả kết quả về đúng màn hình gọi trước đó. Như vậy, sự khác biệt chỉ nằm ở “cách người dùng thao tác với bản đồ”, còn “ý nghĩa nghiệp vụ của bước chọn vị trí” không thay đổi.

Tương tự, với chức năng check-in vé, bản mobile ưu tiên quét camera trực tiếp thông qua `expo-camera`. Tuy nhiên, trên web, camera thường gặp các ràng buộc như không chạy trên HTTP, bị người dùng từ chối cấp quyền, hoặc hoạt động không ổn định giữa các trình duyệt. Thay vì xem đây là lý do để bỏ chức năng, hệ thống bổ sung một nhánh dự phòng cho web là nhập mã QR thủ công. Cách làm này cho thấy tư duy thiết kế tập trung vào nghiệp vụ thay vì phụ thuộc tuyệt đối vào thiết bị: bản chất của check-in không phải là “phải quét bằng camera”, mà là “xác minh dữ liệu vé và cập nhật trạng thái tham dự một cách an toàn”. Camera chỉ là một phương tiện thuận tiện để đưa dữ liệu vào hệ thống.

Điểm quan trọng hơn là dù giao diện nền tảng khác nhau, cả mobile lẫn web đều gọi chung các service nghiệp vụ và cùng đi tới một backend duy nhất. Điều đó giúp tránh tình trạng mỗi nền tảng tự phát triển một logic kiểm tra riêng. Khi thay

đổi quy tắc xác minh check-in hoặc quy tắc lấy vị trí, tác giả chỉ cần cập nhật một tầng nghiệp vụ thay vì phải đồng bộ thủ công ở nhiều nơi. Có thể xem đây là một giải pháp thích ứng theo lớp: tầng giao diện chấp nhận khác biệt nền tảng, còn tầng service và tầng backend vẫn giữ vai trò nguồn chân lý duy nhất của hệ thống.

5.1.3 Kết quả đạt được

Kết quả rõ nhất của giải pháp này là hệ thống duy trì được khả năng chạy đa nền tảng mà không phải hy sinh các chức năng quan trọng. Người dùng mobile vẫn có trải nghiệm tự nhiên với bản đồ và camera thiết bị, trong khi người dùng web vẫn sử dụng được các chức năng tương đương với một số điều chỉnh hợp lý theo đặc thù trình duyệt. Điều này làm tăng đáng kể giá trị thực tế của sản phẩm trong giai đoạn thử nghiệm và triển khai.

Về mặt kỹ thuật, giải pháp giúp giảm trùng lặp mã nguồn nghiệp vụ. Những gì thực sự khác biệt mới bị tách riêng sang tệp `.web.tsx`; các phần như điều hướng, service, xác thực, gọi API và cấu trúc dữ liệu vẫn được tái sử dụng. Nhờ đó, chi phí bảo trì thấp hơn so với việc viết hai phiên bản độc lập. Quan trọng hơn, giải pháp này có thể tổng quát hóa cho nhiều chức năng khác của ứng dụng đa nền tảng: khi gặp khác biệt phần cứng hoặc môi trường, không nhất thiết phải bỏ chức năng hoặc nhân đôi toàn bộ hệ thống, mà có thể giữ một lõi nghiệp vụ chung và chỉ thay adapter tương tác ở lớp biên.

5.2 Giải pháp bảo toàn tính nhất quán cho luồng đặt vé nhiều hạng kết hợp thanh toán và check-in

5.2.1 Bài toán và bối cảnh

Nghiệp vụ trung tâm của hệ thống là cho phép người dùng đặt vé tham gia sự kiện. Nếu chỉ dừng ở mức mỗi sự kiện có một mức giá cố định và không có xác nhận tham dự, bài toán tương đối đơn giản. Tuy nhiên, hệ thống của đề án đi theo hướng thực tế hơn: mỗi sự kiện có thể có nhiều hạng vé với giá và quota khác nhau; người dùng có thể đặt nhiều vé trong một giao dịch; sau thanh toán, hệ thống phải phát hành vé điện tử; khi đến sự kiện, ban tổ chức phải kiểm tra được vé nào hợp lệ, vé nào đã dùng, vé nào thuộc sự kiện khác hoặc chưa thanh toán. Chính sự nối tiếp của nhiều bước này khiến bài toán không còn là “lưu một bản ghi mua vé”, mà là bài toán bảo toàn tính nhất quán của cả một chuỗi nghiệp vụ.

Khó khăn lớn nhất nằm ở việc dữ liệu từ phía client không thể được tin cậy hoàn toàn. Nếu frontend gửi lên mức giá đã tính sẵn, backend chấp nhận trực tiếp thì rất dễ phát sinh nguy cơ sửa giá ở phía client. Nếu vé được sinh ngay khi người dùng bấm đặt mà chưa gắn với trạng thái thanh toán, hệ thống có thể phát hành vé chưa trả tiền. Nếu check-in chỉ kiểm tra mã tồn tại mà không kiểm tra sự kiện, trạng thái

thanh toán hoặc trạng thái đã dùng, người tổ chức có thể quét nhầm hoặc chấp nhận vé không hợp lệ. Như vậy, thách thức ở đây không nằm ở một hàm đơn lẻ, mà nằm ở việc thiết kế luồng dữ liệu sao cho mỗi bước đều kiểm tra lại các giả định quan trọng.

5.2.2 Giải pháp

Giải pháp được áp dụng là dồn toàn bộ quy tắc kiểm tra quan trọng về backend và tách nghiệp vụ thành các pha rõ ràng: kiểm tra dữ liệu đặt vé, tạo vé ở trạng thái chờ, tạo giao dịch thanh toán, xác nhận thanh toán, sau đó mới phát hành dữ liệu QR và cho phép check-in. Trong controller đặt vé, backend không sử dụng giá client gửi lên như một nguồn chân lý tuyệt đối. Thay vào đó, hệ thống tra cứu hạng vé theo `tierName` ngay trên tài liệu sự kiện, tính lại `expectedTotal` từ giá của hạng vé và số lượng vé, rồi so sánh với số tiền được gửi từ client. Nếu hai giá trị không khớp, yêu cầu bị từ chối. Đây là một bước nhỏ nhưng rất quan trọng, vì nó chặn được lớp lỗi phổ biến nhất ở các ứng dụng thương mại điện tử đơn giản: tin vào giá phía client.

Sau khi xác thực giá và quota, hệ thống tạo nhiều bản ghi `Ticket` ứng với số lượng vé mua, nhưng tất cả đều bắt đầu ở trạng thái `paymentStatus = pending`. Đồng thời, hệ thống chỉ tạo một bản ghi `Payment` duy nhất cho toàn giao dịch. Cách tổ chức này mang lại hai lợi ích. Thứ nhất, người dùng vẫn có thể mua nhiều vé trong một lần thao tác. Thứ hai, hệ thống phân biệt được “đơn vị quyền tham dự” là từng vé, còn “đơn vị giao dịch tài chính” là một thanh toán. Đây là tách biệt có ý nghĩa dài hạn, bởi nếu sau này cần hoàn tiền, thống kê doanh thu hoặc tích hợp cổng thanh toán thật, lớp `Payment` có thể được mở rộng mà không phá vỡ mô hình vé.

Khi thanh toán được xác nhận thành công, backend mới cập nhật hàng loạt trạng thái vé sang `paid`, tăng số lượng vé đã bán của đúng hạng vé tương ứng, tính lại tổng số người tham gia của sự kiện, tạo thông báo thanh toán và sinh dữ liệu QR cho từng vé. Việc sinh QR ở giai đoạn sau thanh toán thay vì ở giai đoạn đặt giữ cho hệ thống chặt chẽ hơn: chỉ vé đã thanh toán mới có tư cách trở thành vé điện tử hợp lệ. Dữ liệu QR được thiết kế gọn theo dạng `ticketId|eventId|token`, nghĩa là mã quét không chỉ trỏ tới một vé, mà còn gắn chặt với một sự kiện cụ thể và có thêm token phụ trợ để giảm tính đoán trước.

Phần check-in tiếp tục tuân theo nguyên tắc “mọi xác nhận quan trọng đều thực hiện ở backend”. Khi người tổ chức quét hoặc nhập dữ liệu mã, backend lần lượt kiểm tra: người thực hiện có đúng là organizer của sự kiện hay không; mã có đúng định dạng không; vé trong mã có thuộc đúng sự kiện đang check-in không; vé có

thực sự tồn tại không; vé đã thanh toán chưa; vé đã check-in trước đó chưa. Chỉ khi vượt qua toàn bộ các bước này, hệ thống mới cập nhật trạng thái `checked`, thời điểm `checkedAt` và người thực hiện `checkedBy`. Nhờ vậy, check-in không chỉ là thao tác đánh dấu, mà là bước xác minh cuối cùng của toàn chuỗi từ đặt vé tới tham dự.

5.2.3 Kết quả đạt được

Giải pháp này giúp hệ thống đạt được một mức nhất quán nghiệp vụ khá tốt trong phạm vi đồ án. Các nguy cơ như sửa giá ở phía client, đặt vượt quota của hạng vé, sinh vé trước khi thanh toán, dùng vé sai sự kiện hoặc check-in lặp lại đều đã được chặn bằng các bước xác thực cụ thể ở phía backend. So với cách cài đặt “frontend tính gì backend lưu nấy”, cách tiếp cận hiện tại an toàn hơn và có tính mở rộng tốt hơn.

Một kết quả khác là luồng nghiệp vụ trở nên rõ ràng và dễ kiểm thử hơn. Vì mỗi pha đều có trách nhiệm riêng, việc viết use case, biểu đồ trình tự và test cho các nhánh thành công hoặc thất bại cũng thuận lợi hơn. Ở góc độ tổng quát, đây là một giải pháp có thể áp dụng cho nhiều hệ thống đặt chỗ hoặc bán vé khác: không để nghiệp vụ quan trọng trộn lẫn trong một bước duy nhất, mà chia thành các pha có trạng thái trung gian rõ ràng, để mỗi bước đều có thể được xác minh và phục hồi khi cần.

5.3 Giải pháp xây dựng cơ chế nhắc lịch và thông báo hợp nhất, hạn chế gửi trùng

5.3.1 Bài toán và bối cảnh

Đối với một ứng dụng sự kiện, trải nghiệm của người dùng không kết thúc ở thời điểm đặt vé. Sau khi đã quan tâm hoặc mua vé, người dùng còn cần được nhắc nhở đúng lúc để không bỏ lỡ sự kiện; người tổ chức cũng cần nhận tín hiệu về các hành vi quan trọng như thanh toán thành công hoặc tương tác từ người dùng. Nếu không có lớp thông báo phù hợp, ứng dụng rất dễ trở thành một nơi “đặt xong rồi quên”, khiến giá trị sử dụng thực tế giảm mạnh.

Vấn đề là thông báo trong hệ thống không phải chỉ có một loại. Ở dự án này, có thông báo thanh toán, lời mời sự kiện, nhắc lịch trước giờ diễn ra, và các cập nhật liên quan tới chat. Nếu mỗi loại được xử lý theo một kiểu khác nhau, hệ thống sẽ nhanh chóng trở nên rời rạc: nơi lưu ở database, nơi chỉ gửi push, nơi không kiểm soát việc gửi trùng, nơi không quan tâm người dùng đã tắt thông báo hay chưa. Đặc biệt với thông báo nhắc lịch, nguy cơ gửi trùng là khá lớn, nhất là khi scheduler chạy định kỳ hoặc khi server khởi động lại.

5.3.2 Giải pháp

Giải pháp được lựa chọn là xây dựng một trục thông báo hợp nhất, trong đó `Notification` đóng vai trò thực thể trung tâm để lưu toàn bộ thông báo nội bộ, còn `push notification` là lớp phát bổ sung ra thiết bị khi có token phù hợp. Với cách làm này, mọi thông báo quan trọng đều có dấu vết trong hệ thống, giúp người dùng có thể xem lại trong giao diện thông báo kể cả khi `push` bị trượt hoặc không mở được ngay thời điểm nhận.

Riêng đối với nhắc lịch sự kiện, tác giả triển khai một scheduler chạy theo chu kỳ 5 phút. Thay vì chỉ lọc sự kiện theo ngày đơn thuần, hệ thống tách riêng `date` và `time` rồi ghép lại thành thời điểm bắt đầu thực tế của sự kiện. Sau đó, scheduler xét hai cửa sổ thời gian quan trọng là trước 1 ngày và trước 1 giờ. Cách tiếp cận này giải quyết một vấn đề thường gặp khi lưu ngày và giờ tách rời: nếu chỉ so sánh theo trường ngày, hệ thống khó xác định chính xác thời điểm nhắc cho những sự kiện diễn ra vào các khung giờ khác nhau trong cùng một ngày.

Để tránh gửi trùng, giải pháp sử dụng hai lớp bảo vệ. Lớp thứ nhất là một `Map` trong bộ nhớ để ghi nhận những thông báo đã được gửi ở cửa sổ hiện tại, tránh việc scheduler lặp lại cùng một hành động trong thời gian ngắn. Tuy nhiên, chỉ dựa vào bộ nhớ là chưa đủ vì khi server khởi động lại thì trạng thái này mất đi. Do đó, lớp thứ hai là truy vấn ngược vào collection `Notification` để kiểm tra xem với cùng người dùng, cùng sự kiện và cùng nội dung nhắc, hệ thống đã tạo thông báo hay chưa. Chỉ những người nhận chưa có thông báo tương ứng mới tiếp tục được gửi `push` và ghi mới vào database. Đây là một điểm mà tác giả đánh giá là quan trọng, vì nó biến cơ chế nhắc lịch từ “gần đúng” thành “đủ tin cậy để dùng thật”.

Một quyết định thiết kế khác là xác định người nhận theo nghĩa nghiệp vụ thay vì chỉ dựa trên danh sách nhắc thủ công. Với mỗi sự kiện sắp diễn ra, hệ thống lấy hợp của hai tập: người tổ chức sự kiện và những người đã có vé ở trạng thái `paid`. Cách xác định này hợp lý hơn cho luồng hiện tại vì người thực sự cần được nhắc là người có trách nhiệm tổ chức và người chắc chắn sẽ tham dự. Trên cùng nền tảng đó, lời mời sự kiện và thông báo thanh toán cũng được ghi vào chung hệ thống thông báo, từ đó giao diện notification ở frontend có thể hỗ trợ đồng nhất các thao tác như xem danh sách, đánh dấu đã đọc, đánh dấu một phần hoặc xóa thông báo.

5.3.3 Kết quả đạt được

Kết quả của giải pháp là hệ thống có được một lớp thông báo tương đối thống nhất giữa backend và frontend, đồng thời tạo nền tảng tốt để mở rộng sau này. Người dùng không chỉ nhận `push notification` khi có điều kiện phù hợp, mà còn có một hộp thư thông báo nội bộ để tra cứu lại các sự kiện đã xảy ra. Điều này đặc

biệt hữu ích với lời mời tham gia sự kiện hoặc nhắc lịch trước giờ diễn ra.

Ở góc độ kỹ thuật, giá trị lớn nhất của giải pháp là hạn chế được hiện tượng gửi trùng vốn rất dễ xuất hiện trong các hệ thống scheduler đơn giản. Việc kết hợp giữa bộ nhớ tạm và kiểm tra database cho thấy một hướng xử lý thực tế: không kỳ vọng một cơ chế đơn lẻ giải quyết toàn bộ bài toán, mà kết hợp nhiều lớp kiểm soát với chi phí hợp lý. Đây là một kinh nghiệm có thể tổng quát hóa cho các bài toán gửi thông báo định kỳ khác, không chỉ trong ứng dụng sự kiện mà còn trong các ứng dụng nhắc việc, lớp học, đặt lịch hay chăm sóc khách hàng.

5.4 Giải pháp tích hợp AI hỗ trợ tạo nội dung sự kiện với cơ chế dự phòng nhiều tầng

5.4.1 Bài toán và bối cảnh

Một khó khăn thực tế của người tổ chức sự kiện, đặc biệt ở những ứng dụng hướng cộng đồng, là họ thường không gặp khó trong việc chọn ngày hay địa điểm, mà gặp khó ở khâu mô tả nội dung sao cho rõ ràng, hấp dẫn và nhất quán. Nếu để trống hoàn toàn bước này, nhiều sự kiện dễ có tiêu đề sơ sài, mô tả nghèo nàn hoặc hạng vé được đặt tên thiếu hợp lý, từ đó làm giảm chất lượng chung của nền tảng. Vì vậy, tác giả mong muốn hệ thống không chỉ là nơi “đăng sự kiện”, mà còn hỗ trợ người tổ chức hình thành nội dung ban đầu.

Tuy nhiên, khi đưa AI vào một đồ án theo hướng ứng dụng, bài toán không dừng ở việc gọi được một mô hình sinh văn bản. Dịch vụ AI bên ngoài có thể lỗi, hết hạn khóa, trả về dữ liệu sai cấu trúc hoặc có độ trễ lớn. Nếu frontend phụ thuộc hoàn toàn vào một lời gọi không ổn định như vậy, trải nghiệm tạo sự kiện sẽ bị ảnh hưởng mạnh. Nói cách khác, thách thức không chỉ là “có AI”, mà là “tích hợp AI theo cách không làm hỏng luồng nghiệp vụ chính”.

5.4.2 Giải pháp

Giải pháp được chọn là đưa AI về phía backend dưới dạng một service độc lập, đồng thời áp dụng cơ chế fallback nhiều tầng. Ở tầng đầu tiên, hệ thống có thể dùng OpenAI làm nhà cung cấp ưu tiên. Nếu cấu hình yêu cầu, hệ thống cũng hỗ trợ gọi Ollama như một mô hình nội bộ hoặc cục bộ. Nếu cả hai cách đều không khả dụng, hệ thống vẫn không trả lỗi ngay lập tức mà rơi về một tầng cuối là sinh gợi ý theo template. Như vậy, luồng tạo sự kiện không bao giờ bị chặn hoàn toàn chỉ vì một nhà cung cấp AI không phản hồi.

Một điểm quan trọng khác của giải pháp là không nhận nội dung AI ở dạng văn bản tự do, mà ép đầu ra về một cấu trúc JSON có schema rõ ràng gồm ba thành phần: tiêu đề, mô tả và danh sách hạng vé. Sau khi nhận dữ liệu, backend còn tiếp

tục chuẩn hóa và kiểm tra lại để bảo đảm tên hạng vé không rỗng, giá là số hợp lệ, quota dương và số lượng hạng vé nằm trong phạm vi cho phép. Cách làm này giúp giảm đáng kể rủi ro một mô hình sinh ra câu trả lời đẹp nhưng không dùng được trực tiếp trong hệ thống. Về bản chất, AI chỉ đóng vai trò đề xuất; quyền kiểm soát cấu trúc dữ liệu vẫn thuộc về lớp nghiệp vụ.

Ở phía frontend, mô-đun này được tích hợp ngay trong màn hình tạo sự kiện. Người tổ chức có thể cung cấp danh mục sự kiện, vị trí, ngày dự kiến, nội dung mô tả hiện tại và một lời nhắc ngắn về định hướng chương trình. Sau khi nhận gợi ý, frontend tự động điền dữ liệu vào biểu mẫu và cho phép người dùng tiếp tục chỉnh sửa trước khi gửi tạo sự kiện thật. Điều này cho thấy AI không thay thế người tổ chức, mà hoạt động như một công cụ hỗ trợ tăng tốc bước khởi tạo nội dung.

Điểm mà tác giả đánh giá là có giá trị trong giải pháp này là tư duy “AI là một dịch vụ không hoàn toàn tin cậy”. Vì thế, toàn bộ cách tích hợp đều đi theo hướng chống phụ thuộc tuyệt đối: có schema để kiểm soát cấu trúc, có sanitization để kiểm soát dữ liệu, có fallback để giữ luồng nghiệp vụ không bị gián đoạn, và có khả năng chuyển đổi nhà cung cấp qua biến môi trường. Đây là hướng tiếp cận phù hợp hơn với ứng dụng thực tế so với việc gọi trực tiếp một API AI từ frontend và hy vọng mọi phản hồi đều hợp lệ.

5.4.3 Kết quả đạt được

Kết quả trước hết là người tổ chức có thêm một công cụ hỗ trợ rất thực dụng khi tạo sự kiện. Thay vì bắt đầu từ biểu mẫu trống, họ có thể nhận được một bộ khung ban đầu gồm tiêu đề, mô tả và 1–3 hạng vé hợp lý để tiếp tục tinh chỉnh. Điều này giúp giảm đáng kể ma sát ở bước khởi tạo, nhất là với các sự kiện cộng đồng nhỏ hoặc các trường hợp người dùng chưa quen cách viết nội dung quảng bá.

Ở góc độ kỹ thuật, giải pháp chứng minh rằng có thể tích hợp AI vào một hệ thống nghiệp vụ mà không làm mất tính ổn định của hệ thống. Ngay cả khi không có khóa OpenAI, khi Ollama không hoạt động hoặc khi mạng có vấn đề, chức năng vẫn đưa ra được một gợi ý tối thiểu nhờ tầng template fallback. Đây là một đóng góp mà tác giả đánh giá là đáng giá vì nó chuyển AI từ vai trò “tính năng trình diễn” sang vai trò “thành phần có kiểm soát trong kiến trúc phần mềm”.

5.5 Nhận xét chung về các đóng góp nổi bật

Bốn giải pháp trên phản ánh bốn lát cắt khác nhau của cùng một bài toán xây dựng ứng dụng sự kiện. Giải pháp thứ nhất xử lý khác biệt nền tảng ở lớp biên; giải pháp thứ hai bảo đảm tính đúng đắn của luồng nghiệp vụ cốt lõi; giải pháp thứ ba tăng khả năng gắn kết và quay lại của người dùng thông qua thông báo; còn giải pháp thứ tư mở rộng hệ thống theo hướng thông minh nhưng vẫn giữ được tính ổn

định. Nếu chỉ nhìn riêng từng tính năng, các kết quả này có thể xem là các chức năng đơn lẻ. Nhưng khi nhìn dưới góc độ kỹ thuật, chúng cho thấy một định hướng nhất quán: mọi quyết định đều cố gắng đưa hệ thống tiến gần hơn tới mức có thể sử dụng thật, thay vì chỉ dừng ở mức minh họa ý tưởng.

Từ quá trình thực hiện các giải pháp này, tác giả rút ra một nhận xét quan trọng: ở đồ án theo hướng sản phẩm, giá trị không nằm ở chỗ dùng bao nhiêu công nghệ mới, mà nằm ở chỗ xử lý được bao nhiêu tình huống “xấu” của bài toán thực tế. Khả năng chạy được trên nhiều môi trường, khả năng chống sai lệch dữ liệu từ client, khả năng tránh gửi thông báo trùng, hay khả năng giữ cho tính năng AI không phá vỡ luồng nghiệp vụ chính, đều là những ví dụ cho tư duy đó. Đây cũng chính là nhóm đóng góp mà tác giả xem là nổi bật nhất của đồ án.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đồ án đã tập trung giải quyết bài toán đặt vé và quản lý sự kiện trên thiết bị di động theo hướng xây dựng một hệ thống tích hợp nhiều nghiệp vụ trong cùng một nền tảng. So với các sản phẩm định hướng cộng đồng như Meetup, hệ thống của đồ án chưa đặt trọng tâm vào quy mô cộng đồng rộng hoặc hệ sinh thái người dùng lớn, nhưng đã hướng nhiều hơn tới sự gắn kết giữa khâu khám phá sự kiện, đặt vé, lưu vé điện tử, nhận thông báo và kiểm soát tham dự [3]. So với các nghiên cứu học thuật về gợi ý sự kiện hoặc gợi ý địa điểm tổ chức, chẳng hạn các công trình về đề xuất sự kiện trong môi trường mạng xã hội và tối ưu lựa chọn cho nhóm người dùng [4]–[6], sản phẩm của đồ án không đi sâu vào mô hình đề xuất thông minh, nhưng lại có ưu điểm ở việc hiện thực hóa một luồng nghiệp vụ tương đối hoàn chỉnh dưới dạng ứng dụng có thể sử dụng thử nghiệm trên thiết bị di động. Nói cách khác, các công trình nghiên cứu tương tự mạnh về phân tích và tối ưu hóa quyết định, trong khi hệ thống của đồ án mạnh hơn ở khía cạnh tích hợp chức năng và hiện thực hóa sản phẩm.

Trong suốt quá trình thực hiện đồ án, kết quả đạt được nổi bật nhất là xây dựng được một hệ thống gồm ứng dụng di động và backend có khả năng vận hành các nghiệp vụ cốt lõi của bài toán. Cụ thể, hệ thống đã hỗ trợ đăng ký và đăng nhập người dùng, quản lý hồ sơ, tìm kiếm và lọc sự kiện, xem chi tiết sự kiện, lưu sự kiện yêu thích, tạo sự kiện, quản lý hạng vé, đặt vé, xác nhận thanh toán, phát hành vé điện tử, sinh mã QR cho vé, check-in tại sự kiện, nhắn tin, gửi lời mời, gửi thông báo và nhắc lịch tự động. Ngoài ra, hệ thống còn có cơ chế phân quyền giữa người dùng, người tổ chức và quản trị viên, đồng thời tích hợp chức năng gợi ý nội dung sự kiện bằng AI để hỗ trợ người tổ chức khi tạo sự kiện mới. Những kết quả này cho thấy mục tiêu chính của đề tài là xây dựng một ứng dụng đặt vé sự kiện mobile-first có tính tích hợp đã được đáp ứng ở mức khả thi.

Bên cạnh các kết quả đạt được, đồ án vẫn còn một số hạn chế. Thứ nhất, hệ thống mới dừng ở mức thử nghiệm và chưa tích hợp đầy đủ cổng thanh toán thực tế ở môi trường triển khai thương mại. Thứ hai, các chức năng gợi ý thông minh mới chỉ dừng ở hỗ trợ tạo nội dung sự kiện, chưa phát triển thành cơ chế đề xuất sự kiện cá nhân hóa cho người dùng cuối. Thứ ba, hệ thống chưa đi sâu vào các vấn đề triển khai quy mô lớn như cân bằng tải, giám sát hệ thống, tối ưu hiệu năng cho lượng truy cập rất cao hoặc thống kê phân tích hành vi người dùng trên quy mô thực tế. Ngoài ra, việc kiểm thử hiện nay mới phù hợp ở mức chức năng chính,

chưa bao phủ đầy đủ các tình huống kiểm thử tự động và các bài toán vận hành ngoài thực địa.

Qua quá trình thực hiện đồ án, có thể rút ra một số bài học kinh nghiệm quan trọng. Việc phân tách rõ vai trò giữa frontend di động, backend nghiệp vụ và cơ sở dữ liệu giúp cho quá trình phát triển và bảo trì thuận lợi hơn. Bên cạnh đó, việc tổ chức hệ thống theo các mô-đun như xác thực, sự kiện, vé, thanh toán, chat và thông báo cho thấy hiệu quả rõ rệt khi cần bổ sung hoặc điều chỉnh chức năng. Một kinh nghiệm khác là với các bài toán có nhiều vai trò tham gia như người dùng, người tổ chức và quản trị viên, việc mô hình hóa use case và quy trình nghiệp vụ từ sớm có ý nghĩa rất quan trọng trong việc tránh thiếu sót chức năng và giảm xung đột khi triển khai. Cuối cùng, đồ án cho thấy rằng một hệ thống sự kiện hiện đại không chỉ cần giao diện đẹp hay quy trình mua vé đơn giản, mà còn cần tính liên kết chặt chẽ giữa các nghiệp vụ để đem lại trải nghiệm xuyên suốt cho người dùng.

6.2 Hướng phát triển

Trong giai đoạn tiếp theo, trước hết cần tiếp tục hoàn thiện các chức năng đã được xây dựng trong đồ án để hệ thống có thể tiến gần hơn tới mức sẵn sàng triển khai thực tế. Một số công việc cần ưu tiên gồm tích hợp cổng thanh toán thật thay cho luồng xác nhận thanh toán mô phỏng, tăng cường cơ chế xử lý lỗi và xác thực đầu vào ở các API quan trọng, hoàn thiện trải nghiệm giao diện trên nhiều kích thước thiết bị, và nâng cao độ ổn định của các chức năng thông báo đẩy, nhắc lịch và chat thời gian thực. Bên cạnh đó, cần bổ sung kiểm thử tự động cho các nghiệp vụ trọng yếu như tạo sự kiện, đặt vé, thanh toán và check-in để giảm rủi ro khi mở rộng hệ thống. Việc tối ưu hóa phần tìm kiếm, lọc sự kiện, quản lý hình ảnh sự kiện và đồng bộ dữ liệu giữa ứng dụng khách với máy chủ cũng là những hạng mục cần thiết để sản phẩm hoạt động tin cậy hơn trong môi trường thực tế.

Sau khi các chức năng hiện tại được hoàn thiện, hệ thống có thể được mở rộng theo nhiều hướng mới nhằm nâng cao chất lượng và giá trị sử dụng. Hướng thứ nhất là phát triển cơ chế gợi ý sự kiện cá nhân hóa dựa trên lịch sử tương tác, sở thích, vị trí và hành vi tham gia của người dùng, từ đó đưa hệ thống tiến gần hơn tới các hướng nghiên cứu đã được đề cập trong các công trình học thuật. Hướng thứ hai là mở rộng phần quản trị và phân tích dữ liệu, chẳng hạn bổ sung dashboard thống kê cho người tổ chức và quản trị viên để theo dõi lượt quan tâm, số vé đã bán, tỷ lệ check-in, phản hồi của người dùng và hiệu quả của từng sự kiện. Hướng thứ ba là mở rộng khả năng triển khai bằng cách đưa hệ thống lên hạ tầng đám mây, hỗ trợ lưu trữ ảnh tập trung, tăng khả năng mở rộng theo số lượng người dùng và bổ sung cơ chế sao lưu, giám sát, ghi log tập trung. Cuối cùng, hệ thống có thể tiếp

tục được phát triển theo hướng đa nền tảng và đa dịch vụ, ví dụ hỗ trợ thêm web quản trị, tích hợp bản đồ và định vị sâu hơn, hỗ trợ đa ngôn ngữ, đăng nhập liên thông hoặc mở rộng sang các mô hình sự kiện có sơ đồ chỗ ngồi, bán vé theo khu vực và quản lý chương trình chi tiết theo thời gian.

TÀI LIỆU THAM KHẢO

- [1] S. Kemp, *Digital 2026: Global overview report*, DataReportal, Published in October 2025 for the Digital 2026 series, 2025. [Online]. Available: <https://datareportal.com/reports/digital-2026-global-overview-report> (visited on 06/22/2026).
- [2] S. Kemp, *Digital 2026: Vietnam*, DataReportal, Published in late 2025 for the Digital 2026 series, 2025. [Online]. Available: <https://datareportal.com/reports/digital-2026-vietnam> (visited on 06/22/2026).
- [3] Meetup, *About meetup*, 2026. [Online]. Available: <https://www.meetup.com/about/> (visited on 06/22/2026).
- [4] F. Cena, S. Likavec, I. Lombardi, and C. Picardi, *Should I stay or should I go? improving event recommendation in the social web*, 2014. arXiv: 1407.0153 [cs.IR]. [Online]. Available: <https://arxiv.org/abs/1407.0153>.
- [5] G. Liao, X. Huang, N. N. Xiong, and C. Wan, *An intelligent group event recommendation system in social networks*, 2020. arXiv: 2006.08893 [cs.SI]. [Online]. Available: <https://arxiv.org/abs/2006.08893>.
- [6] J. S. Zhang, M. Gartrell, R. Han, Q. Lv, and S. Mishra, *GEVR: An event venue recommendation system for groups of mobile users*, 2019. arXiv: 1903.10512 [cs.SI]. [Online]. Available: <https://arxiv.org/abs/1903.10512>.
- [7] M. Jones, J. Bradley, and N. Sakimura, *Json web token (jwt)*, RFC 7519, 2015. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519> (visited on 06/22/2026).